Introducción a System Design

Diseño de sistemas

8.20 - 9.30 am



Gonzalo Martinez, 2025

Contenido hoy

- El miércoles dedicaré parte de la clase para hablar del proyecto y la primera entrega.
- Hoy es el inicio de la materia de System Design, nos centraremos en:
- **Disponibilidad**
- **Escalabilidad**

Objetivo de la unidad

¿Qué es?

- Aprender a diseñar un sistema en producción y eficiente en costos desde el primer momento
- Cada decisión que tomemos abarcará tematicas de:
  - **Product Owner**
  - **Arquitecto (Architect)**
  - **Ingeniero (Engineer)**
- System Design es:
  - Decidir **arquitectura**
  - Decidir **componentes**
  - Decidir **módulos**
- **Cada componente lo pensaremos desde un punto de vista práctico a nivel de ingeniería**
- **Cada decisión tiene una ventaja/desventaja (tradeoff)**

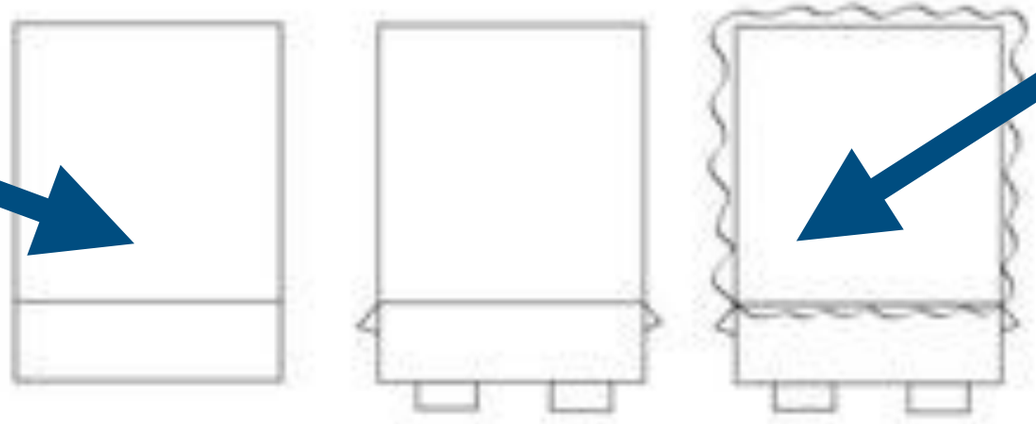
Cada clase se presentará un problema real de negocio y en conjunto diseñamos la solución

Parte del contenido escrito es en inglés

System Design is like sketching

Broad stroke

Day 0 solution



Evolve

Find bottlenecks for each component and rearchitect

Comenzamos con un diseño simple y cada clase va aumentando la complejidad del mismo

Finer details

Go into implementation details

Performance optimizations & Cost efficiency

Antes de diseñar es importante

ENTENDER EL CONTEXTO

- When is the best **time to market**? **(MUY IMPORTANTE)**
  - **The business, not the architect, decides the project timing.**
  - Time can be our enemy because, often, even skills and money can’t change the timeline of the product.
  - **El curso tiene un inicio y finalización fijado con anterioridad**
- What is the **skill level of the team**? **(MUY IMPORTANTE)**
  - **Nuestro equipo no es el que queremos, es el que tenemos***

Antes de diseñar es importante

ENTENDER EL CONTEXTO

- What is our **system's performance sensitivity**? (**MUY IMPORTANTE**)
  - **Posiblemente no necesiten tanto performance*.**
  - Ejemplo: Spring Boot (Java) framework opensource estándar puede soportar como mínimo cientos de request por segundo. Asumiendo que como mínimo podría manejar 50 req/seg eso da 4.32 millones de req/dia.
- Lo anterior es suficiente para una primera versión de un producto. Luego, si aumentan los números de request eso implica que el producto es más usado, en consecuencia más exito lo que se traduce en más dinero.
- **Más dinero puede costear una v2, o una v3 del sistema.**

Contexto

Dominio del negocio

Servicios financieros	**Gobierno**
Seguros	**Transporte**
Salud	**Viajes y Turismo**
Ciencias Biológicas	**Automotriz**
Energía	**Retail**

Necesidad del negocio

Somos un negocio

Incrementar los ingresos	Proteger los ingresos	Reducir los costos	Evitar nuevos costos
Son los cambios que realizan los clientes para aumentar las ventas a clientes nuevos o existentes. Esto incluye deleitar a los clientes o intervenir para **aumentar la cuota de mercado**	Estos cambios incluyen mejoras e innovaciones incrementales para **mantener la cuota de mercado** y los ingresos actuales	Estos cambios sirven para **reducir los costos** actuales	Se trata de cambios para **evitar costos** que nuestros clientes saben que podrían afectarles

Ejemplos KPI

Métrica cuantitativa - Evaluar progreso objetivo empresarial

RETAIL	Sales Growth Rate	La tasa de crecimiento de las ventas es un indicador que mide la variación porcentual de las ventas de un minorista en un periodo de tiempo determinado.
RETAIL	Gross Margin	El margen bruto es un indicador que mide el porcentaje de los ingresos por ventas que el minorista obtiene como beneficio antes de deducir el coste de las mercancías vendidas.
RETAIL	Inventory Turnover	La rotación de existencias es un indicador que mide la frecuencia con la que se venden y reponen las existencias de un minorista en un periodo de tiempo determinado.
RETAIL	Customer Retention Rate	La tasa de retención de clientes es un indicador que mide el porcentaje de clientes que siguen haciendo negocios con un minorista durante un periodo de tiempo determinado.
RETAIL	Average Transaction Value (ATV)	El valor medio de transacción puede utilizarse para evaluar la eficacia del marketing y las ventas de un minorista.
FINANCIAL SERVICES	Return on Equity (ROE)	El rendimiento de los fondos propios es la capacidad de una empresa de generar valor para el negocio y para los inversores a partir de los recursos propios de la empresa.
FINANCIAL SERVICES	Return on Assets (ROA)	El rendimiento de los activos indica qué parte del valor que la empresa ha invertido en la operación ha retornado al negocio en forma de beneficios. Un ROA elevado indica que la empresa utiliza sus activos de forma eficiente para generar beneficios.
FINANCIAL SERVICES	Net Interest Margin (NIM)	El margen de interés neto mide la diferencia entre los ingresos por intereses que un banco obtiene de sus préstamos y los intereses que paga por sus depósitos.
FINANCIAL SERVICES	Asset Quality Ratio	El ratio de calidad de los activos es la cantidad de préstamos de un banco que se consideran morosos. Un AQR alto indica que el banco tiene un bajo nivel de préstamos dudosos.
FINANCIAL SERVICES	Debt-to-Equity Ratio (D/E)	El ratio de deuda sobre fondos propios es la cantidad de deuda que tiene una empresa en relación con sus fondos propios. Un ratio D/E elevado indica que la empresa está muy apalancada.
FINANCIAL SERVICES	Cost per Account Opened	Coste medio en que incurre un banco para abrir una nueva cuenta. Un coste elevado por cuenta abierta puede indicar que el banco no es eficiente en sus actividades de marketing y ventas.

Estrategia para diseñar

- Las interacciones del usuario con el sistema le llamamos:
    - **User's Journey**.
- Cada decisión de diseño va a afectar al usuario final (impacto de negocio), por lo que todo tenemos que diseñar pensando en el negocio.
- Un sistema tiene como mínimo tres tipos de usuarios:
    - **Cliente Final**
    - **Ingenieros que lo construyen**
    - **El que financia la inversión (empresa)**
- Si afecta al usuario afecta al negocio.

Recordar que a nivel de ingeniería:

Business > Product > Engineering

Cada decisión es un tradeoff
Ganas algo y pierdes algo

Sean pesimistas, asuman que cualquier componente puede fallar.
La realidad no es determinsta

El objetivo de esta unidad es que adopten una mentalidad de ingeniería para resolver distintos aspectos del proyecto

Resumiendo todo lo que han visto en la carrera -de forma práctica-

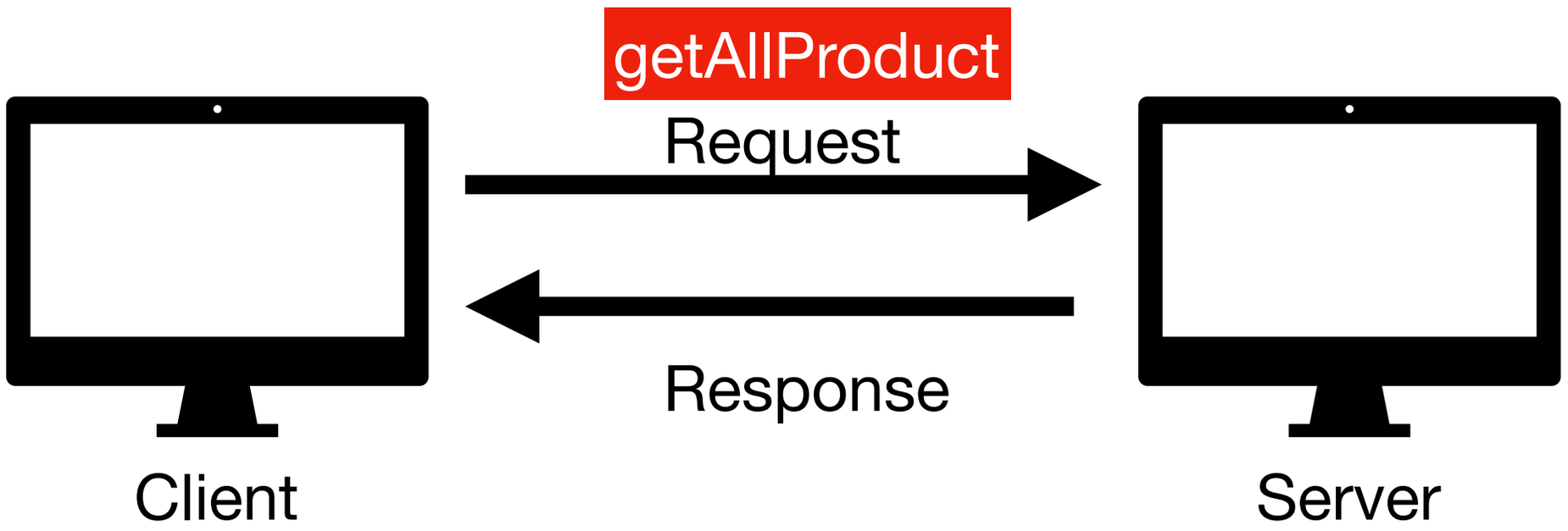
Problema:

Listar productos de un ecommerce

Listar productos ecommerce

Para una tienda pequeña (100 items)

1. Agregar un nuevo producto
2. Actualizar/borrar producto existente
3. Listar todos los productos (cátalogo)
4. Cliente debe acceder rápidamente al cátalogo
5. Compra y venta queda fuera de este problema



Problema: listar productos ecommerce

Métricas de negocio a priorizar

Sales Growth Rate	**La tasa de crecimiento de las ventas es un indicador que mide la variación porcentual de las ventas de un minorista en un periodo de tiempo determinado.**
Customer Retention Rate	**La tasa de retención de clientes es un indicador que mide el porcentaje de clientes que siguen haciendo negocios con un minorista durante un periodo de tiempo determinado.**

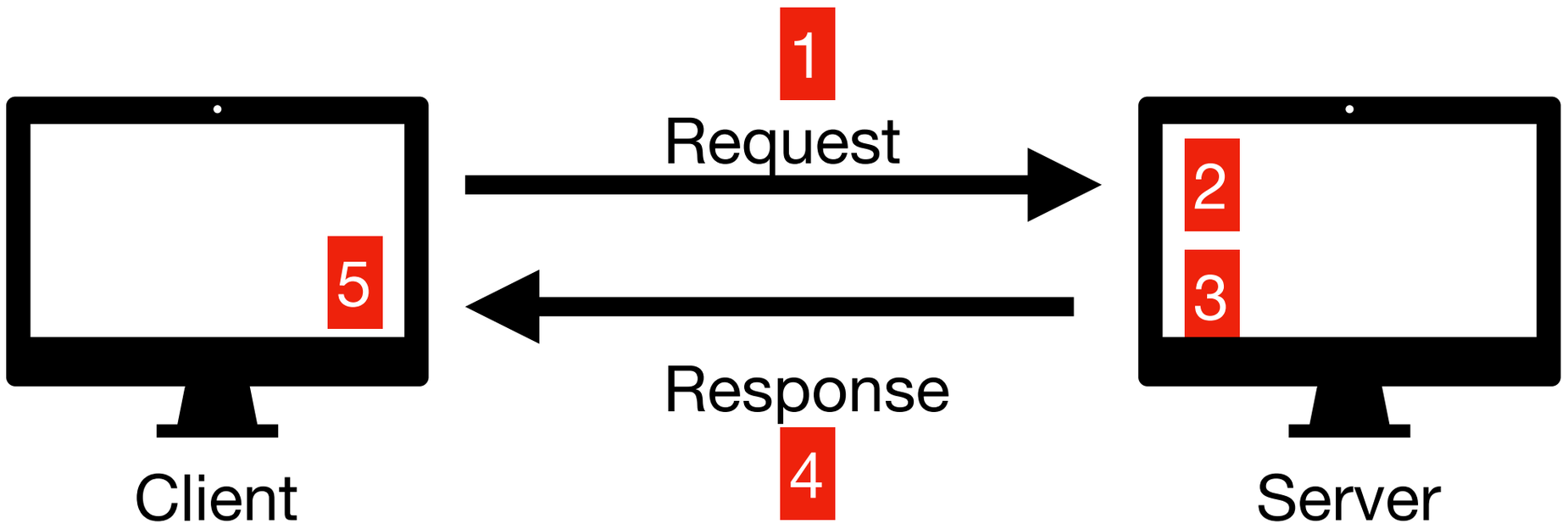
Subproblema 1:
Comunicación entre máquinas

¿Yo me conecto al ecommerce o el ecommerce se conecta a mi?

Modelo Request - Response

Forma más común de comunicación entre máquinas

1. Cliente envía una petición (**request**) al servidor.
2. Servidor decodifica esa petición.
3. Servidor procesa la petición.
4. Servidor envía una respuesta (**response**).
5. Cliente decodifica la respuesta.



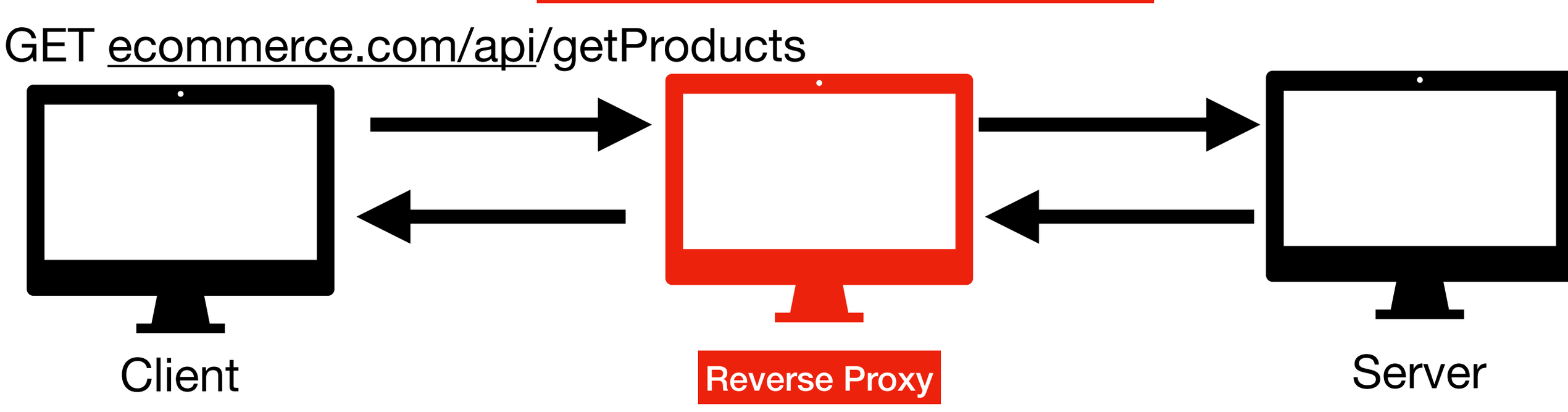
Yo (cliente) me conecto al ecommerce (servidor)

Comunicación entre máquinas

Request - Response: Reverse Proxy

1. El cliente no se conecta directamente al servidor (ejemplo: 192.168.0.10) si no que a través de una dirección (ecommerce.com/api/getProducts). El componente que realiza la redirección es: Reverse Proxy

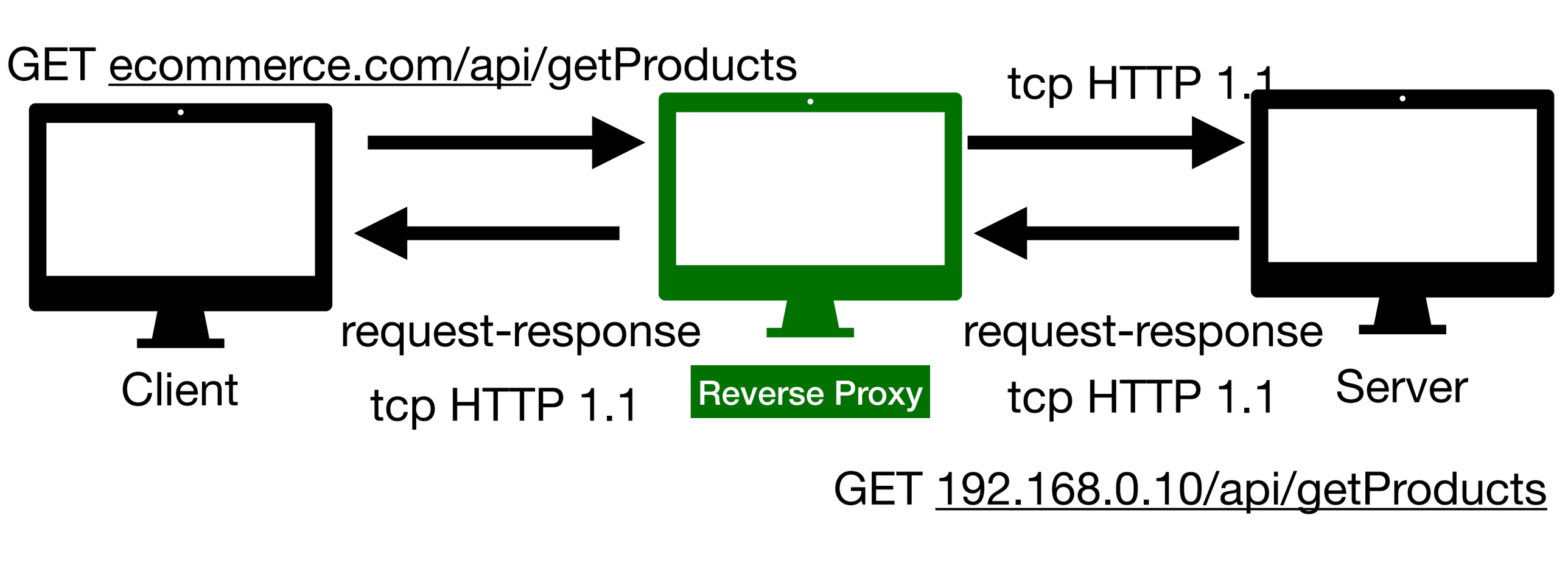
¿Cómo lo implementarían?



Subproblema 1:
Comunicación entre máquinas

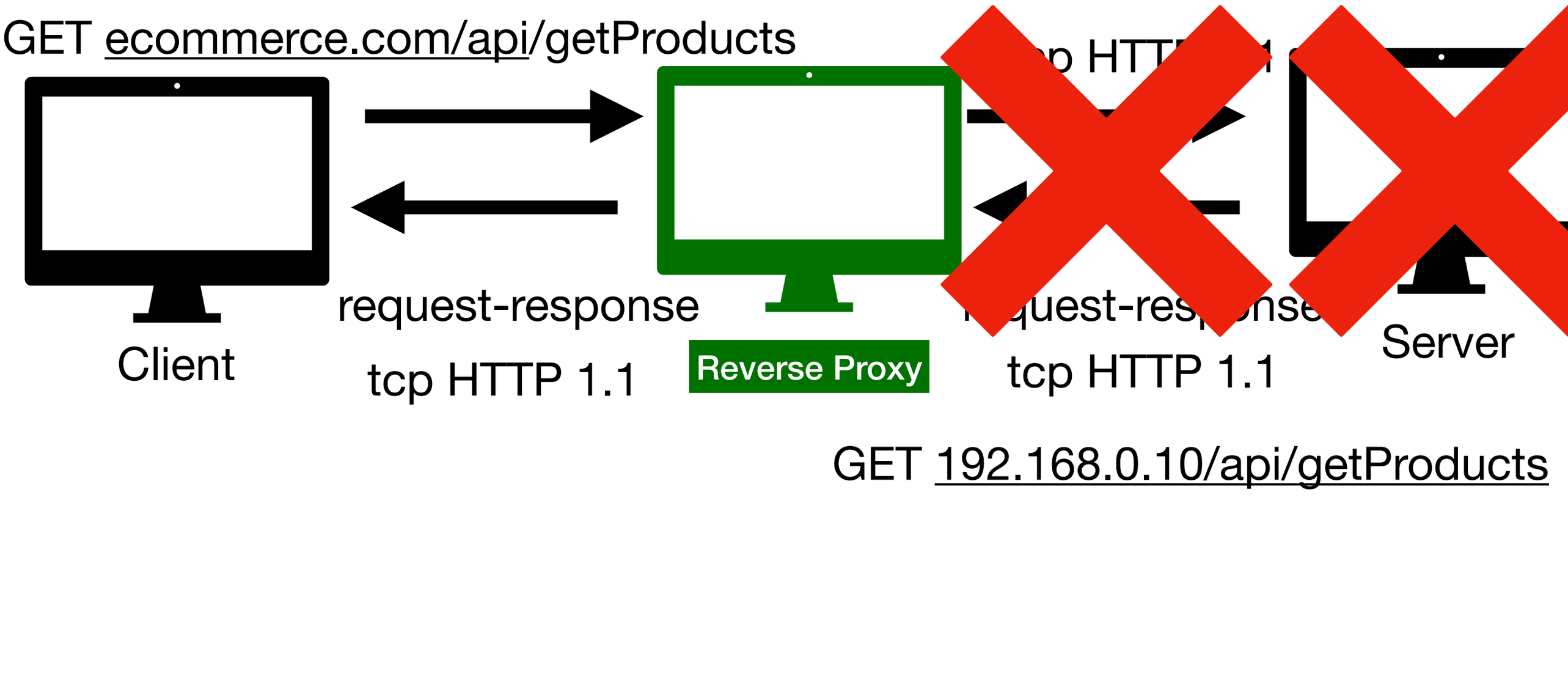
SOLUCIONADO

Concepto: reverse proxy



Subproblema 2: Disponibilidad de Servidor

Se cae servidor backend



Disponibilidad (Availability)

Uber Eats Apps - Impacto en negocio

- **Time: Outage on Friday evening 6 PM - 8 PM**
- **Root Cause: New config deployed**
- **Consecuency: No new orders, no order finalization**
- **Missed Revenue Calculation**:
  - 120 orders/minute * $25 = $3,000 per minute
  - **Total Downtime: 120 minutes**
  - **Missed Revenue: $3,000 * 120 = $360,000**
- Compensation to Restaurant Partners:
  - 120 orders/minute * 120 minutes = 14,400 orders
  - **14,400 orders * $12.5 = $180,000**

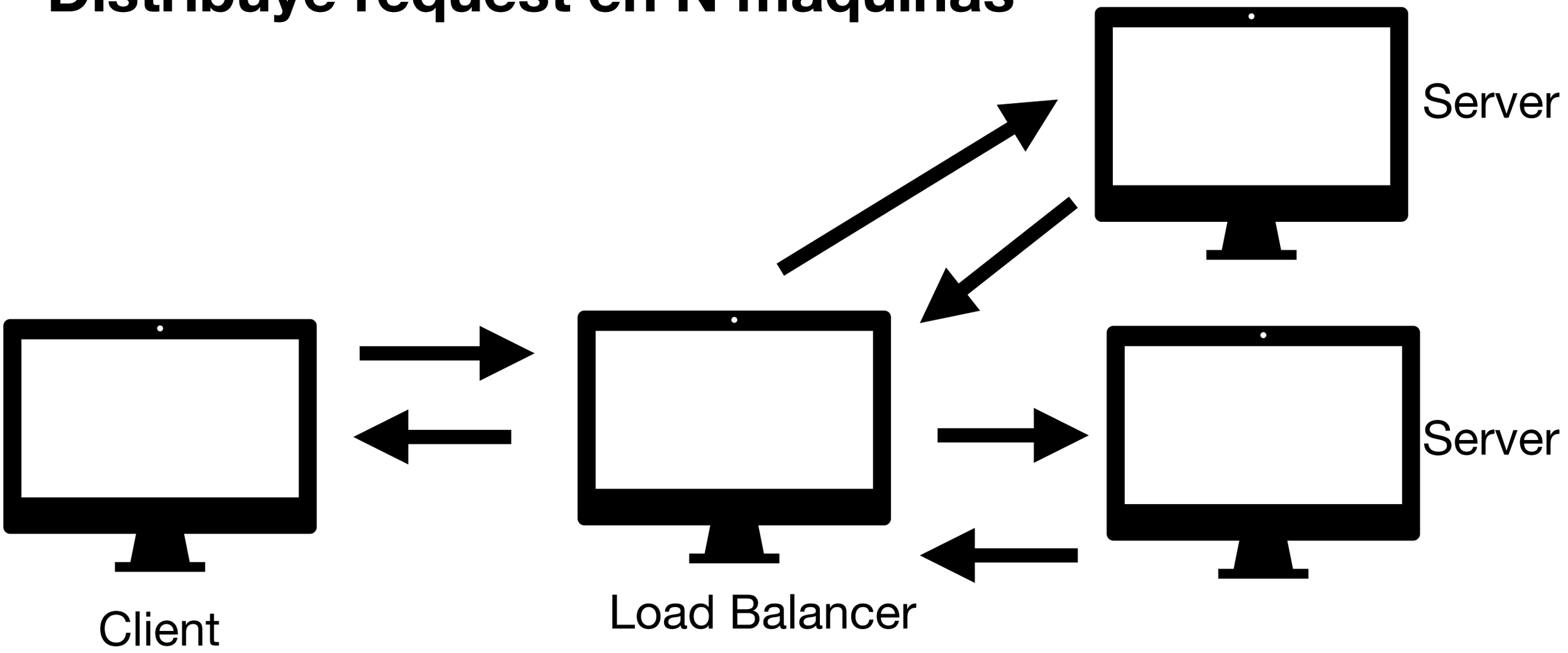
Disponibilidad (Availability)

Uber Eats Apps - Impacto en presupuesto

- **Time: Outage on Friday evening 6 PM - 8 PM**
- **Root Cause: New config deployed**
- **Consecuency: No new orders, no order finalization**
- **Future Revenue Loss**:
  - 5% decline in new customer acquisition over the following quarter
  - Assuming $2 mil per quarter
  - 0,05 * $2,000,000 = $100,000
- **ENGINEERING TIME: On call, postmortem, etc: ~$5000**
- **Total Impact: $645,000**

Solución: Replicas y Load Balancer

Distribuye request en N máquinas

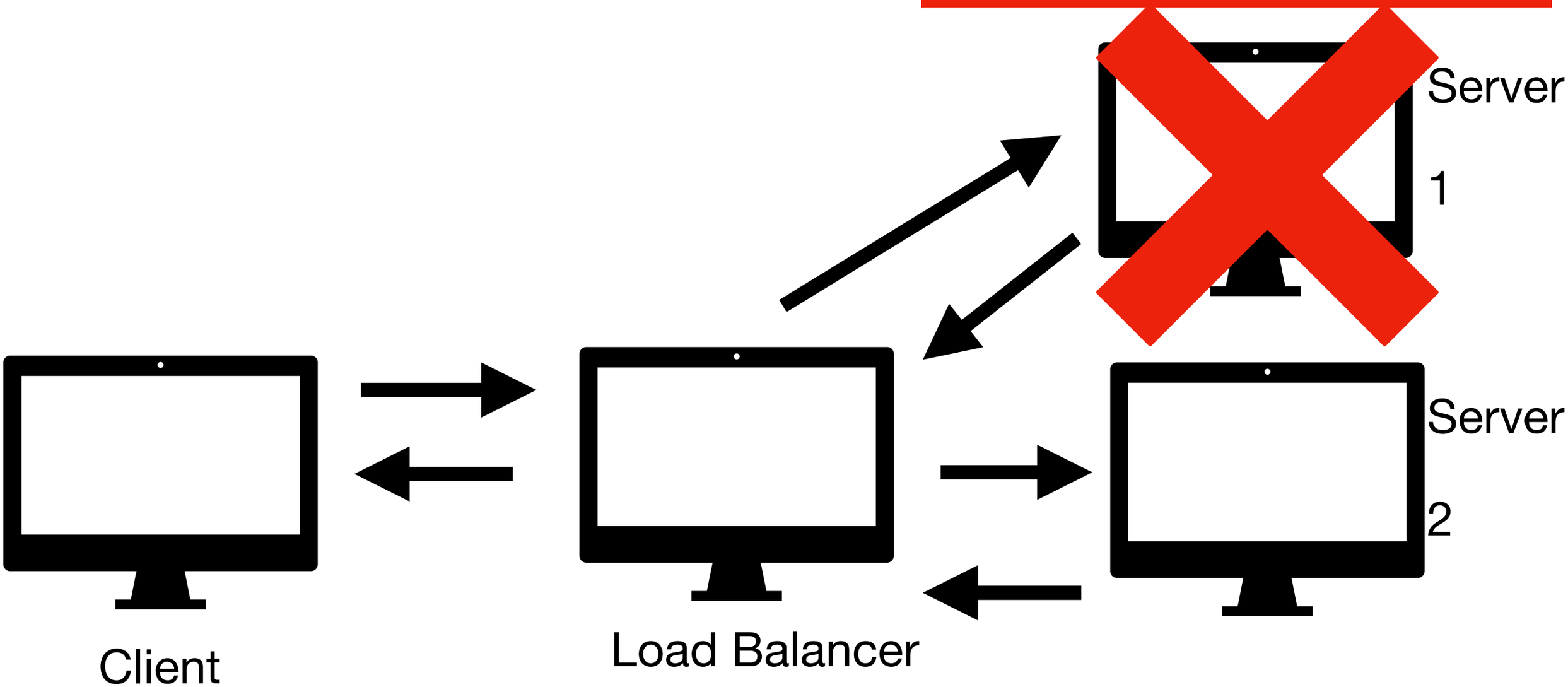


Recuerden: un load balancer es un proxy server pero no necesariamente un proxy server será un load balancer

¿Cómo lo implementarían?

Load Balancer

Si Server1 cae entonces usar Server2



Problema: ¿Cuál es la probabilidad de que caiga Server1?

¿Qué pasa si no se cae en 1 mes?
Estoy pagando una máquina sin usar

Load Balancer

¿Cómo realizarían la distribución de request?

1. Es necesario implementar lógica de ruteo (routing logic)

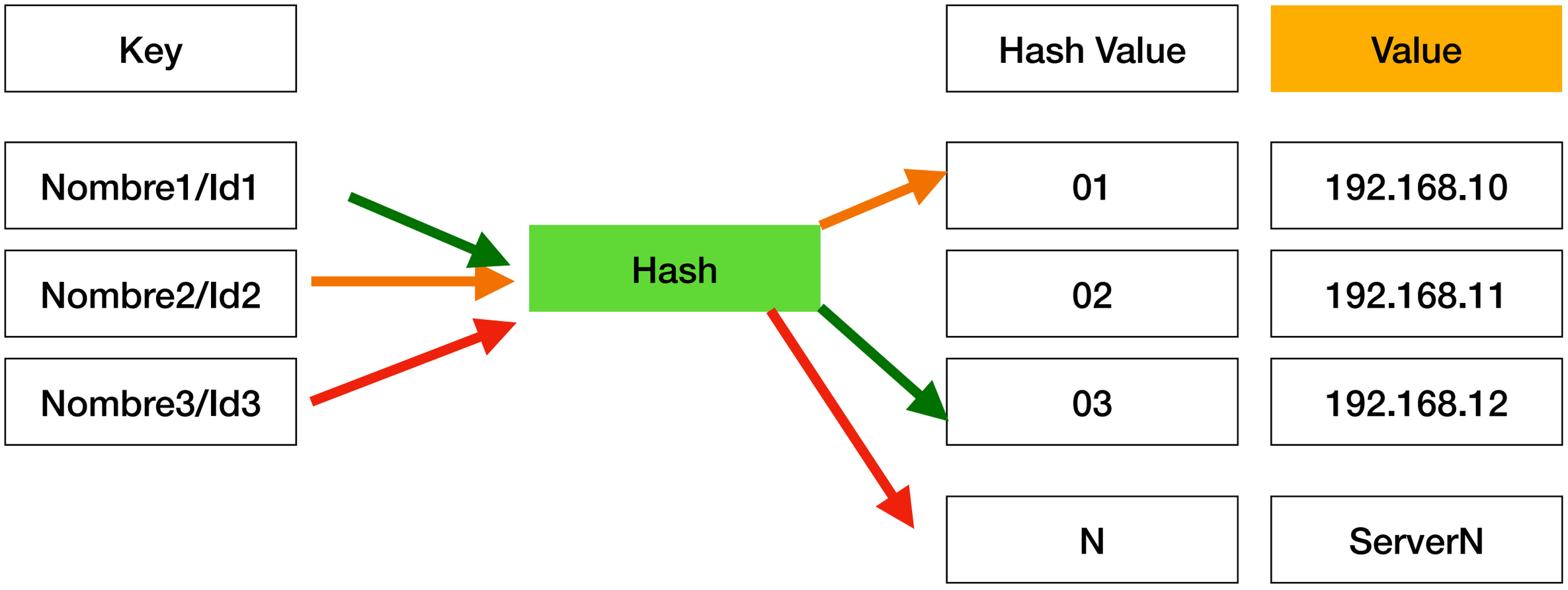
2. Piensen en:

    1. **Eficiencia de costos operacionales** (costo de mantención de máquinas, pago a proveedores, tiempo a nivel de ingeniería).

    2. **Máximizar uso de recursos por los cuáles pagamos.**

Routing

Hash based router



¿Cómo lo implementarían?

¿Tradeoffs?

Routing

Range based router

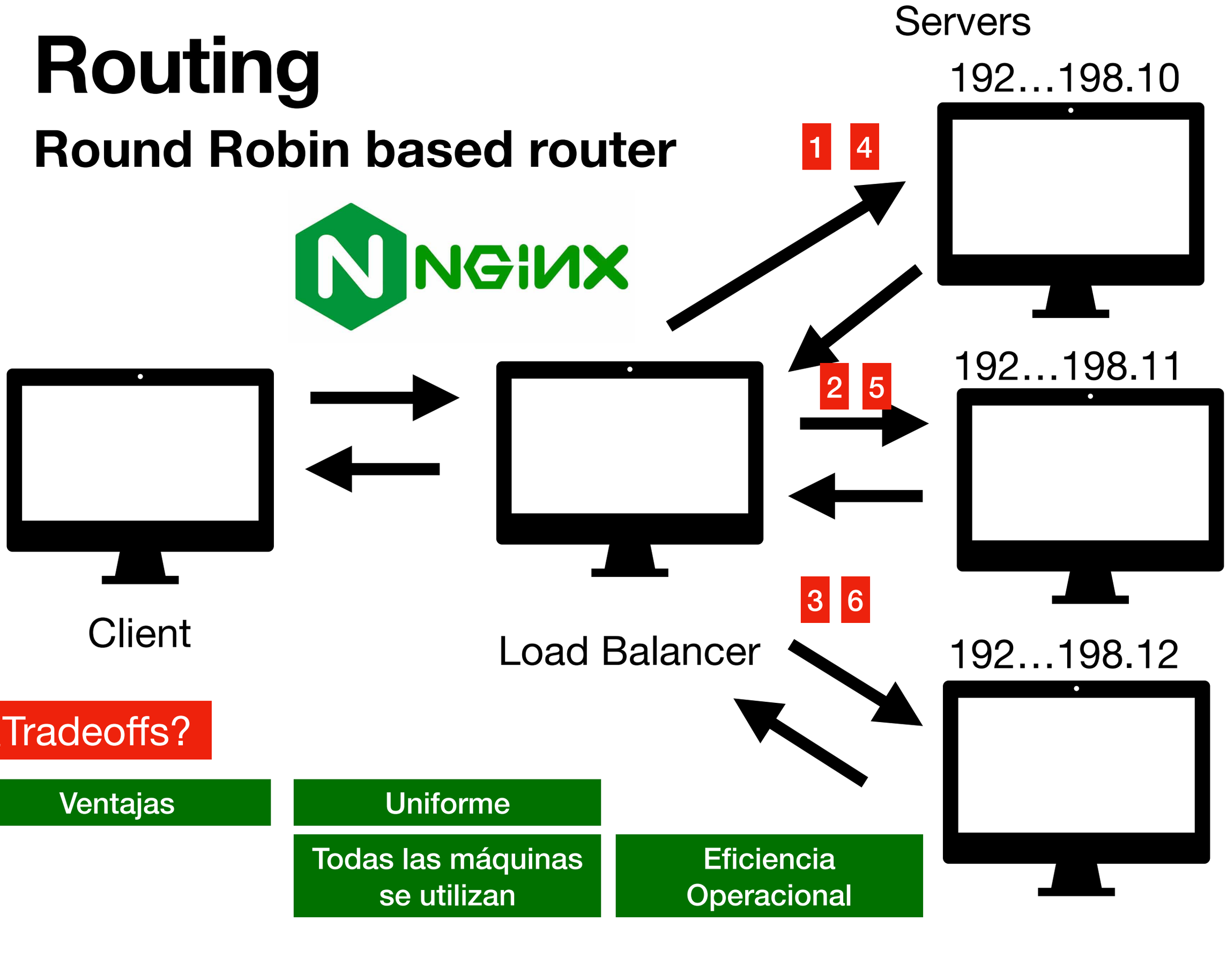
Value	Value Range	Value
Nombre1/Id1	0-10	192.168.10
Nombre20/Id20	11-20	192.168.11
Nombre31/Id31	20-30	192.168.12
	N - N+1	ServerN

¿Cómo lo implementarían?

¿Tradeoffs?

Routing

Round Robin based router



¿Tradeoffs?

Ventajas

Uniforme

Todas las máquinas se utilizan

Eficiencia Operacional

Routing

NGINX - Round Robin

```
/etc/nginx/nginx.conf

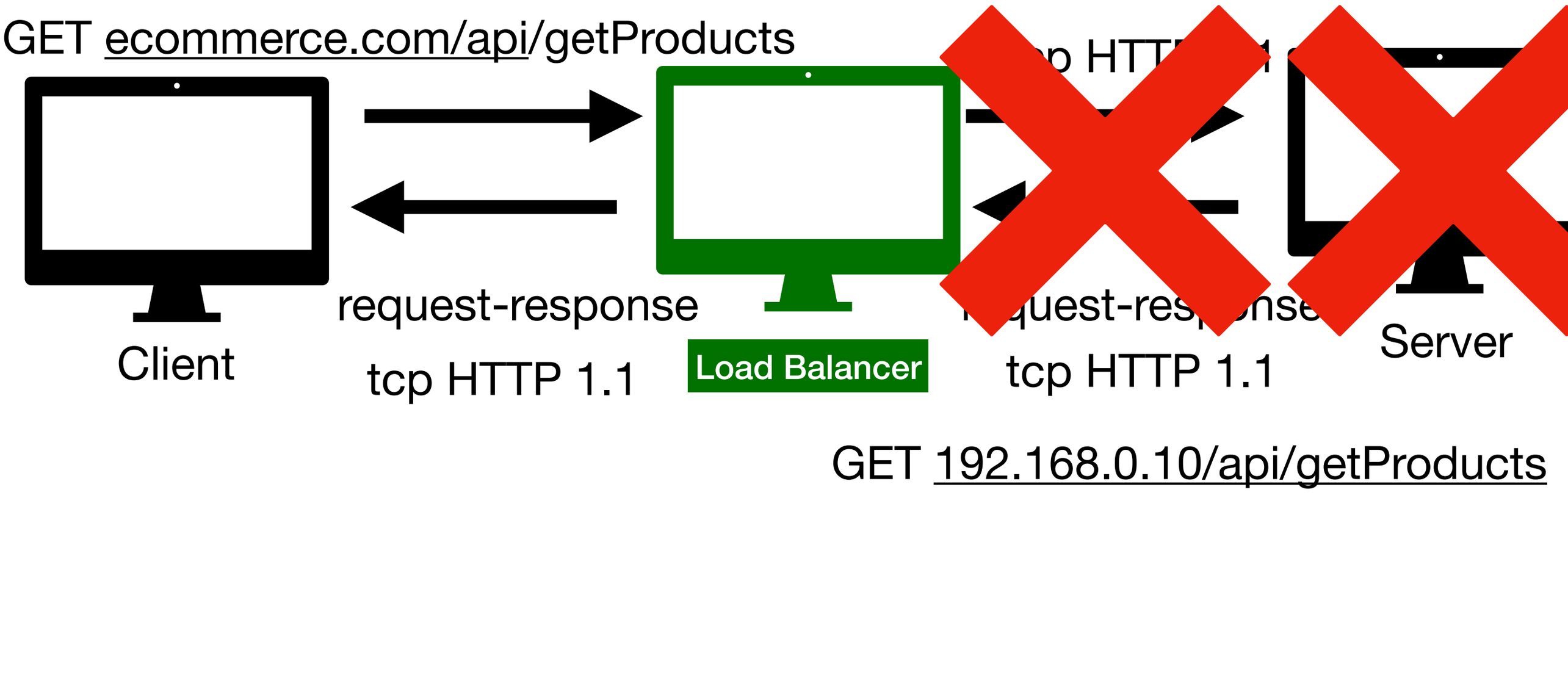
http {
    upstream backend_servers {
        # List your backend servers here (IP addresses or domain names)
        server 192.168.1.2:80;
        server 192.168.1.3:80;
        server 192.168.1.4:80;
        # Round robin is used by default
    }

    server {
        listen 80;
        server_name ecommerce.com;

        location /api/getProducts {
            proxy_pass http://backend_servers;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
    }
}
```

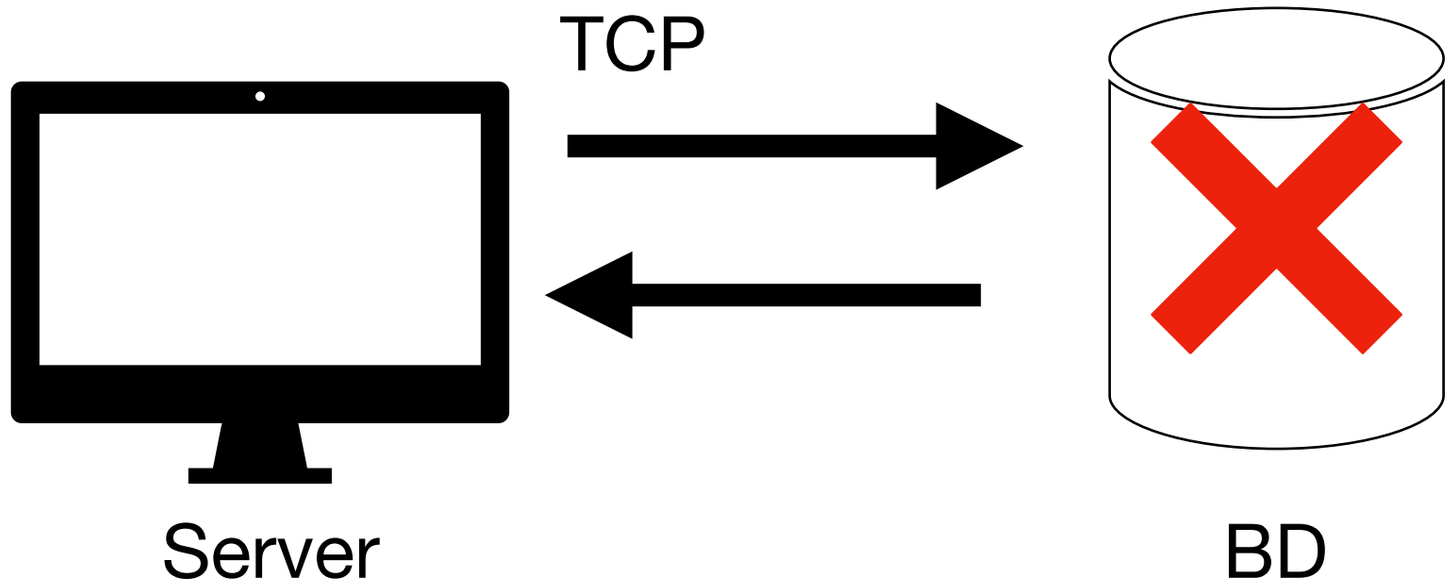
Subproblema 2: Disponibilidad de Servidor

SOLUCIONADO



Subproblema 2.1: Disponibilidad de Base de Datos

Base de Datos caida



Si la BD no esta disponible el servicio no puede operar
Si servicio no puede operar no hay negocio.

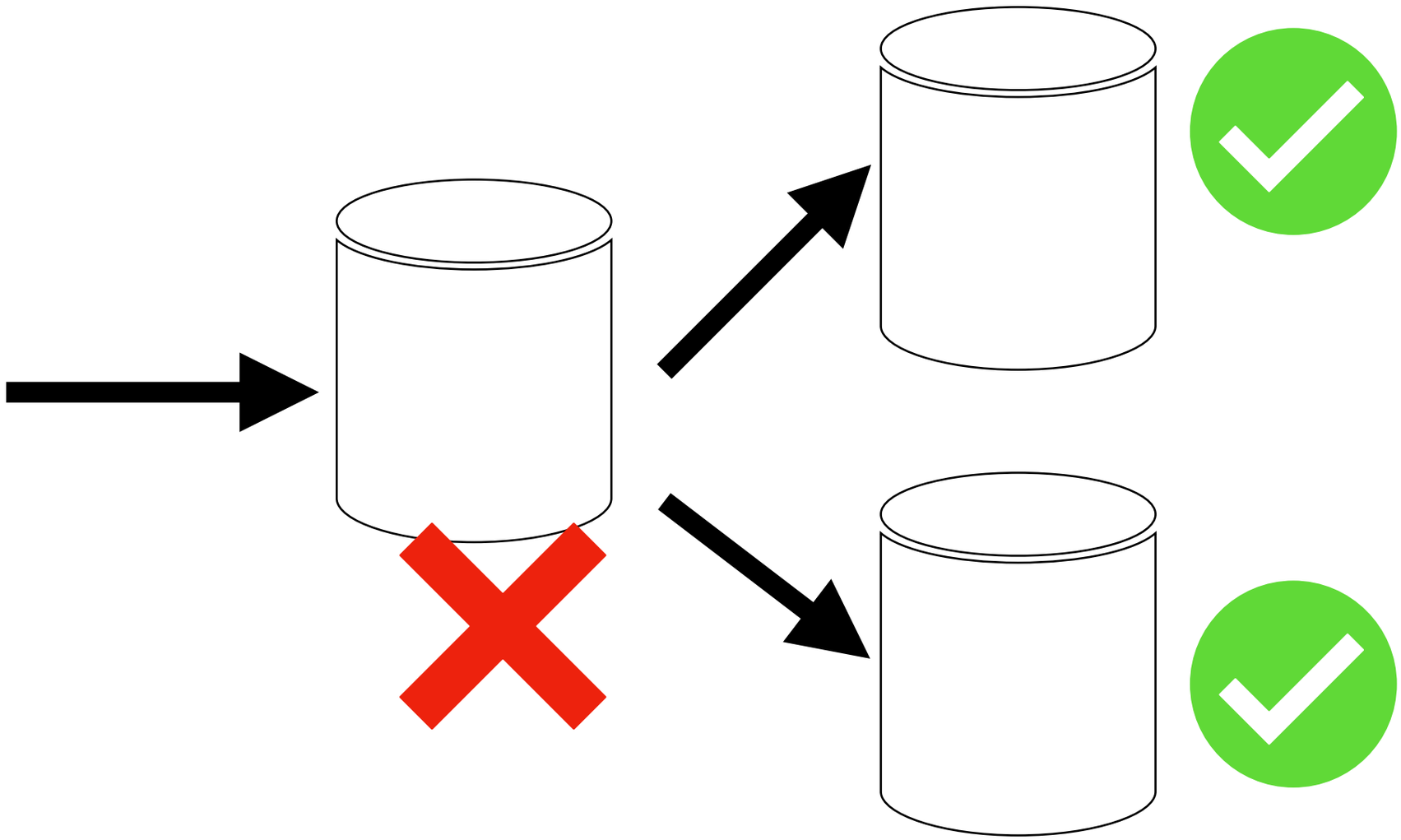
Base de datos

¿Qué sucede si la BD falla?

- Hay dos opciones:
  - **Usar Replicas de la BD**
    - La síncronización entre ellas puede ser síncrona o asíncrona.
  - **Usar Shards**
    - Separar BDs por rangos -recuerden lo que vimos en Load Balancer-

Base de datos - Replicas

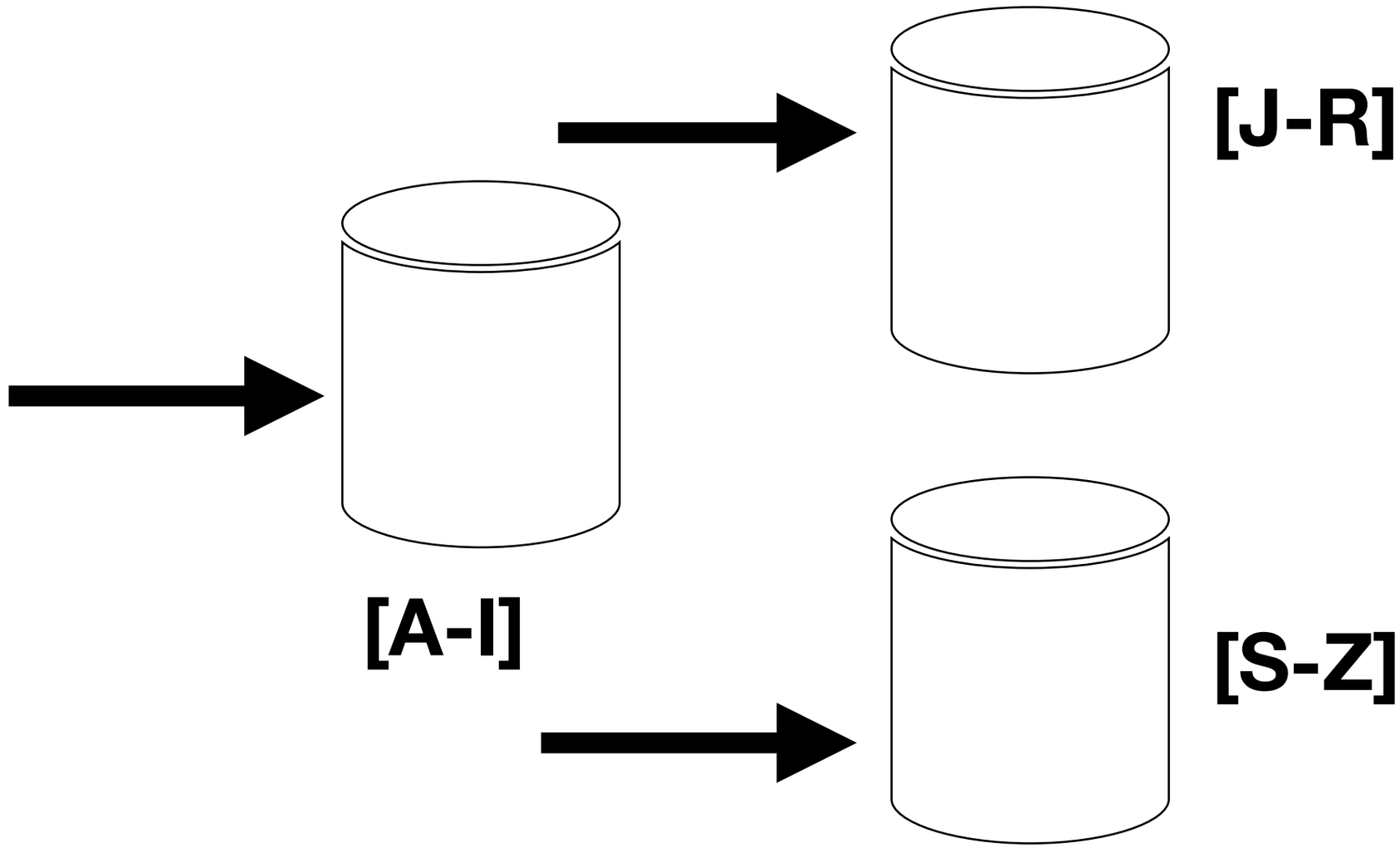
¿Qué sucede si la BD falla?



- Replica la data en n nodos de bases de datos.
- La coordinación puede ser síncrona (sync) o asíncrona (async) - Recordar lo que vimos cuando construimos un load balancer-.

Base de datos - Shard

¿Qué sucede si la BD falla?



- Separar la data en n nodos de bases de datos.

- La separación se puede realizar con RANGOS o HASH BASED -Recordar lo que vimos cuando construimos un load balancer-.

Base de datos - Replicas

Configuración PostgreSQL

```
postgresql.conf

# Enable sufficient WAL information for replication
wal_level = replica

# Allow a number of concurrent WAL sender processes (adjust as needed)
max_wal_senders = 5

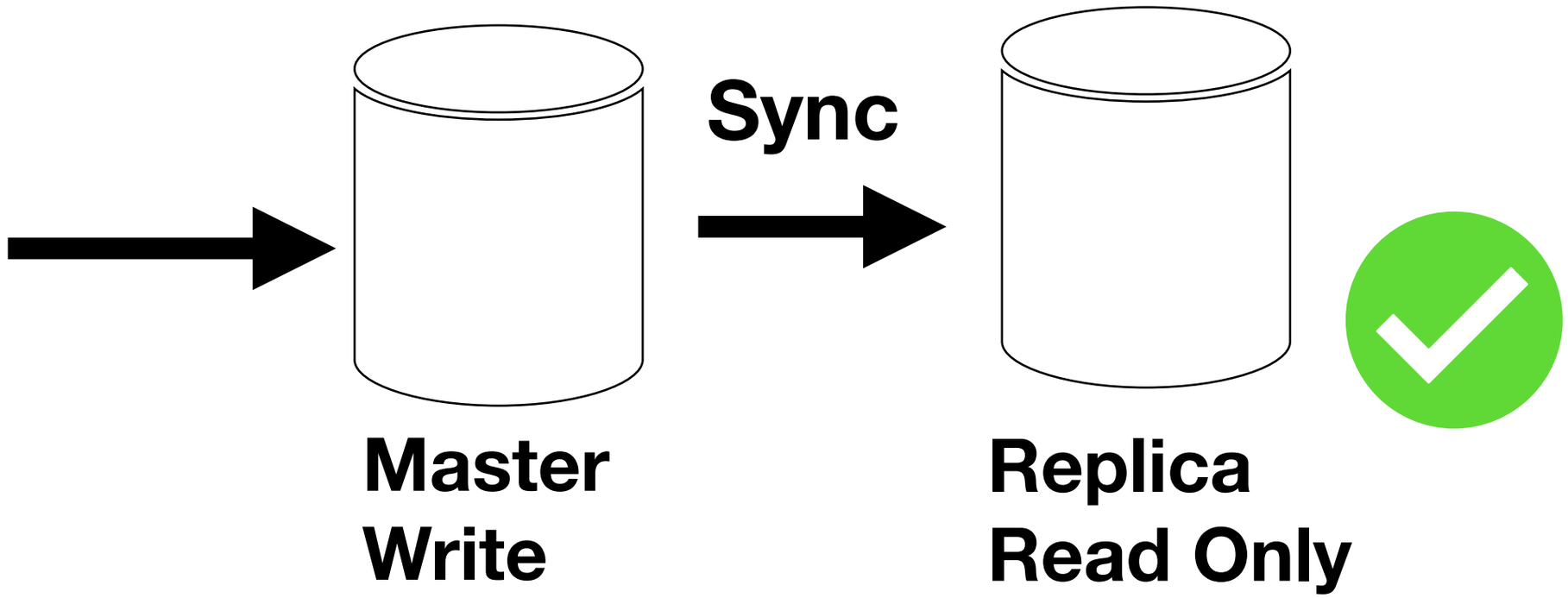
# Optional: Set the size/number of WAL files to retain for replication
wal_keep_size = 512MB  # or use wal_keep_segments on older versions

pg_hba.conf

host replication replicator [replica_ip]/32 md5
```

Base de datos - Replicas

¿Cuál es el camino crítico? Lecturas

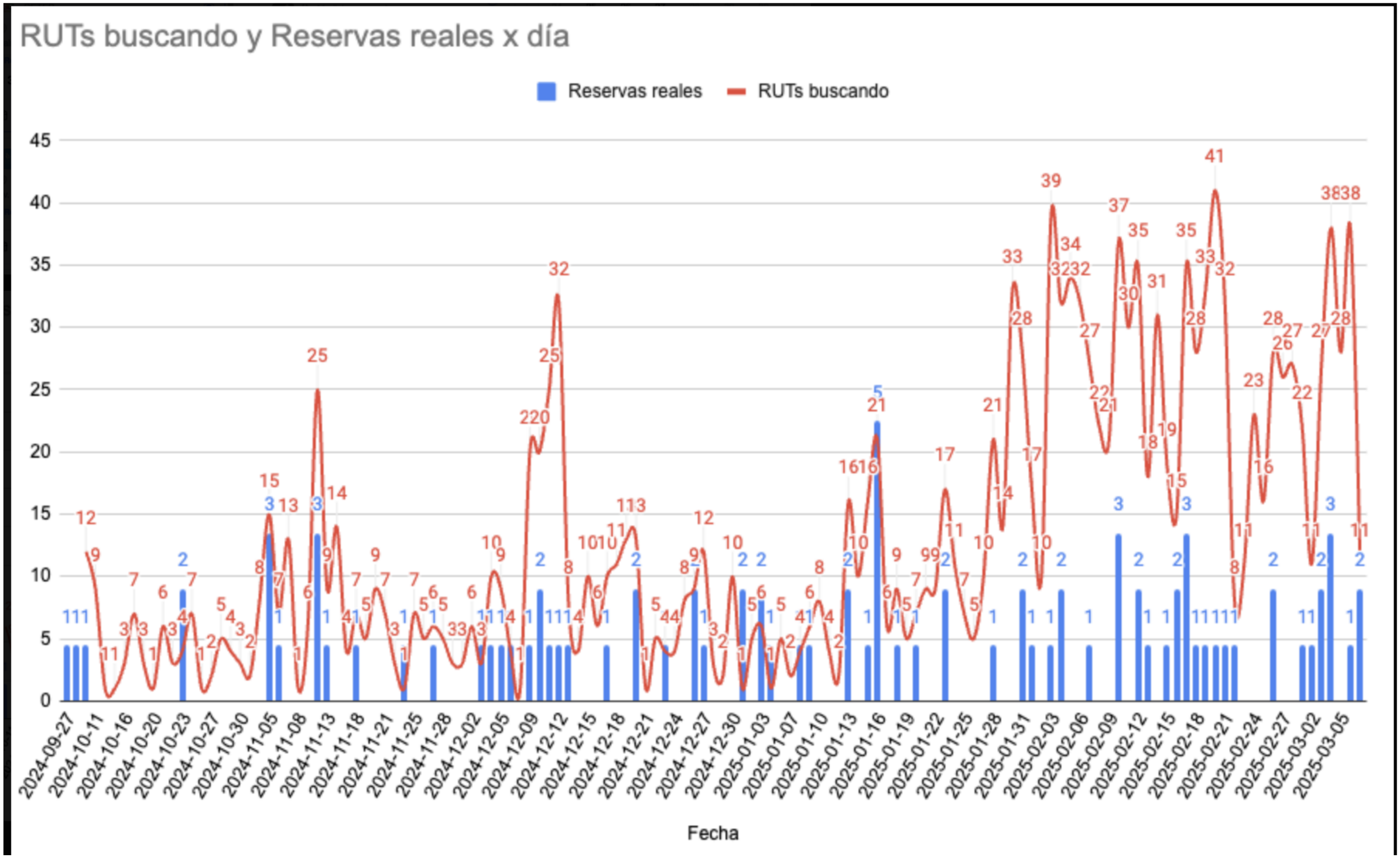


- Lo más frecuente son las lecturas por lo que debemos garantizar la disponibilidad de los datos

- A este punto, lo más importante para negocio es garantizar la disponibilidad del cátalogo

Base de datos - Replicas

Generalmente los sistemas retail y salud son fuertes en lectura



Base de datos - Replicas

Master - Write | Replica - Read Only

- A nivel de código separo las lecturas de las escrituras
- Defino dos conexiones, uno para master y otro para replica
- Si replica o master cae entonces clientes igual pueden ver productos

```
1   const express = require('express');
2   const { Pool } = require('pg');
3
4   const app = express();
5   app.use(express.json());
6
7   const masterPool = new Pool({
8     host: 'master.example.com',
9     port: 5432,
10    user: 'dbuser',
11    password: 'dbpassword',
12    database: 'dbname'
13  });
14
15  const replicaPool = new Pool({
16    host: 'replica.example.com',
17    port: 5432,
18    user: 'dbuser',
19    password: 'dbpassword',
20    database: 'dbname'
21  });
```

Customer Retention Rate

La tasa de retención de clientes es un indicador que mide el porcentaje de clientes que siguen haciendo negocios con un minorista durante un periodo de tiempo determinado.

Base de datos - Replicas

Master - Write | Replica - Read Only

```
23  const productController = {
24    // Create a new product (write to master)
25    createProduct: async (req, res) => {
26      const { name, price } = req.body;
27      try {
28        const client = await masterPool.connect();
29        const result = await client.query(
30          'INSERT INTO products (name, price) VALUES ($1, $2) RETURNING *',
31          [name, price]
32        );
33        client.release();
34        res.status(201).json({ product: result.rows[0] });
35      } catch (err) {
36        console.error('Error inserting product:', err);
37        res.status(500).json({ error: 'Internal Server Error' });
38      }
39    },
40
41    // Retrieve products (read from replica)
42    getProducts: async (req, res) => {
43      try {
44        const client = await replicaPool.connect();
45        const result = await client.query('SELECT * FROM products');
46        client.release();
47        res.status(200).json({ products: result.rows });
48      } catch (err) {
49        console.error('Error fetching products:', err);
50        res.status(500).json({ error: 'Internal Server Error' });
```

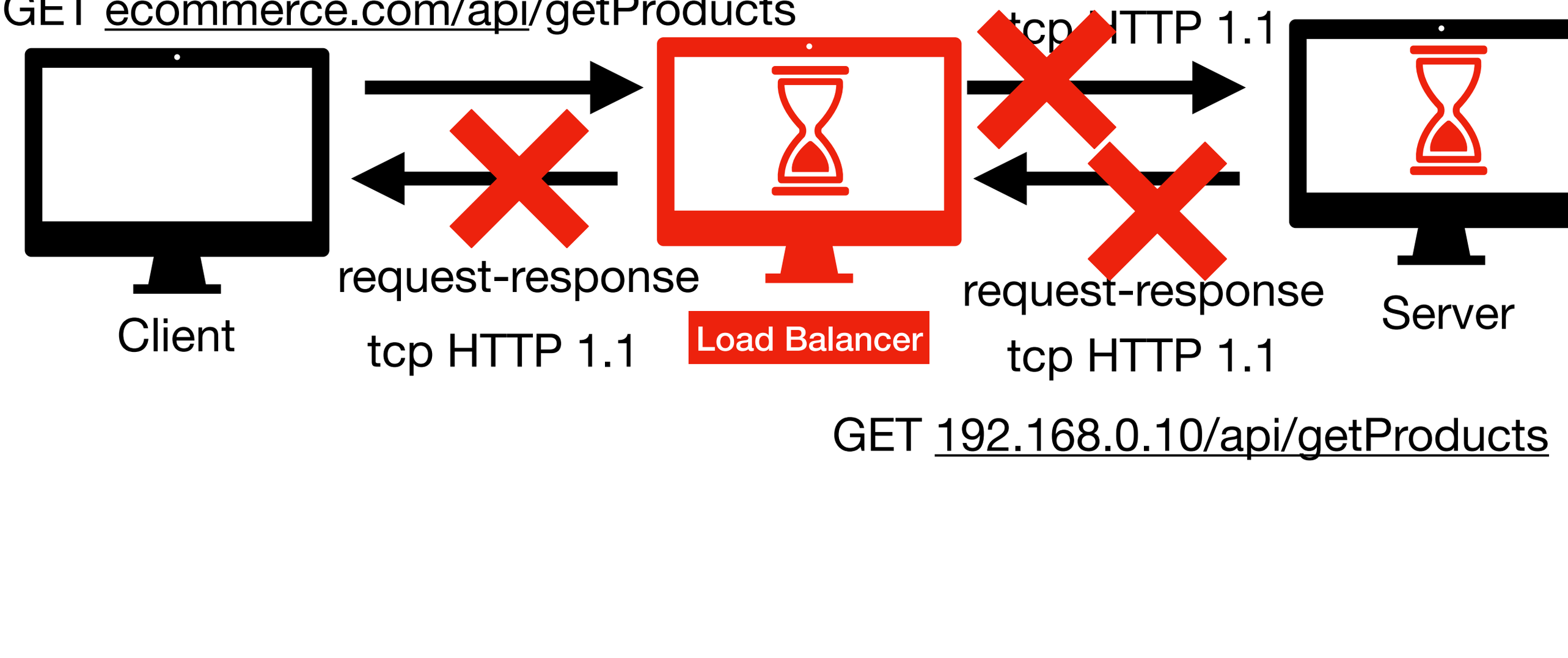
Base de datos - Replicas

Master - Write | Replica - Read Only

```
55    // Define routes using the controller methods
56    app.post('/api/products', productController.createProduct);
57    app.get('/api/products', productController.getProducts);
58
59    const PORT = process.env.PORT || 3000;
60    app.listen(PORT, () => {
61      console.log(`Server running on port ${PORT}`);
62    });
```

Subproblema 3:
¿Qué pasa con la seguridad?
Proteger de Ataques - Saturación

Resource exhaustion - DoS (Denial of Service)



Seguridad (ataque DoS)

Impacto en negocio

- Este tipo de ataques consume recursos sin restricciones, lo que puede provocar la indisponibilidad del sistema o la obtención no autorizada de datos.

- El consumo se realiza a través de cualquier identificador en los sistemas, como usuario, clave, ID, URL, token, entre otros.

- Estos ataques son posibles cuando los sistemas no cuentan con controles adecuados para prevenir ataques de denegación de servicio (DoS) y/o fuerza bruta.

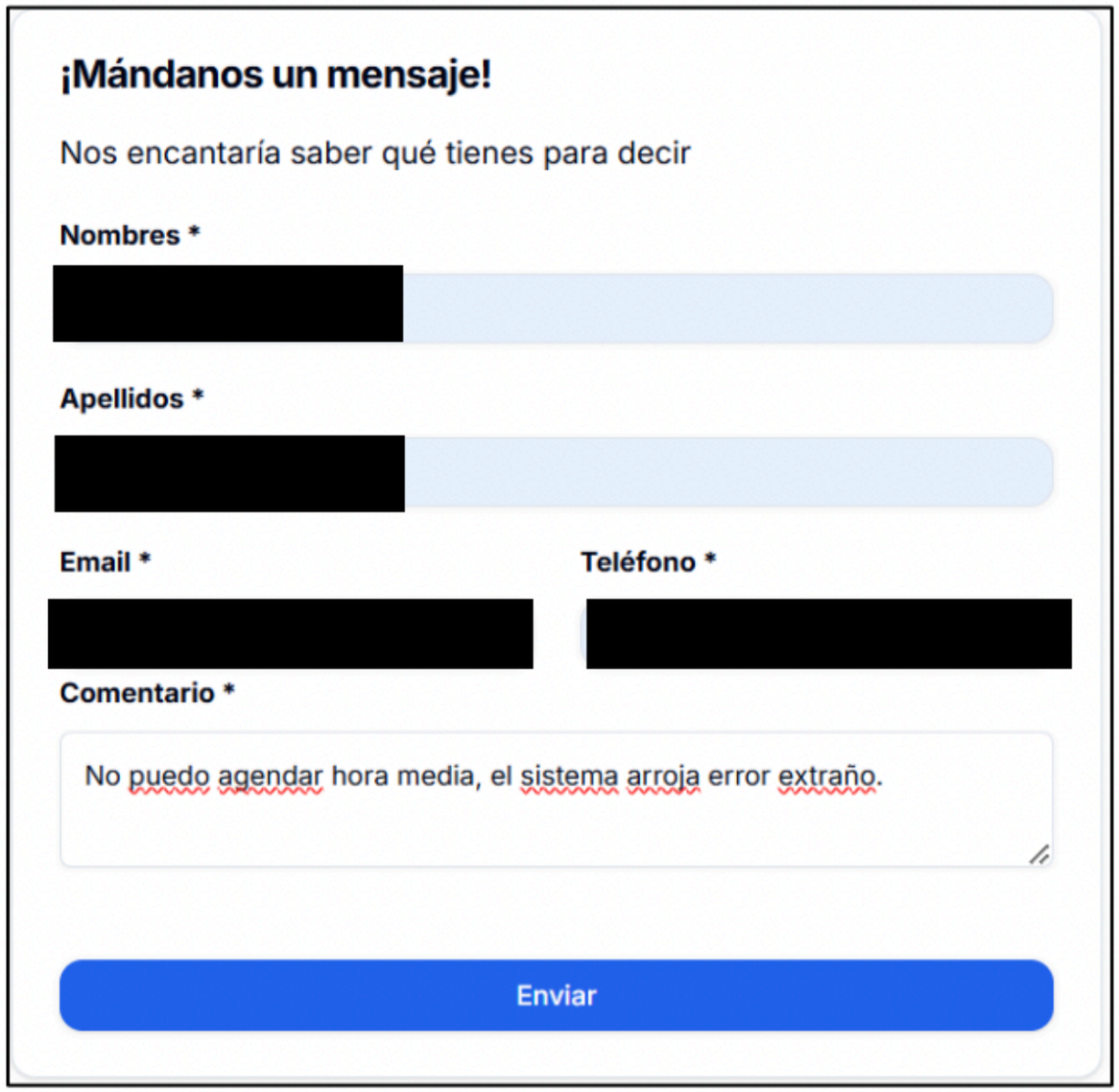
- **Impacto**:

  - **Indisponibilidad o performance del sistema** por el consumo excesivo desde una misma fuente.

  - **Llenado de datos inválidos como peticiones** de contacto al servicio al cliente.

Seguridad (ataque DoS)

Impacto en negocio - Ejemplo formulario contacto



Formulario de contacto llenado.



Petición de asistencia enviada.

Seguridad (ataque DoS)

Impacto en negocio - Ejemplo formulario contacto

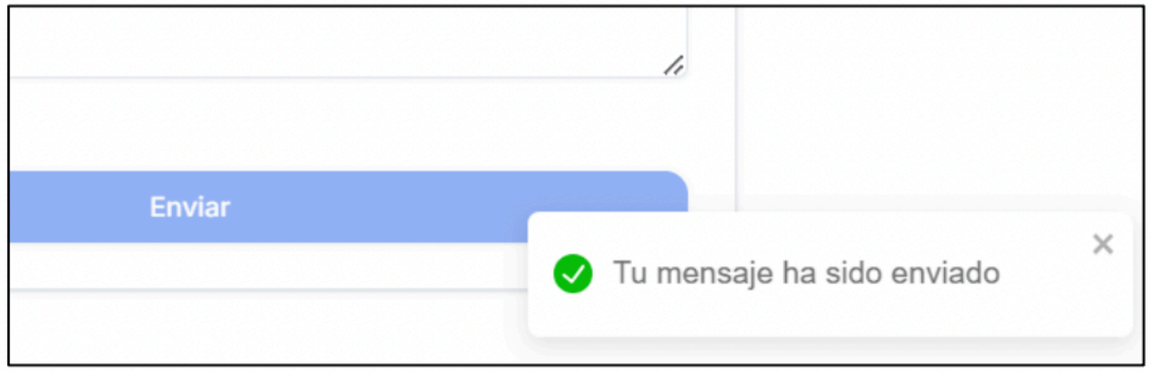
- Se puede obtener los datos de la request y automatizar un script para enviar por ejemplo más de 9000 solicitudes POST a la misma URL.

- **Impacto**:

  - **Si se usa un servicio de envio de correos como MailChimp, se incurre en costos extras.**

  - **Genera alta demanda en los tickets de atención (costos).**

  - **Demandas por acoso telefonico, llenado de buzón, entre otros.**



Petición de asistencia enviada

```
POST /api/v1/contact HTTP/2
Host: [redacted]
Cookie: _ga=GA1.1.1137283567.1730845848; _clck=mdkb7r%7C2%7Cfqn%7C0%7C1770;
eyJraWQiOiI40TQ2NmMyNyliMWFjLTQ40WMtYjcm0SljYjRjZjg50GVjNWMiLCJhbGciOiJSUmI
JmZW5naW51IiwiY29tcGFueSI6IlN0YXJiaWVuIiwiaWQiOiIwMDAwMDAwMC11MTFhLWUxMWItZ
eYP50R3L8lorbfZNcIfwKh3ykFUfkpQanPck3AGeNwitPKfWEfaCZ50v8jV4ZgY8643Rj9TyJ-e
14psmjK6P71CURdWZ1S85GLxShMT4hrZ7criBtinCi2KyWHGYEPwZPRgaIMYQ1H-0iMtBUJ0Gk4
ywf44NITKsG6rbd6ljkkTFVUpFVE-KcR0o0Rj7p1Kyni08aZQBgWpbJrXZDhlaxrnfAnH-stLif
mAt_t-395WTrsE; _clsk=ixi2b2%7C1730889467351%7C34%7C1%7Cj.clarity.ms%2Fcoll
Content-Length: 531
Sec-Ch-Ua-Platform: "Windows"
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (K
Sec-Ch-Ua: "Chromium";v="130", "Google Chrome";v="130", "Not?A_Brand";v="99
Content-Type: application/json
Sec-Ch-Ua-Mobile: ?0
Accept: */*
Origin: [redacted]
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: [redacted]e/inicio/contact
Accept-Encoding: gzip, deflate, br
Accept-Language: en-GB,en-US;q=0.9,en;q=0.8
Priority: u=1, i
Connection: keep-alive
```

Seguridad (ataque DoS)

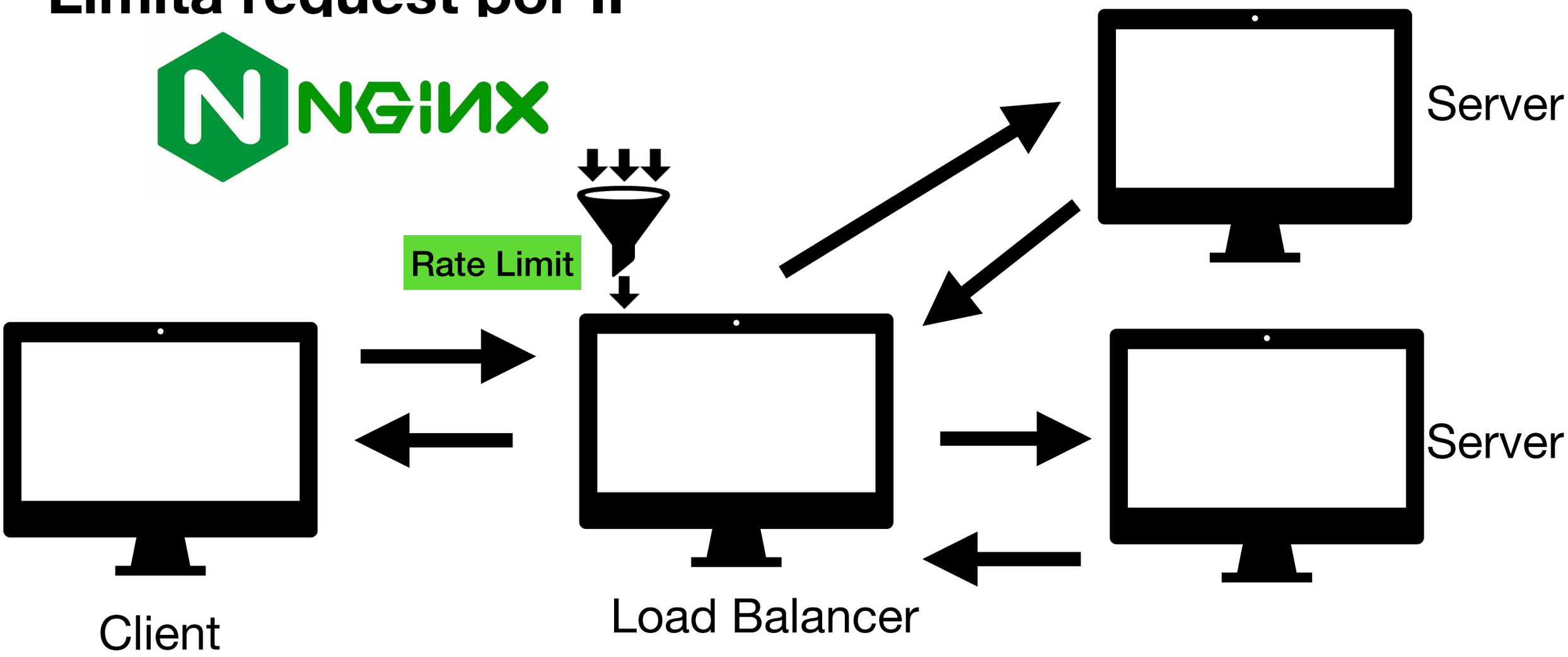
Impacto legal - Ejemplo formulario contacto

- El impacto sobre el negocio está determinado en base a acciones legales en casos de personas que demandan a compañías por acoso telefónico, así como también reclamos o acciones legales u contractuales de terceros por el abuso del sistema al generar múltiples peticiones de contacto hacia otras empresas.

- **Fuente: https://www.bufetes.cl/articulos/victima-de-llamadas-spam-conoce-tus-derechos**

Solución: Rate Limiter

Limita request por IP



CLOUDFLARE®

Configuración a nivel de la librería o tecnología (NGINX, Cloudflare, etc)
También puede ser a nivel de firewall

Solución: Rate Limiter

Ejemplo: Configuración NGINX

```
/etc/nginx/nginx.conf

http {
    # Rate is set to 10 requests per second.
    limit_req_zone $binary_remote_addr zone=mylimit:10m rate=10r/s;

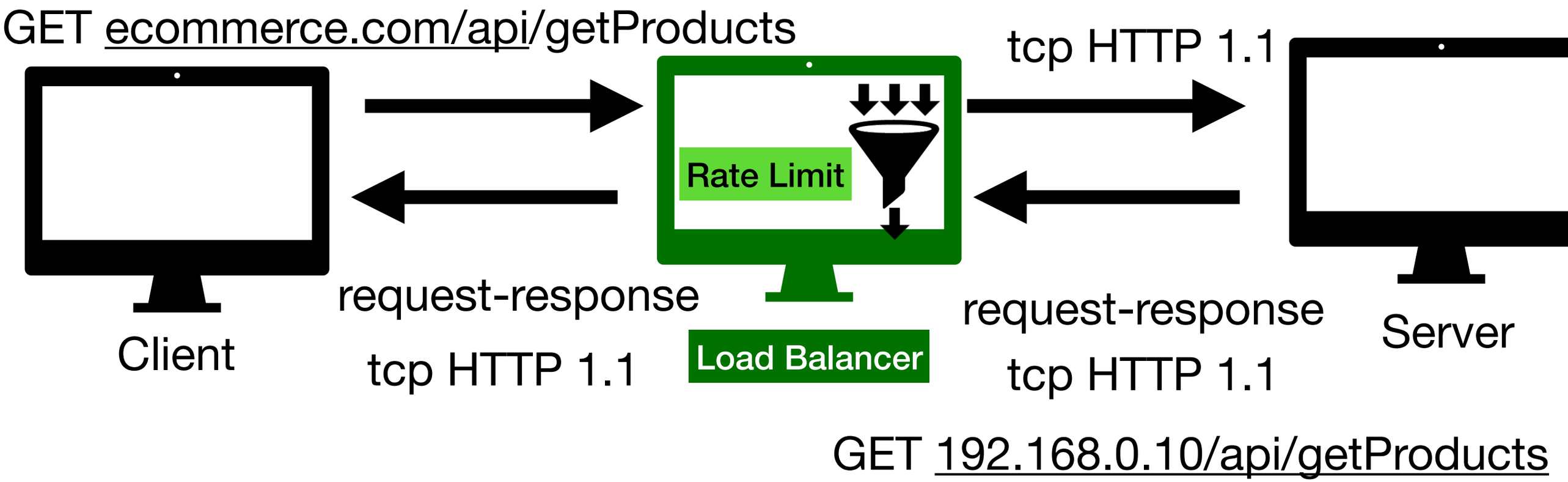
    server {
        listen 80;
        server_name example.com;

        location / {
            # Apply the rate limiting to this location.
            # "burst" allows extra requests (here up to 20) to be queued.
            # "nodelay" means requests within the burst are processed immediately.
            limit_req zone=mylimit burst=20 nodelay;

            proxy_pass http://backend;
        }
    }
}
```

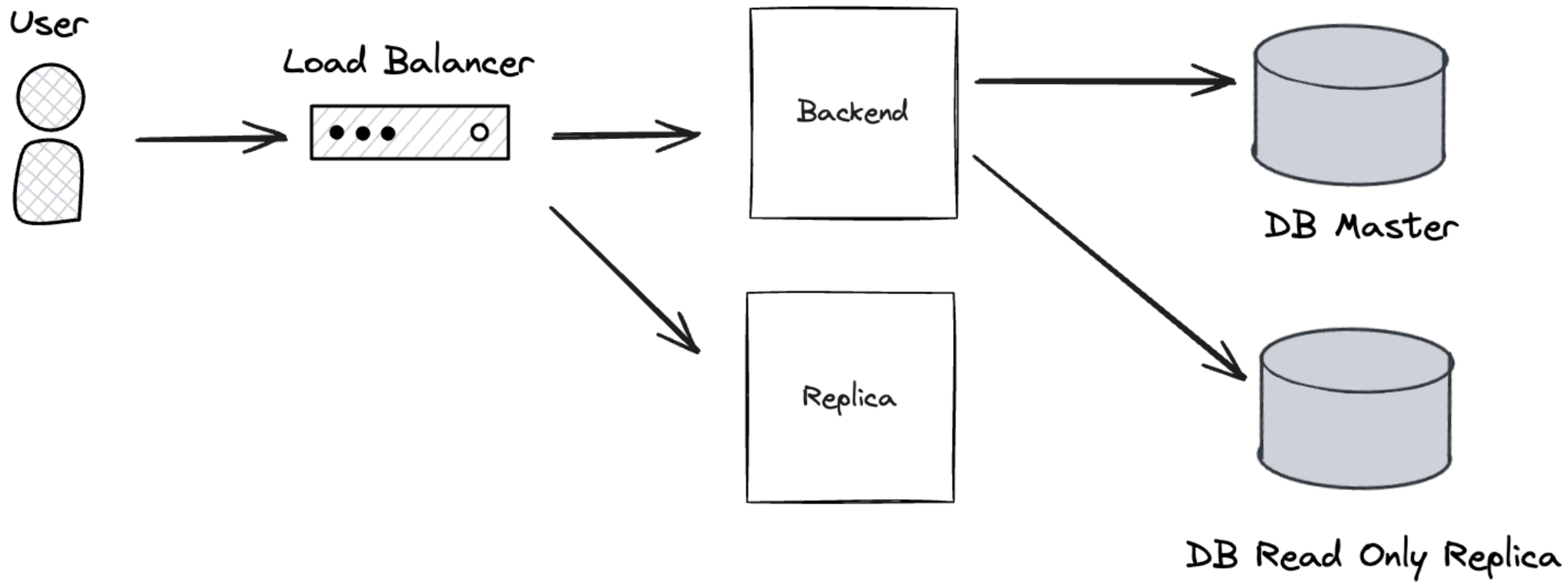
Subproblema 3:
¿Qué pasa con la seguridad?
Proteger de Ataques - Saturación

SOLUCIONADO



Sistema: listar productos ecommerce

Sólo 100 productos



¿Qué aprendimos hoy?

- Toma de decisiones técnicas en base al contexto del negocio
- Poner en perspectiva de negocio cada decisión de arquitectura (tradeoffs)
- Hoy nos centramos en lo básico de disponibilidad, escalabilidad y comunicación.
- Aprender a pensar en:
  - **Negocio**
  - **Producto**
  - **Ingeniería**

Siguiente clase profundizaremos en base de datos, escalabilidad y disponibilidad con otro caso de negocio

Introducción a System Design - Clase 2

Diseño de sistemas
8.20 - 9.30 am



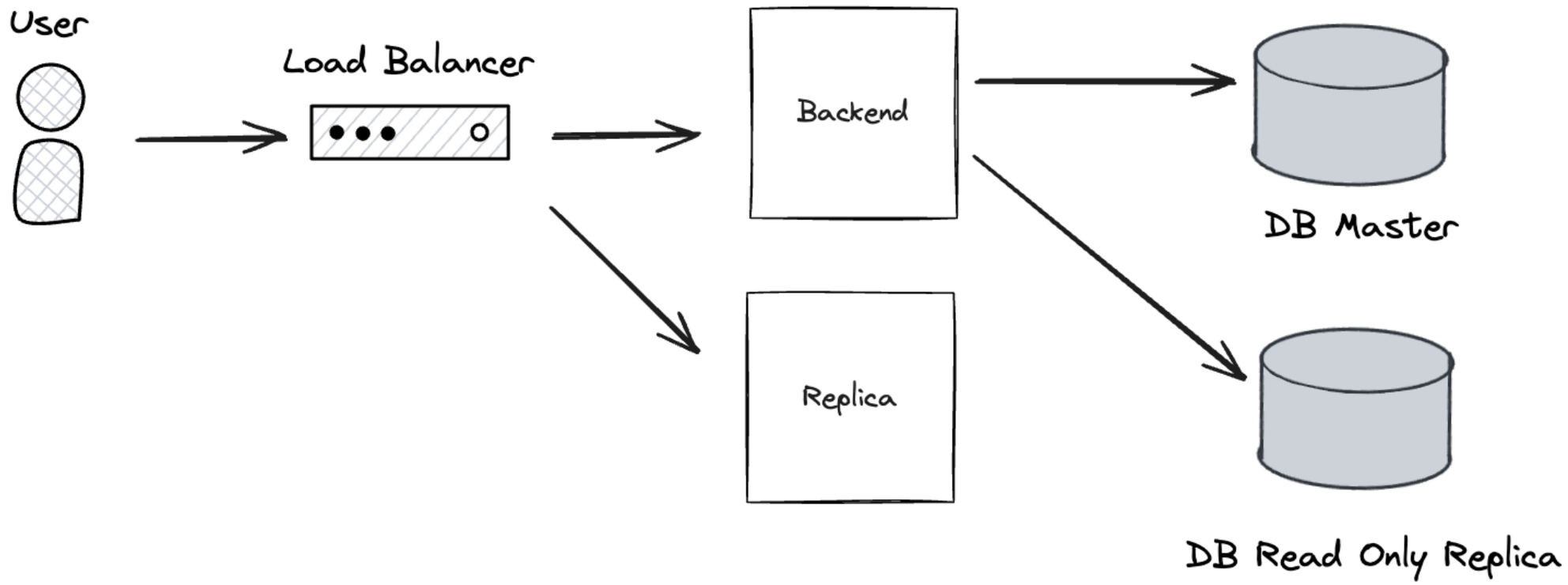
Gonzalo Martinez, 2025

Clase Pasada

- Problema:
  - Sistema que lista los productos de un ecommerce
    - Permite crear/actualizar productos
- Restricción: sólo hay 100 items en la base de datos

Sistema: listar productos ecommerce

Sólo 100 productos

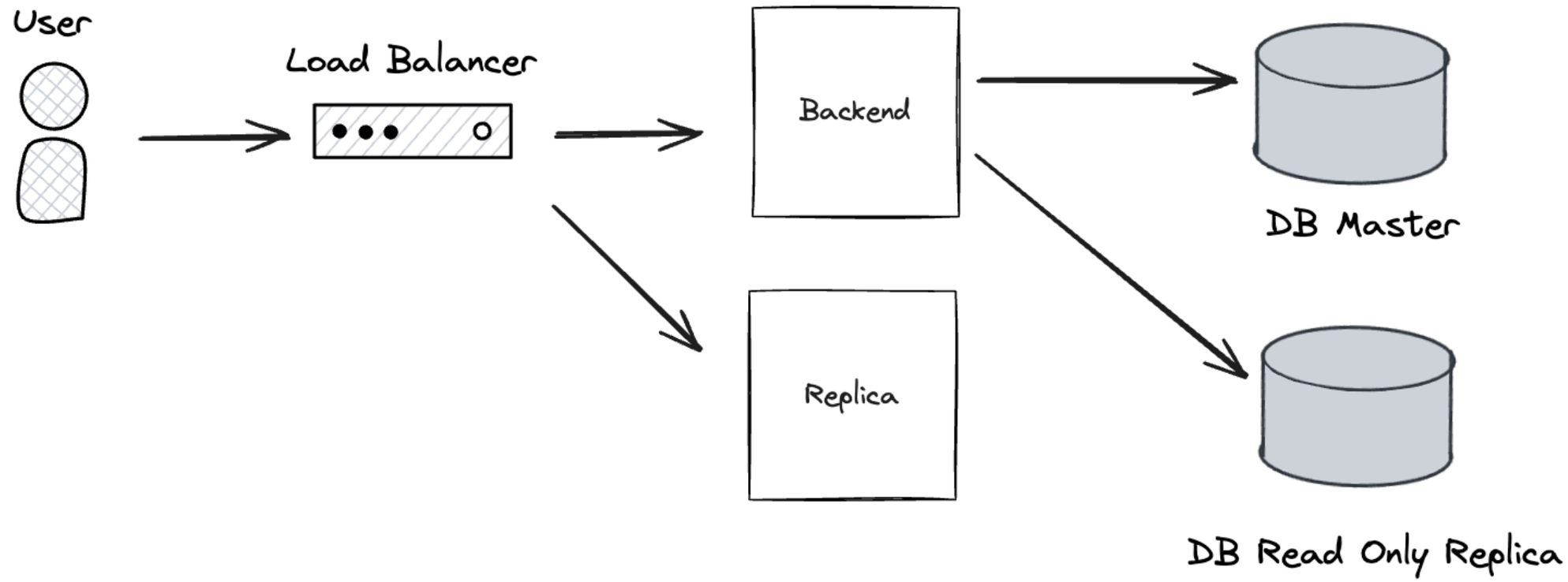


Contenido hoy

- Hoy hablare del proyecto y el contenido de la primera entrega
- Continuación System Design:
- **Cache**
- **Punto de partida es la arquitectura de la clase pasada**

Diseño Sistema Clase Pasada

Sólo 100 productos

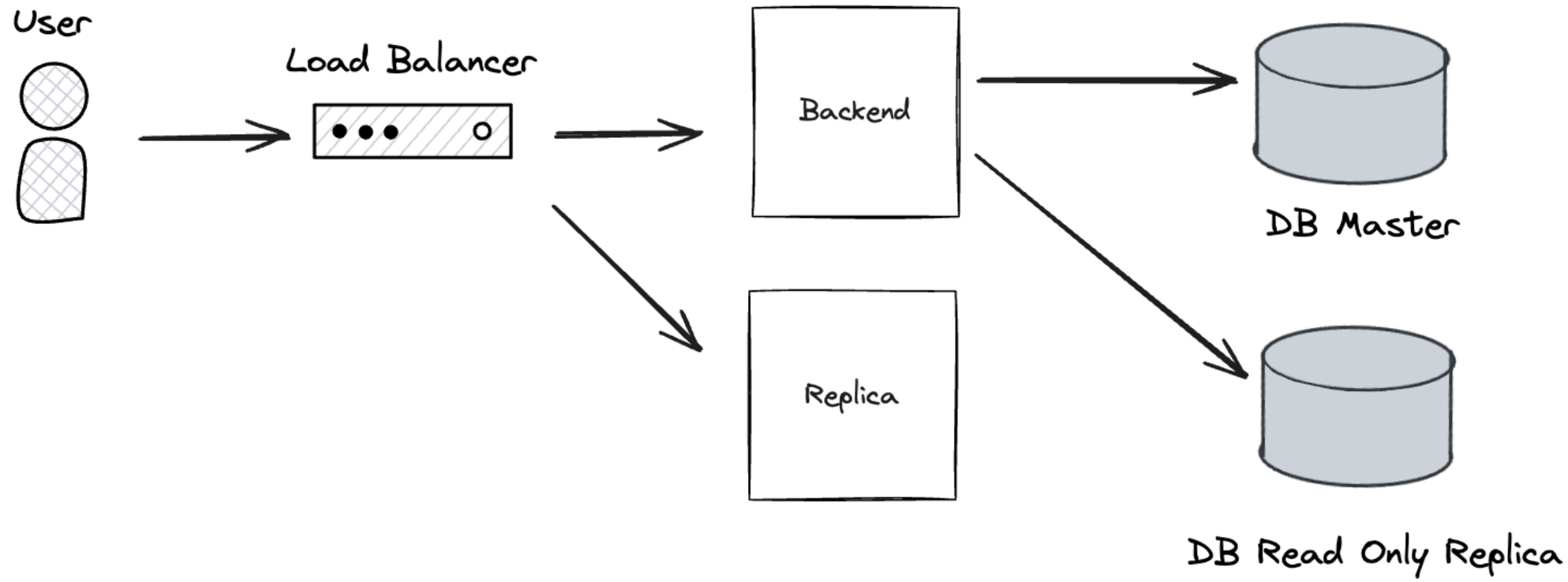


Discusión:
¿Por qué la base de datos replica es sólo lectura?

- Piensen en lo opuesto -

Diseño Sistema Clase Pasada

Sólo 100 productos

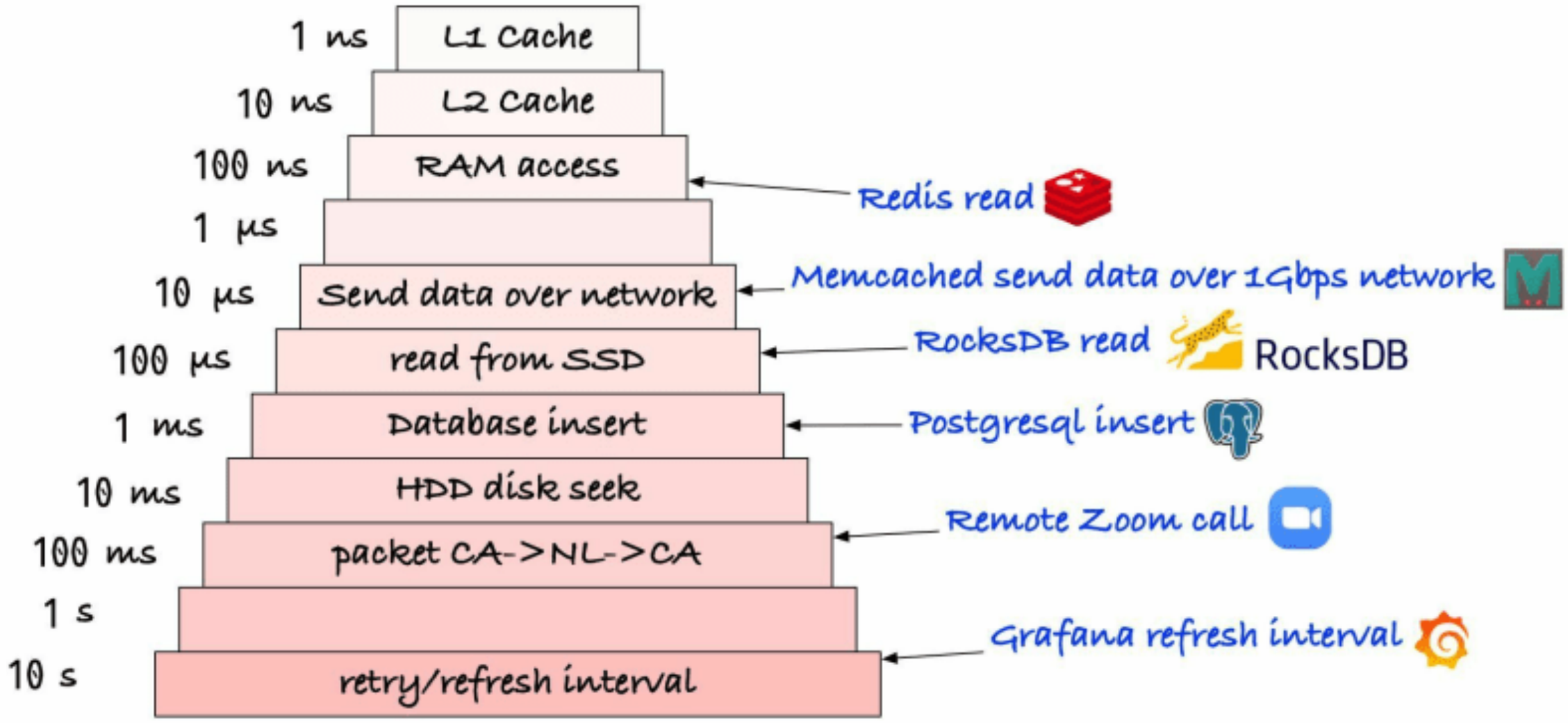


Discusión:
Digamos que 50 productos son los más frecuentes o que no presentan cambios

¿Cómo puedo optimizar el tiempo de respuesta?

Latency Numbers You Should Know

ByteByteGo.com



Referencia Latencia
bytebytego.com

Cache

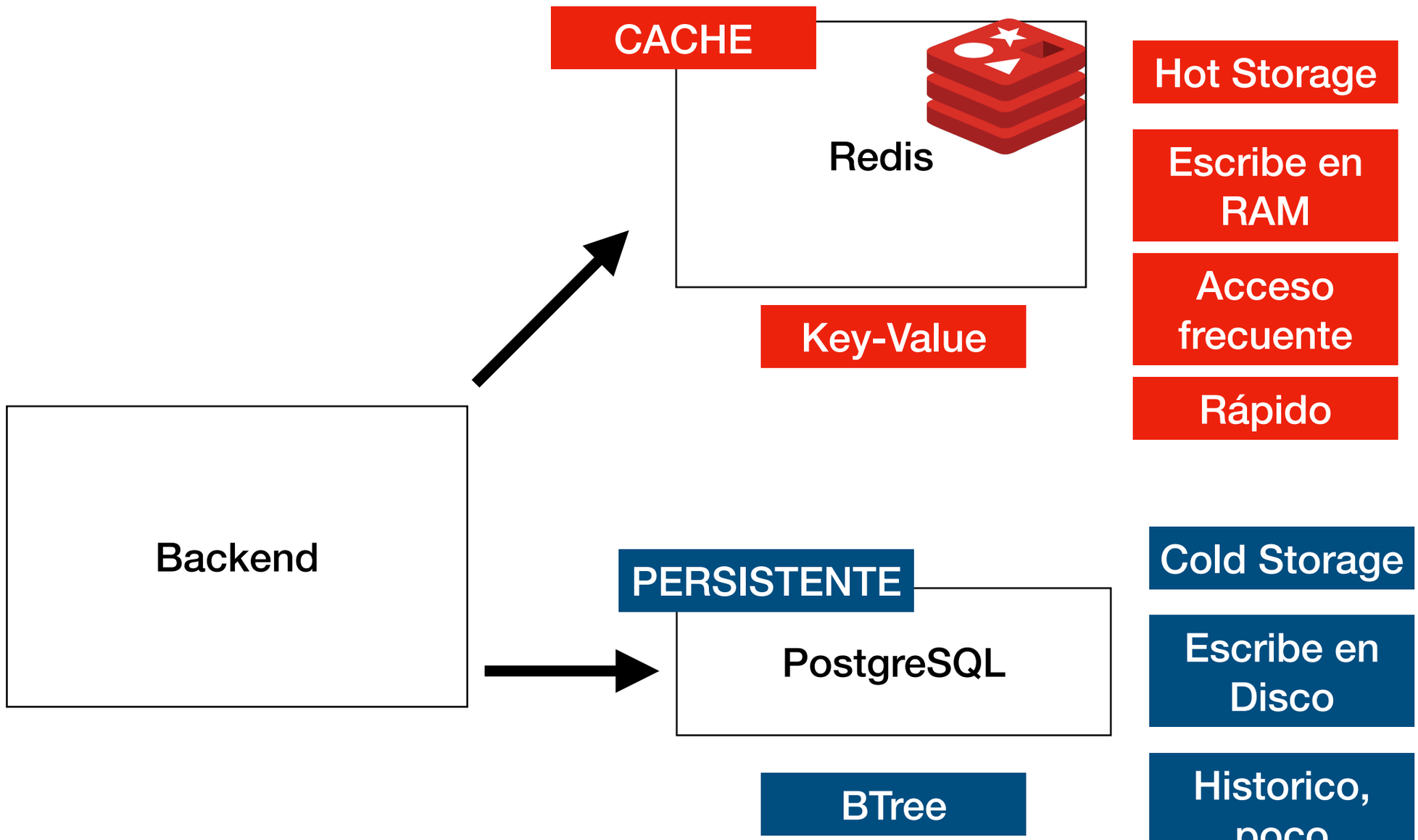
Reduce tiempos de respuesta

- Reduce tiempos de respuesta almacenando cualquier computación o operación I/O.
- Caso de uso: Reducir disk I/O o computacion o network I/O
- Cache son Hash Table glorificado.

Storage

Cold vs Hot

Storage Pattern: Hot vs Cold

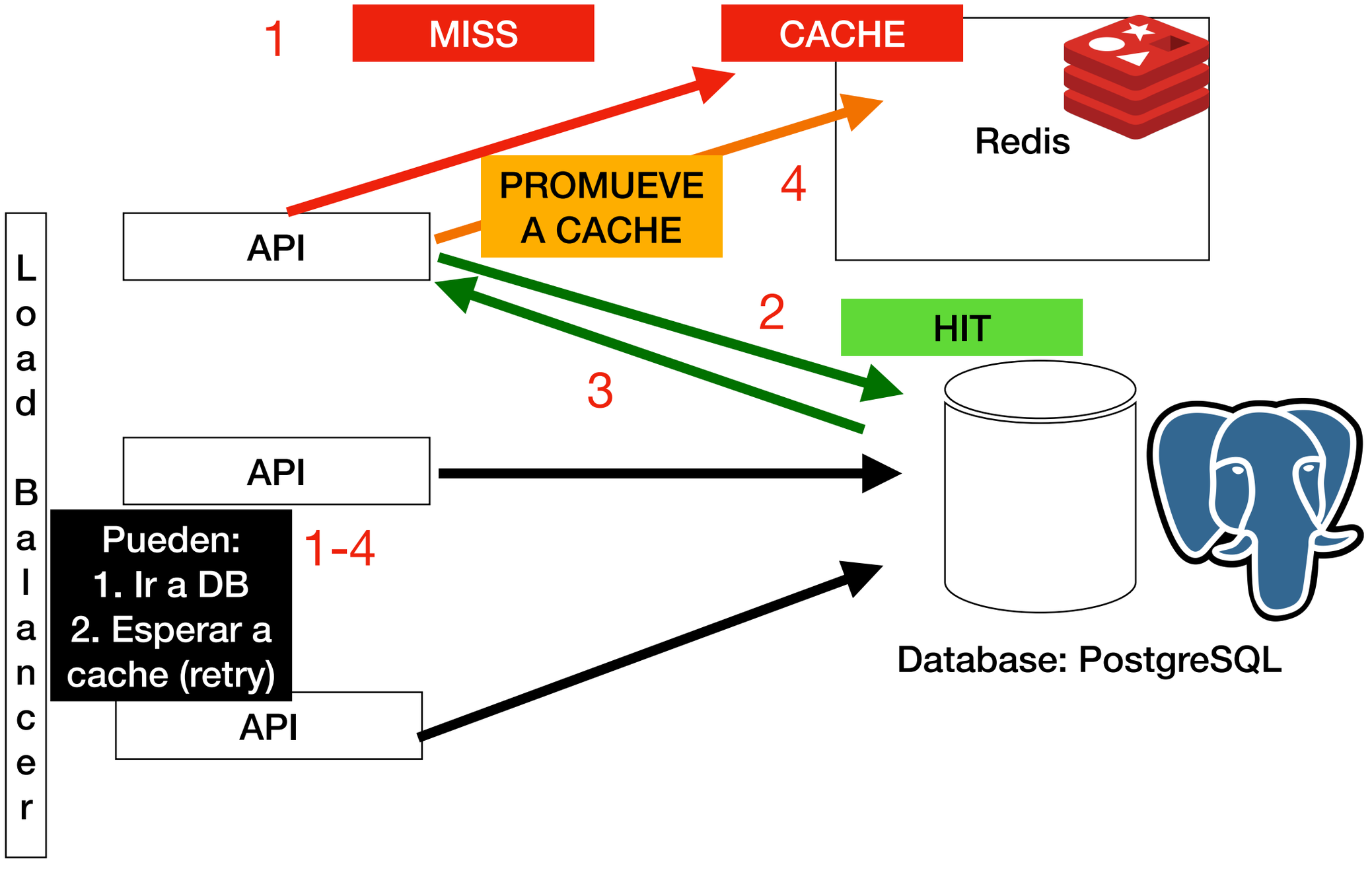


Piensen en tradeoffs

¿Qué sucede si Redis/Cache se cae?

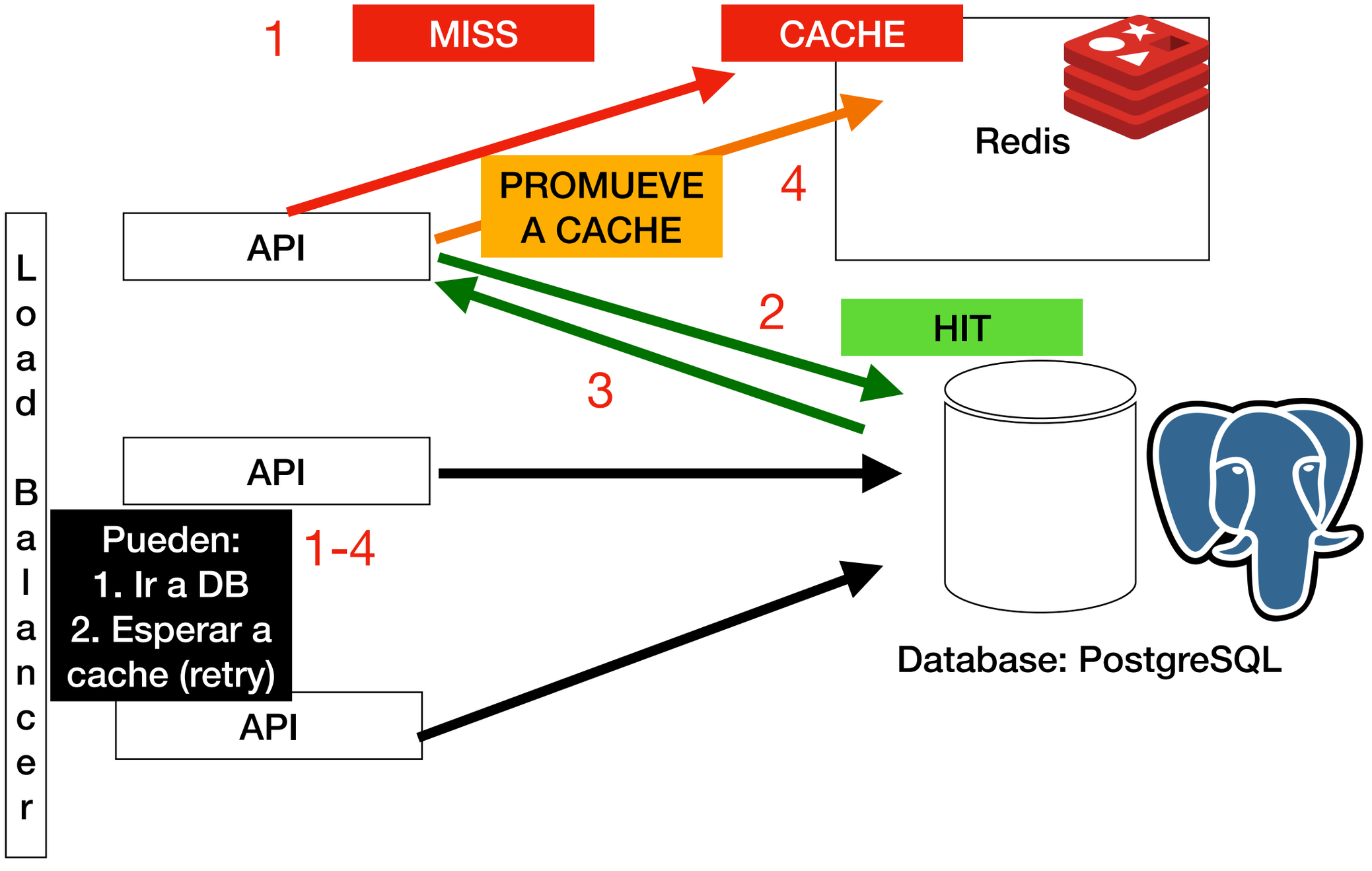
¿Cómo puedo determinar si algo es “frecuente”?

Promover acceso frecuente



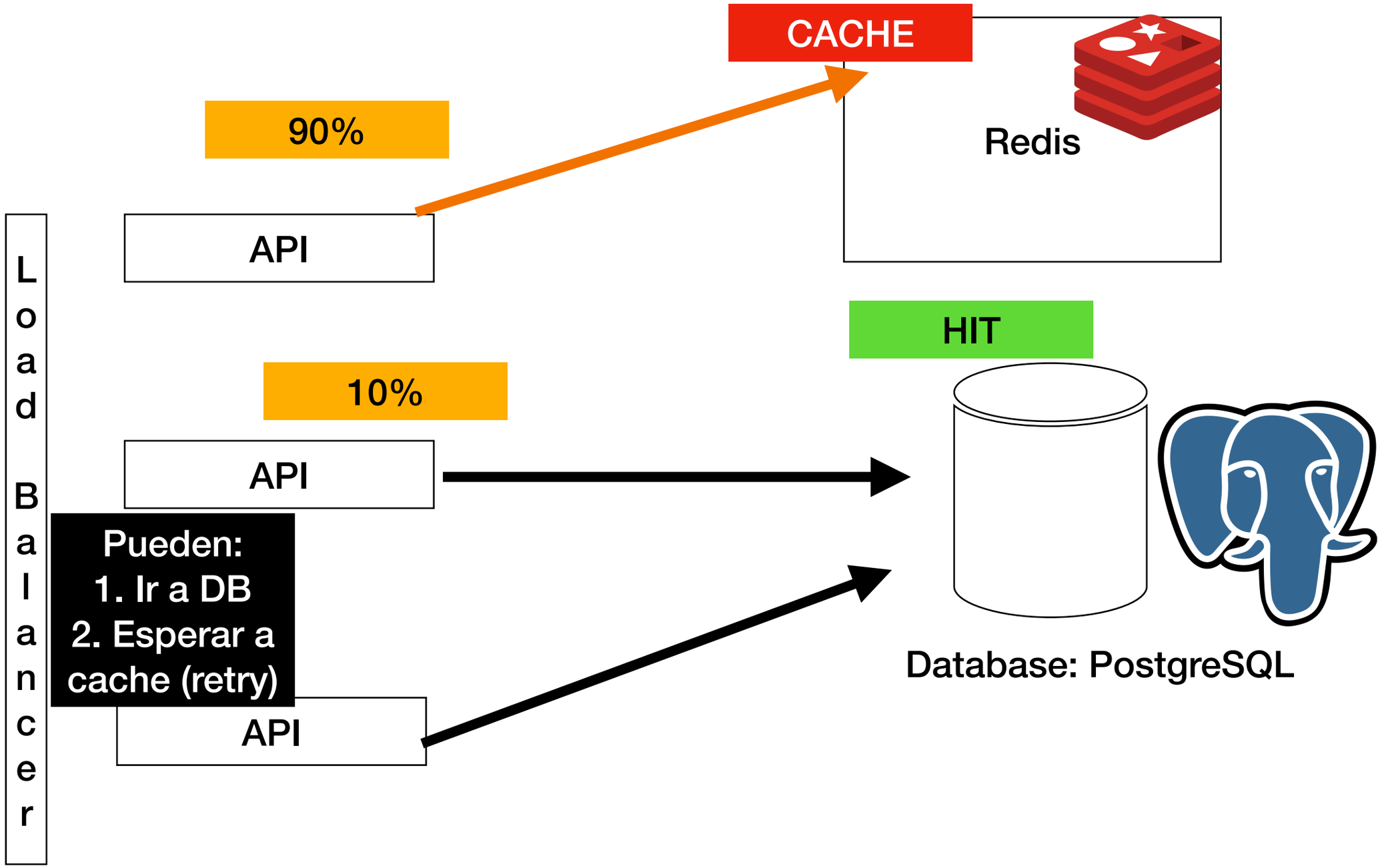
Cache Debouncing: también llamado Request Hedging
Múltiples requests van a cache al mismo tiempo. Una va a la BD, promueve el valor a cache y el resto espera

Problema: ¿Cómo actualizo cache? (discusión en clases)



Cache Eviction: ocurre cuando la cache se encuentra llena y es necesario vaciarlo. También es necesario pensar que necesito y por cuanto tiempo necesito datos en el cache (Time To Live- TTL)

Problema: nunca accedo a DB



Si nunca accedo a BD y sólo ocupo cache, máquina con BD se “enfría”.
Solución: distribuyo request, ejemplo: 90% a Cache, 10% a BD PostgreSQL

¿Qué aprendimos hoy?

- Cache para mejorar rendimiento (tiempos de respuesta) en una aplicación)

Siguiente clase profundizaremos en Cache

Introducción a System Design - Clase 3

Diseño de sistemas
8.20 - 9.30 am



Gonzalo Martinez, 2025

Contenido hoy

- Hoy hablare del proyecto y el contenido de la primera entrega
- Continuación System Design:
- **Cache**
- **Protocolos de Comunicación**
- **Punto de partida es la arquitectura de la clase pasada**

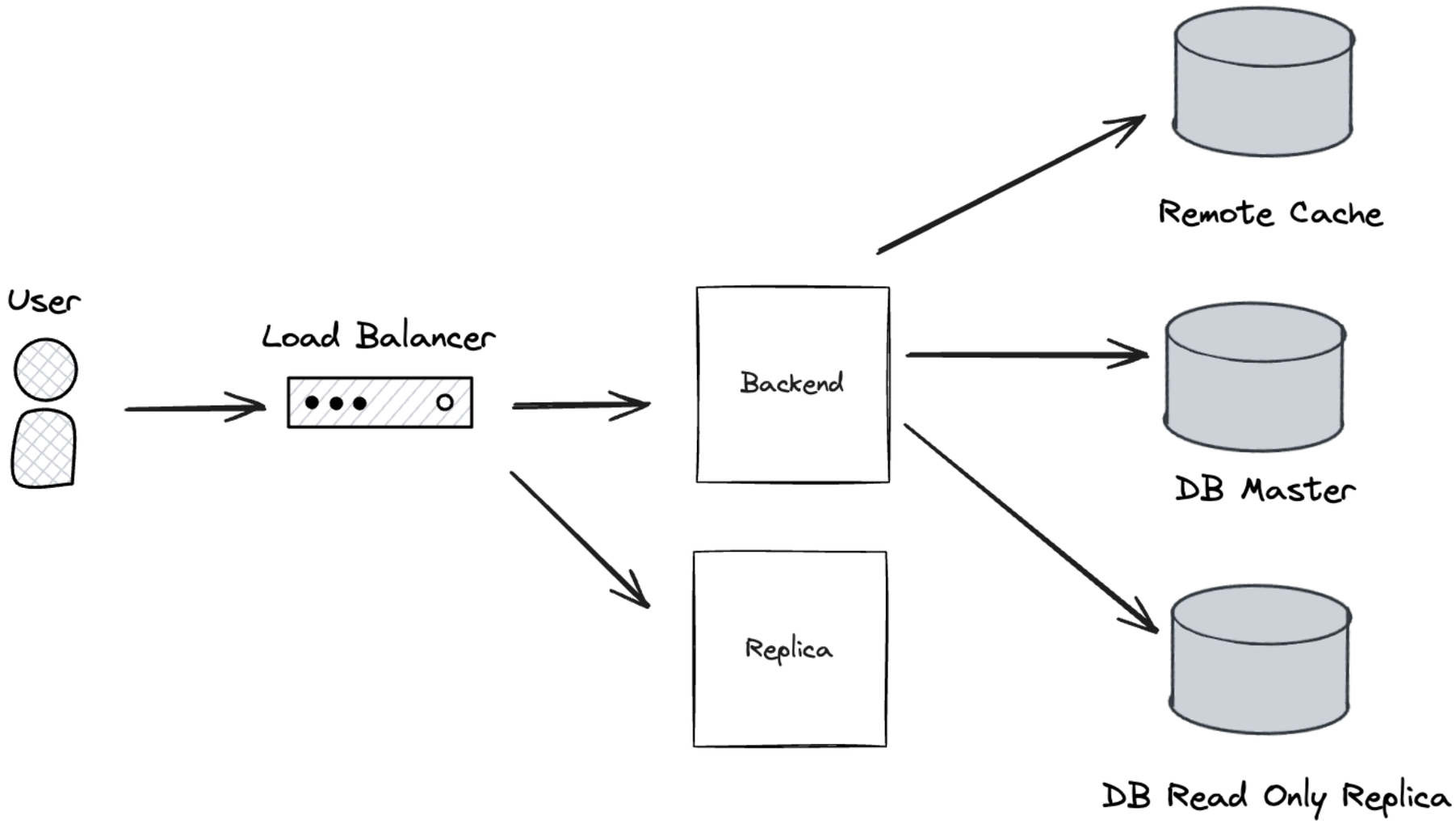
Clase Pasada

- Problema:
  - Sistema que lista los productos de un ecommerce
    - Permite crear/actualizar productos
- Restricción: sólo hay 100 items en la base de datos
- Datos de acceso frecuente (argumento para cache)

Sistema: listar productos ecommerce

Sólo 100 productos

Cache: para accesos frecuentes



Prototipo

Cache usando Redis y Request Hedging

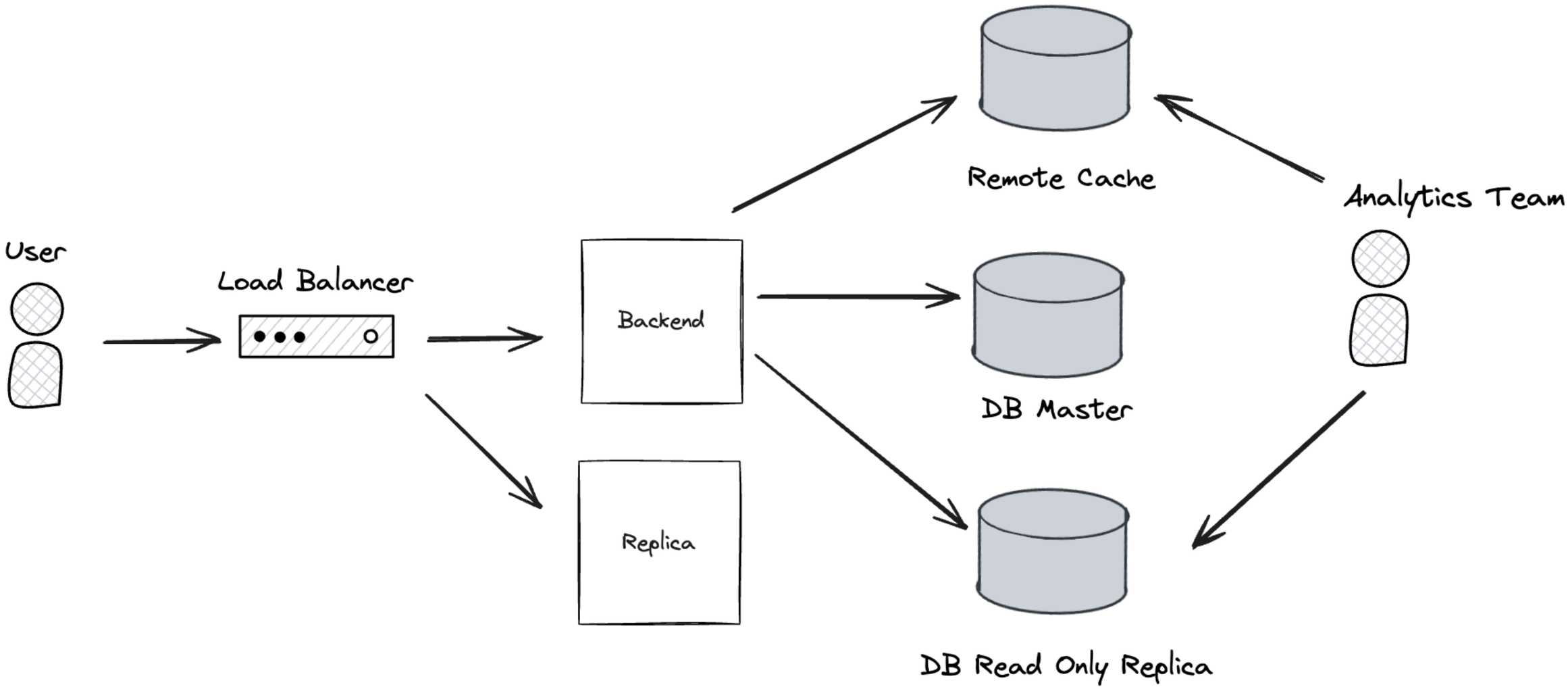
https://github.com/UC-IIC-3143-Software-Development/prototypes/tree/main/07-ecommerce-cache-redis

```
~/src/uc/prototypes/07-ecommerce-cache-redis main ❯ python3 ecommerce_cache_storage_hot_cold.py
Product 4 not in cache, fetching from database (cold storage - MySQL)
Retrieved product 4 in 29.73 ms
Product 3 not in cache, fetching from database (cold storage - MySQL)
Retrieved product 3 in 5.93 ms
Product 2 not in cache, fetching from database (cold storage - MySQL)
Retrieved product 2 in 5.67 ms
Product 3 found in cache (hot storage - Redis)
Retrieved product 3 in 0.91 ms
Product 3 found in cache (hot storage - Redis)
Retrieved product 3 in 0.56 ms
Product 3 found in cache (hot storage - Redis)
Retrieved product 3 in 0.79 ms
Product 1 not in cache, fetching from database (cold storage - MySQL)
Retrieved product 1 in 8.90 ms
Product 1 found in cache (hot storage - Redis)
Retrieved product 1 in 0.61 ms
Product 1 found in cache (hot storage - Redis)
Retrieved product 1 in 0.67 ms
Product 1 found in cache (hot storage - Redis)
Retrieved product 1 in 0.61 ms
```

Ahora pensemos en los usos y costos en producción

Sistema: listar productos ecommerce

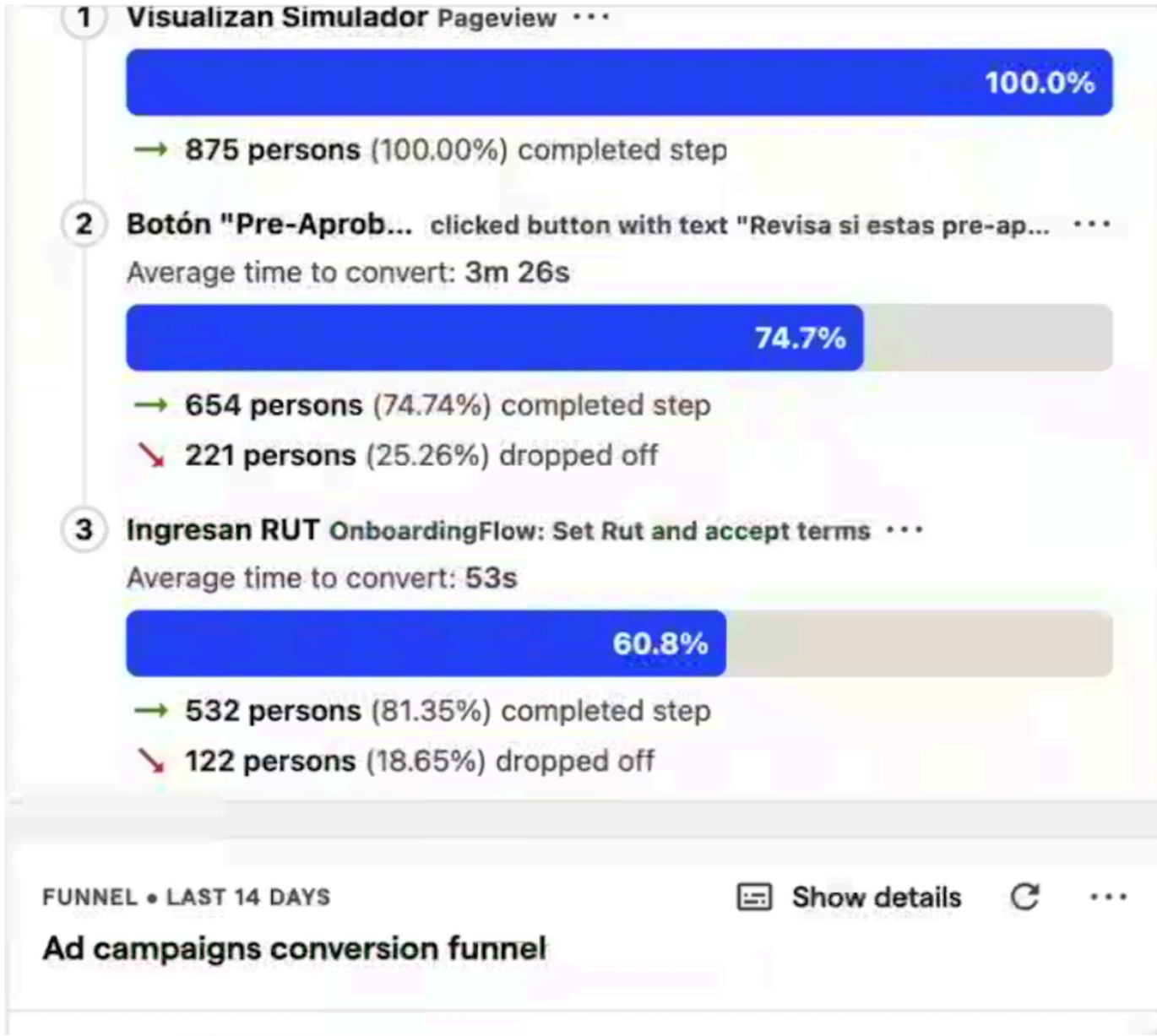
Replica y Cache también es útil para equipos de analytics



Uso de replicas y cache

Equipos de Marketing/Ventas/Analytics (FunnelSales, Dashboards)

- El consumo de BD replica y/o Cache no afecta el desempeño en producción y asimismo protege la fuente original de los datos.



Cache Debounce: enviar parte de las request hacia cache

Prototipo

https://github.com/UC-IIC-3143-Software-Development/prototypes/blob/main/08-cache_debounce/cache_debounce.py

```
~/src/uc/prototypes/08-cache_debounce main ❯ python3 cache_debounce.py
Cache Server: redis://cache.myblogs.com
Database Server: postgresql://db.myblogs.com
Cache Percentage: 30%
Number of Requests: 100000

Total Requests: 100000
Requests to Cache: 29811 (29.81%)
Requests to Database: 70189 (70.19%)
```

Costos Redis - GCP

Cloud Memorystore $35.77 / month

Service type
Memorystore for Redis

Memorystore for Redis configuration Advanced settings

Region
Iowa (us-central1)

Service tier
Basic

Storage amount 1 GiB

Average hours per month 730

Link billing account to view negotiated pricing.

Cost details USD

+ Add to estimate

DATABASES $35.77

Memorystore for Redis
Cloud Memorystore
$35.77

ESTIMATED COST $35.77 / mo

Share

Cache: ¿Cómo podemos optimizar costos?

Discusión en clases

Posterior a la clase subo la respuesta a esta misma slide

Latencia P99

Ejemplo real: métricas de operación (tiempo en segundos)

Métricas de operación: Tiempo de respuesta

Es cada request de dispo (podría ser más de 1 por RUT)



%til	Sem 53	Sem 01	Sem 02	Sem 03	Sem 04	Sem 05	Sem 06	Sem 07
p99	23,7	30,6	30,4	9,8	14,2	16,3	53,9	16,8
p95	23,2	30,5	27,1	8,5	13,3	12,8	30,2	12,7
p75	20,4	21,1	16,7	6,3	8,6	6,8	7,7	7,8
p50	18,8	16,3	7,1	4,7	6,5	5,0	4,8	4,8

- P99 - Semana 7: el 99% de las solicitudes tuvieron un tiempo de respuesta **igual o menor** a 16,8 segundos.

- P95 - Semana 7: el 95% de las solicitudes tuvieron un tiempo de respuesta **igual o menor** a 12,7 segundos.

- P50 (mediana)

Diseño de Sistema

Servicio que consulta la cartelera de cine y estrenos

- Diseñar un sistema que obtenga las peliculas disponibles de un cine y los estrenos de la semana

- Asumir que la información del cine y sus peliculas ya tienen tablas en la bd.

Discusión en clases
Posterior a la clase subo la slide con el diseño

Modelo OSI

Redes

Engineer Work

Layer 7. Application - HTTP/FTP/RPC — Backend, Frontend

Layer 6. Presentation - Encoding, Serialization — Backend, Frontend

Layer 5. Session - Connection establishment, TLS

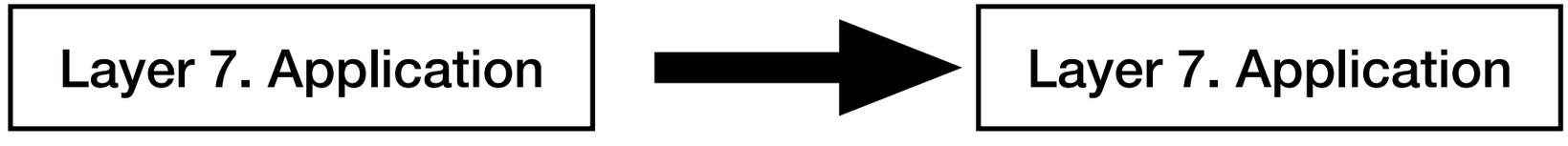
Layer 4. Transport - UDP/TCP — Backend

Layer 3. Network - IP — DevOps/Infra

Layer 2. Data link - Frames, Mac address, Ethernet — DevOps/Infra

Layer 1. Physical - Electric signals, fiber, radio waves

Communication



UDP (User Datagram Protocol)

CONEXIÓN

CONEXIÓN

ESTABLECER CONEXIÓN

ABRIR

REQUEST

SENDER

RESPONSE

RECEIVER

RESPONSE

CLIENTE

SERVIDOR

UDP: NO HANDSHAKE.

SERVER INICIA LA INTERACCIÓN

¿Tradeoffs?

UDP (User Datagram Protocol)

CONEXIÓN

CONEXIÓN

ESTABLECER CONEXIÓN

ABRIR

REQUEST

SENDER

RESPONSE

RECEIVER

RESPONSE

CLIENTE

SERVIDOR

UDP: NO HANDSHAKE.

SERVER INICIA LA INTERACCIÓN

RÁPIDO PERO POSIBLE PÉRDIDA DE PAQUETES

¿EJEMPLOS?

TCP (User Datagram Protocol)

CONEXIÓN

CONEXIÓN

ABRIR

ESTABLECER CONEXIÓN

SYNC

SENDER

RECEIVER

SYNC/ACK

ACK

CLIENTE

SERVIDOR

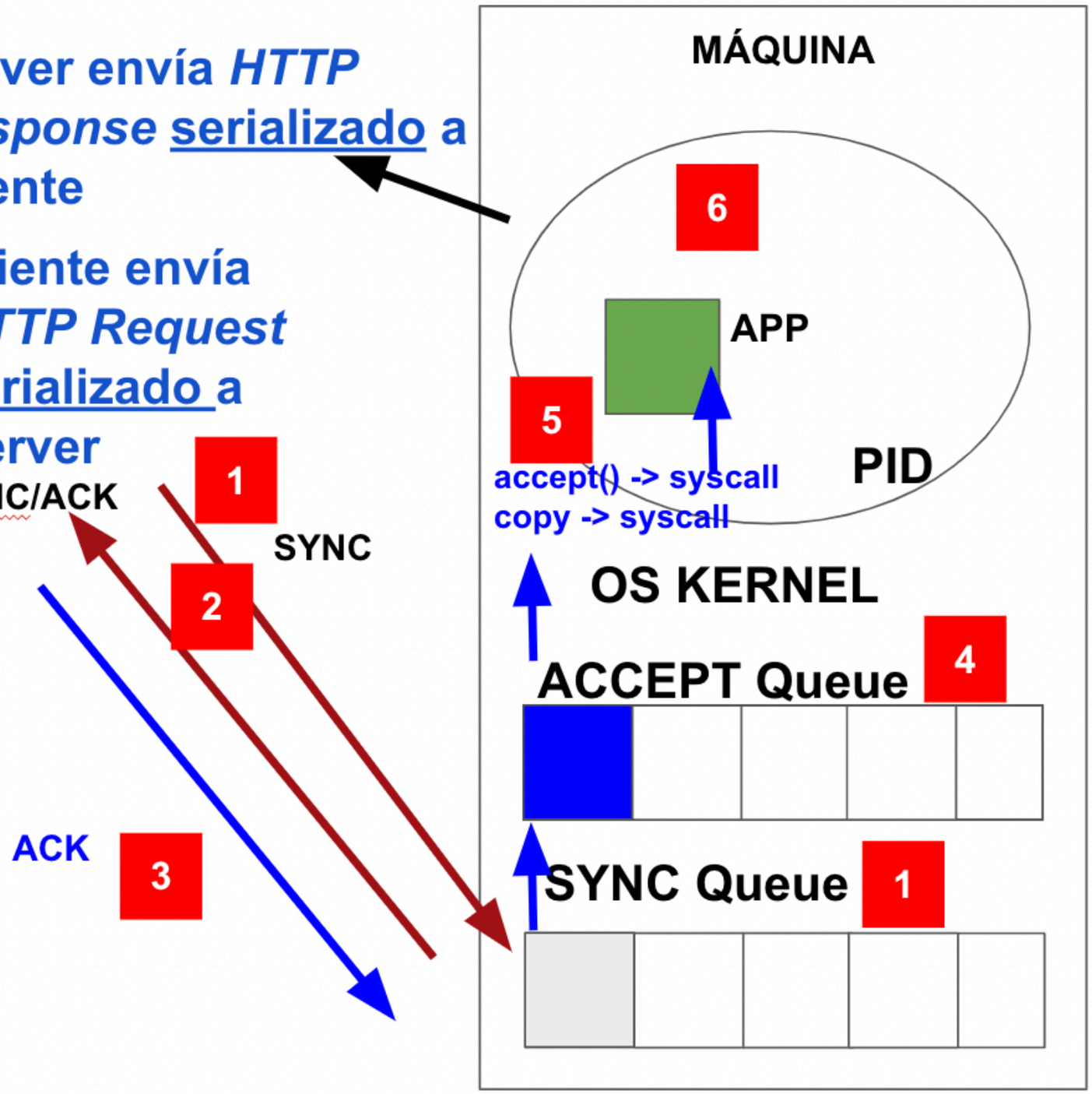
TCP: 3 HANDSHAKE

CLIENTE INICIA LA INTERACCIÓN

LENTO PERO SIN PERDIDA DE PAQUETES

¿EJEMPLOS?

¿Pero por qué TCP es más costoso que UDP?



TCP THREE HANDSHAKE

4 Mueve la información del request de la SYNC queue al ACCEPT queue

5 KERNEL copia la información de la ACCEPT queue al PROCESO DE LA APLICACIÓN

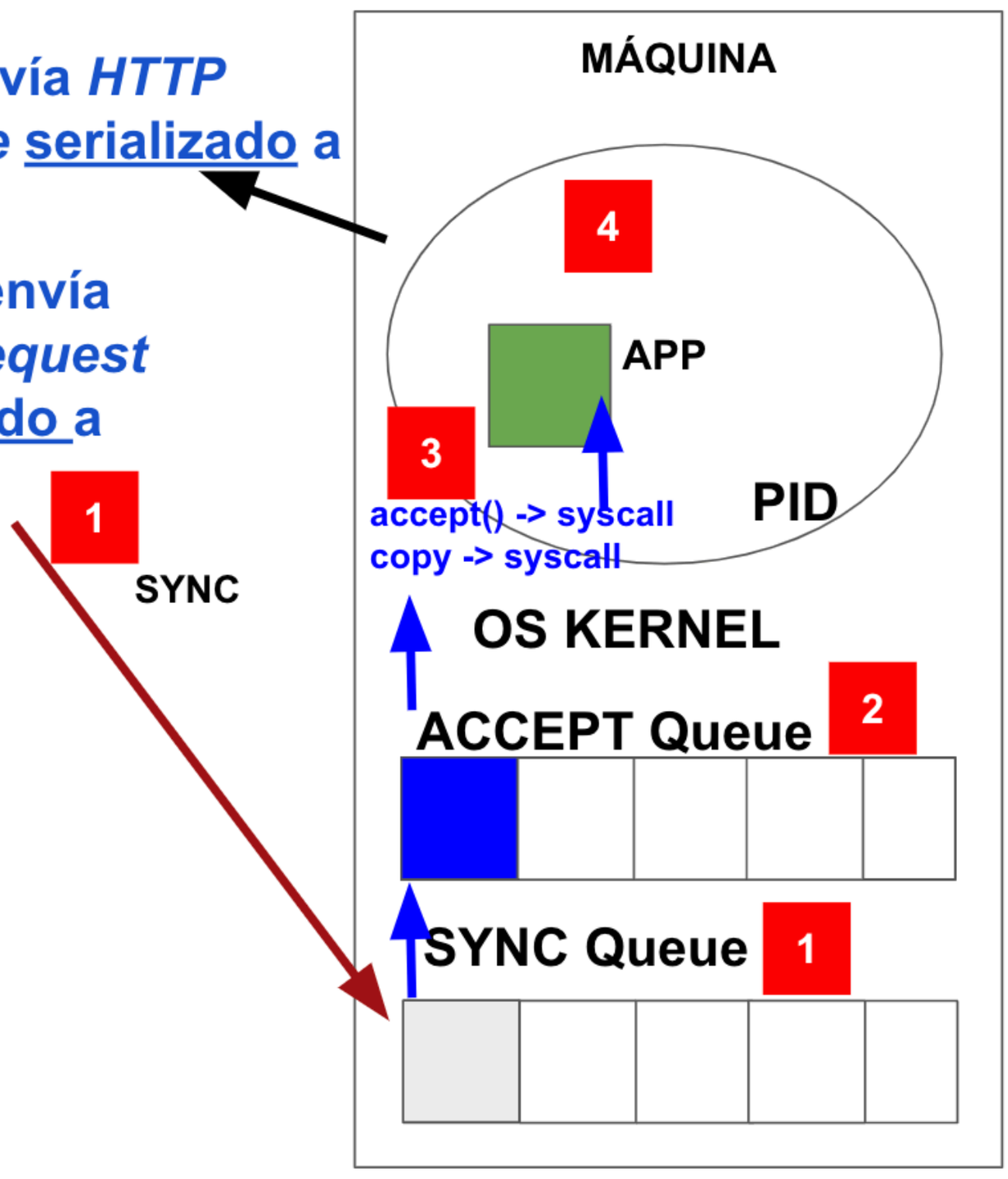
6 APLICACIÓN usa la información almacenada en su proceso (intercepta request)

Paso 2 a 5 es la forma en que el una máquina establece una conexión (entrante o saliente).
Lo mismo ocurre con una BD y/o front.
ACCEPT y SYNC queue son compartidos por todos los procesos del sistema operativo (competencia).

Comunicación confiable

Server envía *HTTP Response* serializado a Cliente

Cliente envía *HTTP Request* serializado a Server



UDP

2 Mueve la información del request de la SYNC queue al ACCEPT queue

3 KERNEL copia la información de la ACCEPT queue al PROCESO DE LA APLICACIÓN

4 APLICACIÓN usa la información almacenada en su proceso (intercepta request)

@levelsio
@levelsio

Subscribe

✅ Improved the other player updates and made them 10x smoother now, it works really well

I do see why people say UDP (with WebRTC) is better than TCP (with WebSockets) now because once packages get stuck with TCP (like if your internet sucks) it becomes like a traffic jam and it sometimes grinds to a halt

While UDP I think just drops the packages and doesn't retry?

Interesting to learn about this, so maybe the reply guy (I forgot his name) was right and we should switch it to WebRTC but make like a main WebRTC signaling server with Python? Nice?



Johnny @johnnymustang_ · Mar 7

The fact you built a game and didn't know UDP vs TCP kind of the epitome of vibe coding

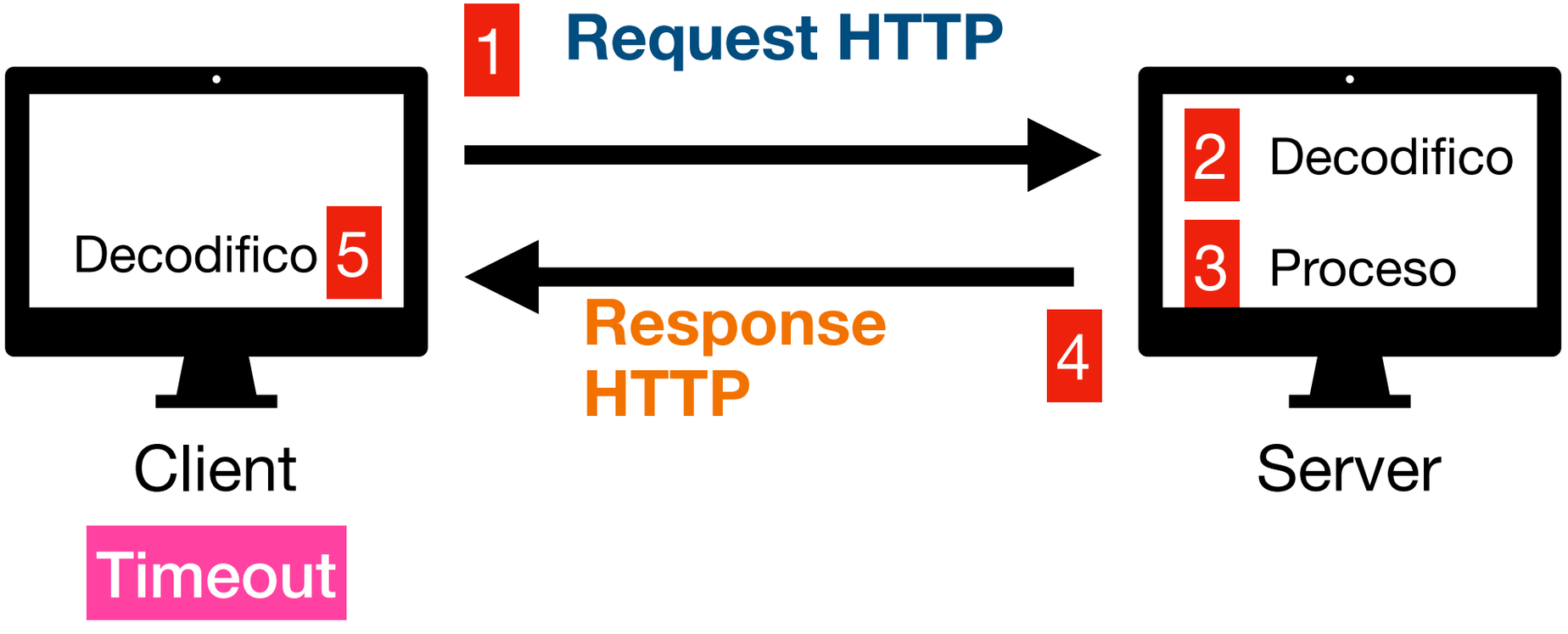

TCP
UDP

Modelo Request

Response

HTTP

- Los humanos necesitamos un **idioma** para comunicarnos.
- En las máquinas a través de Internet, el idioma común en el cuál se comunican es el **protocolo HTTP**.
- HTTP se construye sobre TCP (HTTP 1.x y 2.x, 3.x usa UDP)
- **HTTP : Formato que el cliente y servidor entienden.**



Timeout: tiempo limite para cerrar la conexión

CONEXIÓN

ABRIR

REQUEST

GET /index.html

CONEXIÓN

RESPONSE

CERRAR

CLIENTE

SERVIDOR

HTTP 1.0: Cierra inmediatamente después de recibir el response

Costoso
Pero el cliente no tiene esa responsabilidad

HTTP 1.1: Depende del cliente cuando cerrar la conexión

Barato
Pero el cliente debe saber cuando cerrar la conexión

HTTP

Hypertext Transfer Protocol

- HTTP : Hypertext Transfer Protocol
  - HTTP/0.9 - Publicado en 1991
  - HTTP/1.0 - Publicado en 1996
  - **HTTP/1.1 - Publicado en 1997**
    - **Actualmente es el más usado**
  - HTTP/2 - Publicado en 2015 - **Caso de uso: Streaming**
  - HTTP/3 (QUIC) - Todavía en progreso

Limitaciones del modelo Request Response

No funciona en todos los casos

- No puedo continuamente enviar requests por la red, dado que es **costoso**.
- En consecuencia el modelo Request Response se ve limitado en:
  - **Servicios de notificación** (ej: Notificaciones en el celular)
    - **Es muy costoso preguntar continuamente si tengo una notificación**
  - **Aplicaciones de Chat** (ej: WhatsApp, Telegram)
    - **Es muy costoso preguntar continuamente si alguien me ha hablado.**
  - **Request que toman demasiado tiempo en ser procesados**
    - **Subir una foto** (ej: Instagram, Facebook)
    - **Subir un video** (ej: Youtube)
  - **¿Qué pasa si el cliente se desconecta?**

Mejoremos el modelo Request Response

La ingeniería se inspira en el mundo real

- La comunicación entre humanos puede ser **síncrona** o **asíncrona**
- En la comunicación **sincronía (sync)** el emisor espera la respuesta del receptor.
    - Ejemplo: en la sala de clases el profesor realiza la pregunta a un estudiante. El profesor se queda esperando la respuesta del estudiante y mientras tanto la clase esta en pausa.
- En la comunicación **asíncrona (async)** el receptor puede responder cuando quiera. Mientras tanto el emisor puede realizar otras tareas
    - Ejemplo: alguien le envía un correo electrónico a otra persona. El emisor puede realizar otras tareas mientras espera a que el receptor responda. Por otro lado, el receptor responde el email cuando pueda

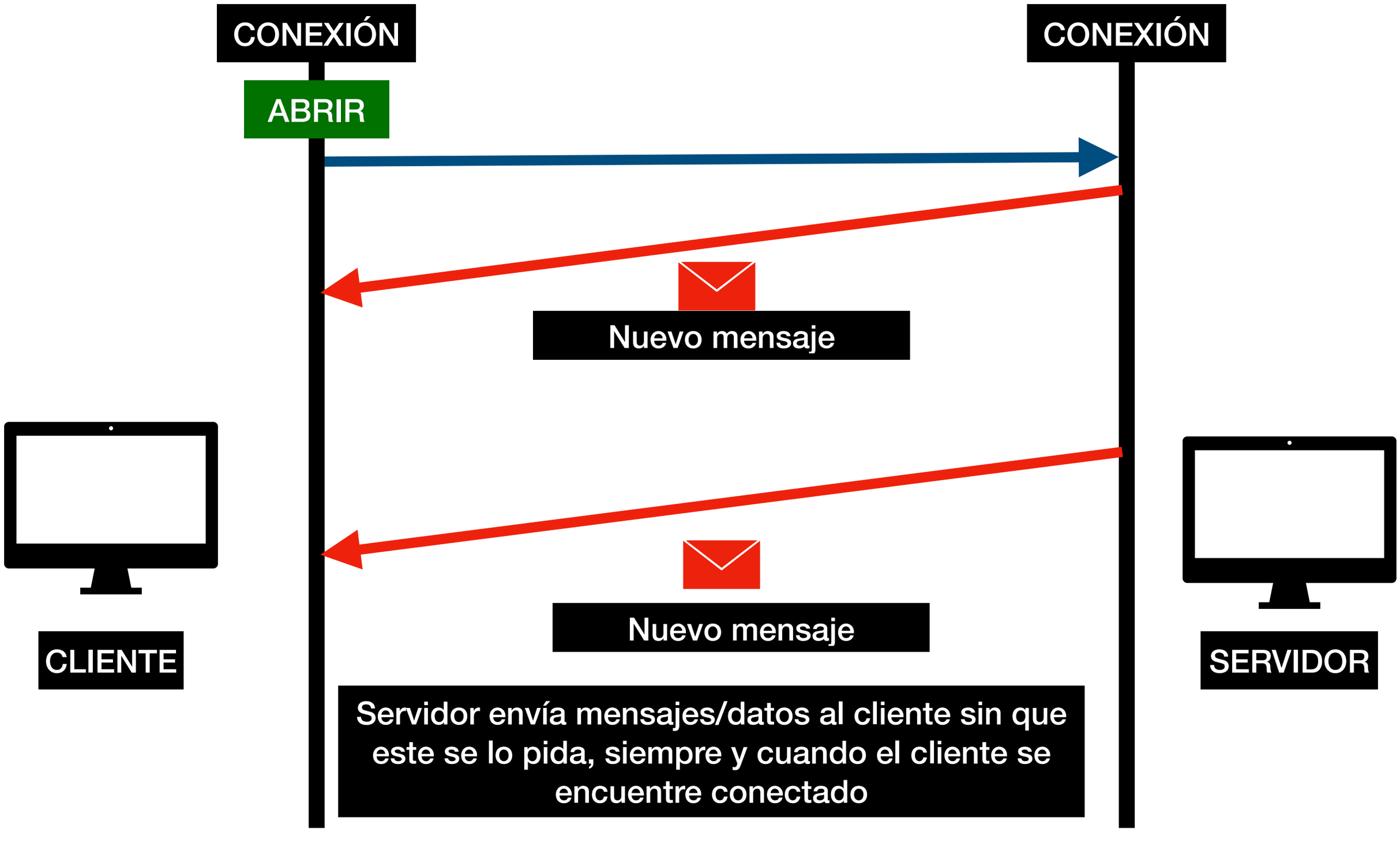
Sync vs Async

Modelo Request Response

- Sincronía es una propiedad del cliente.
- El modelo Request Response en su esencia es **SÍNCRONO (sync)**.
- PERO, al día de hoy la tecnología tiende a ser **ASINCRONA (async)**.
  - Por lo que la sincronía llega a ser una ilusión.
  - Sin embargo, todo depende del contexto. Por ejemplo: realizar una transacción bancaria debe ser síncrono, no puedo permitir al usuario seguir operando con su cuenta hasta que termine la transacción.

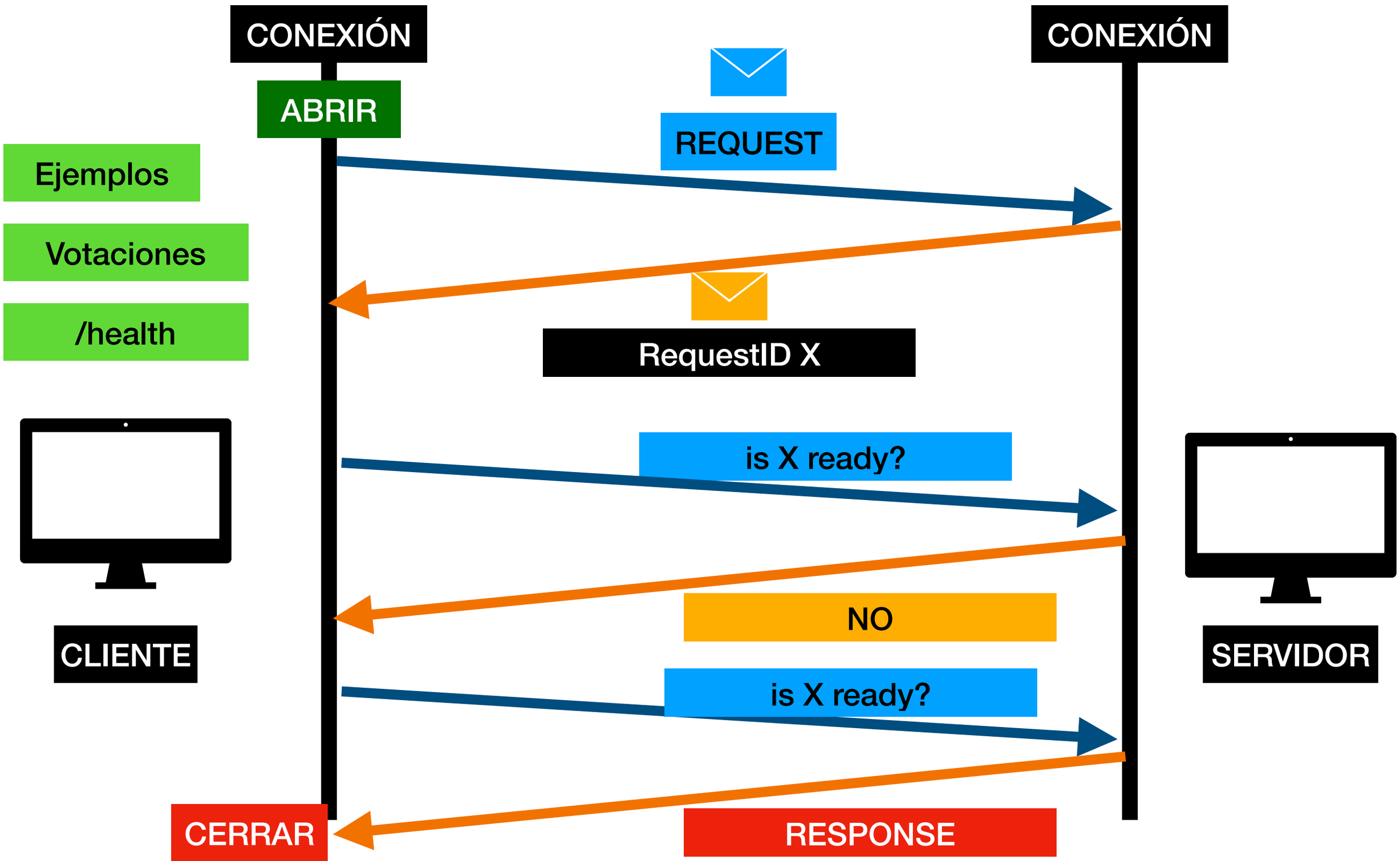
Modelo Push

Lo quiero lo más pronto posible (ej: notificación)



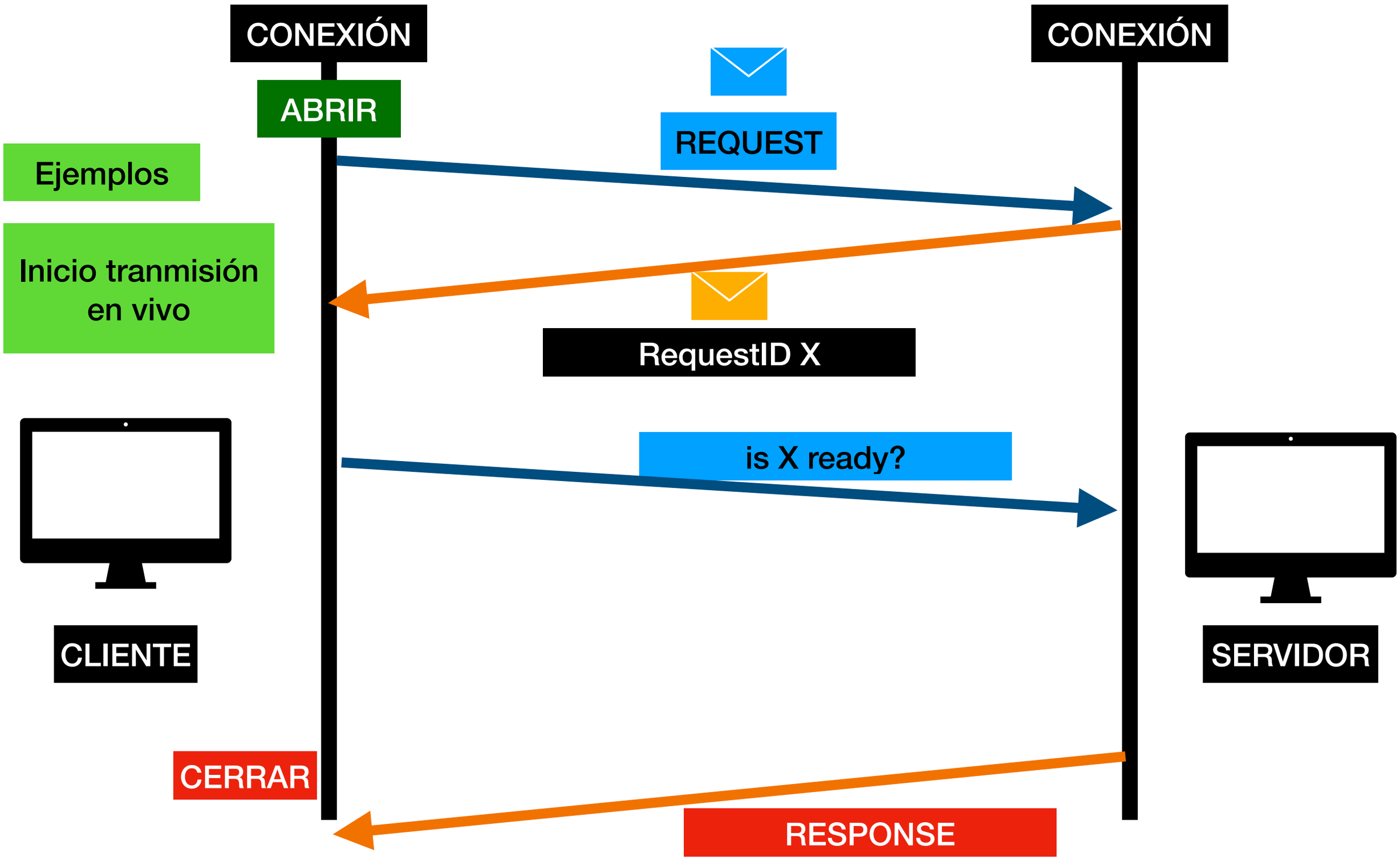
Short Polling

Pregunto constantemente si mi petición esta completa



Long Polling

Request toma demasiado tiempo. Te pregunto más rato. Pero hablame sólo cuando termines



Ejemplo: Librería para conectarse a GCP

polling.py

myenv/lib/python3.12/site-packages/google/api_core/future/polling.py

```
143
144     def result(self, timeout=_DEFAULT_VALUE, retry=None, polling=None):
145         """Get the result of the operation.
146
147         This method will poll for operation status periodically, blocking if
148         necessary. If you just want to make sure that this method does not block
149         for more than X seconds and you do not care about the nitty-gritty of
150         how this method operates, just call it with ``result(timeout=X)``. The
151         other parameters are for advanced use only.
152
```

polling.py

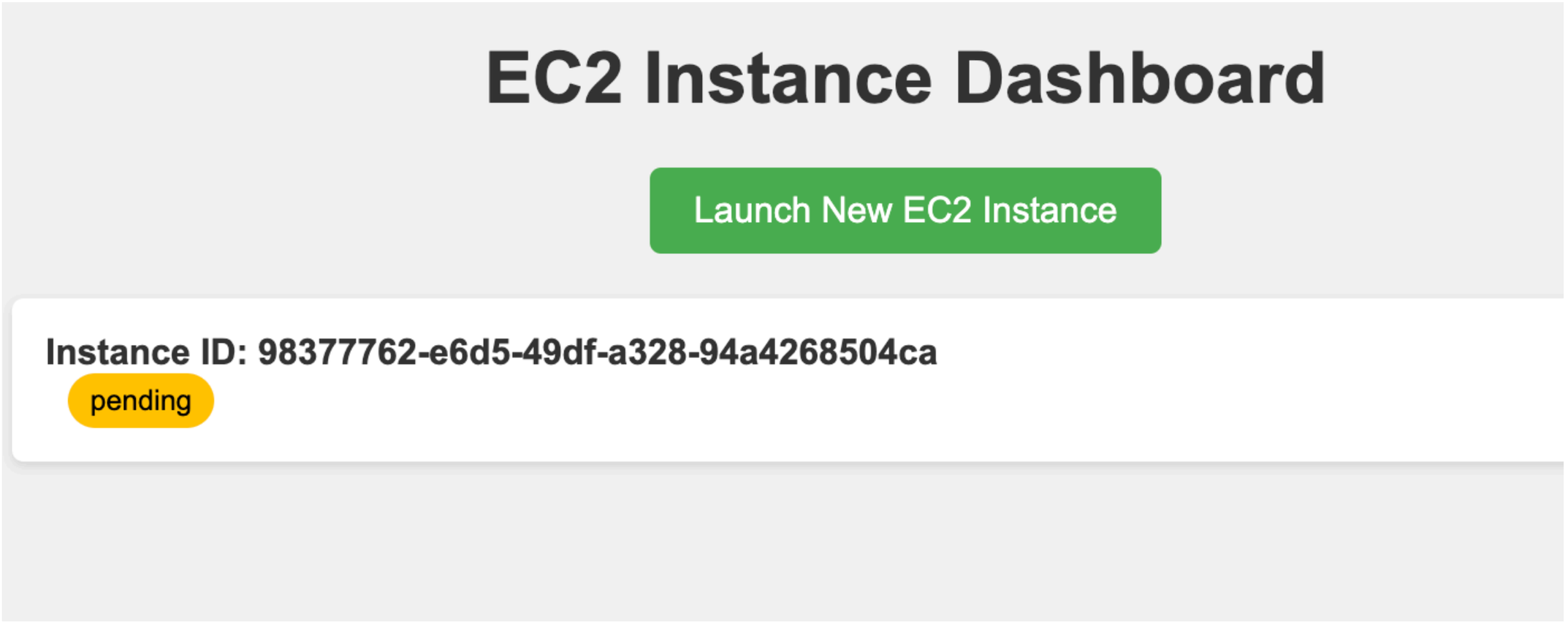
myenv/lib/python3.12/site-packages/google/api_core/future/polling.py › class PollingFuture

```
255
256         self._blocking_poll(timeout=timeout, retry=retry, polling=polling)
257
258         if self._exception is not None:
259             # pylint: disable=raising-bad-type
260             # Pylint doesn't recognize that this is valid in this case.
261             raise self._exception
262
263         return self._result
```

Prototipo

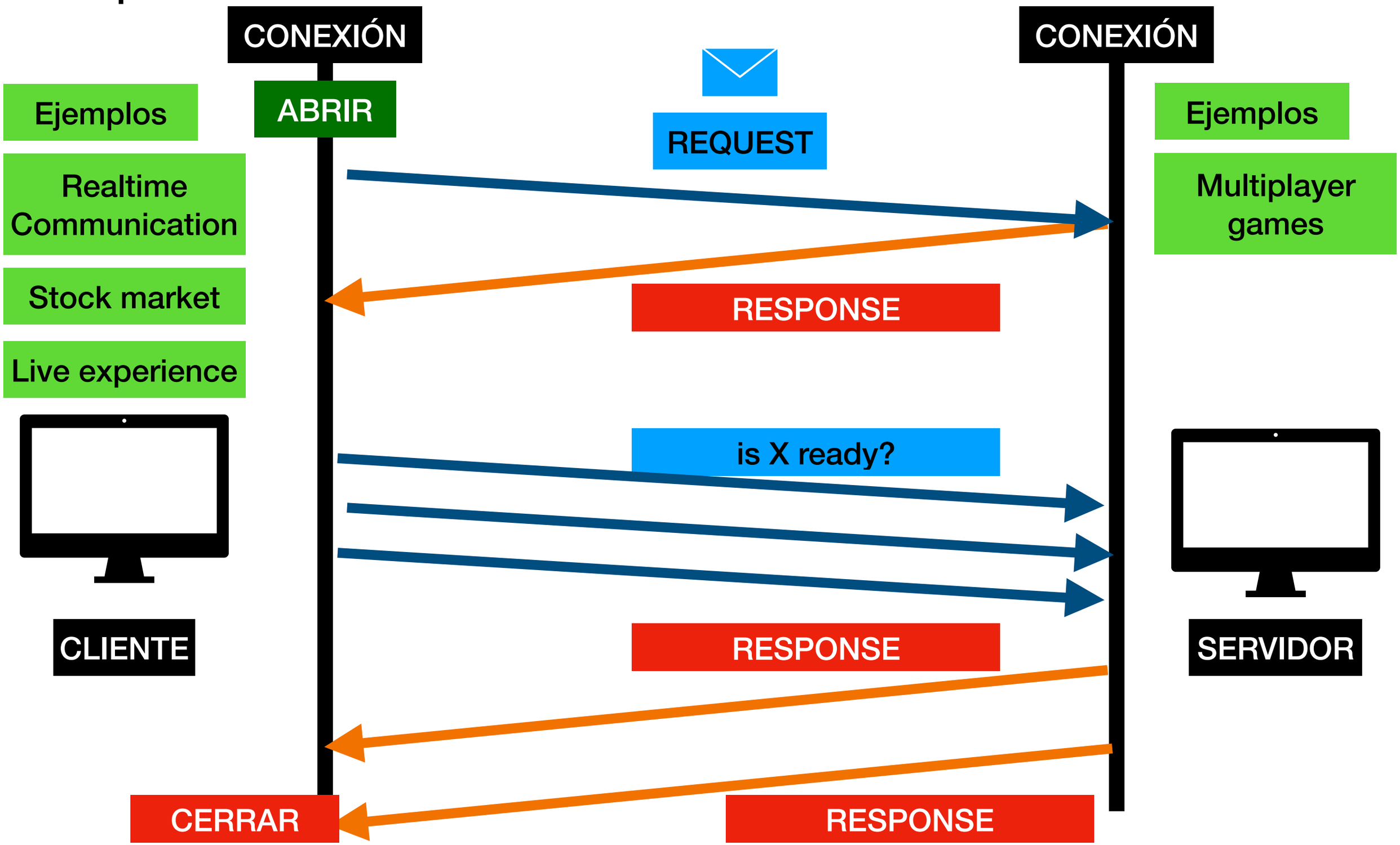
Polling

https://github.com/UC-IIC-3143-Software-Development/prototypes/tree/main/03-polling



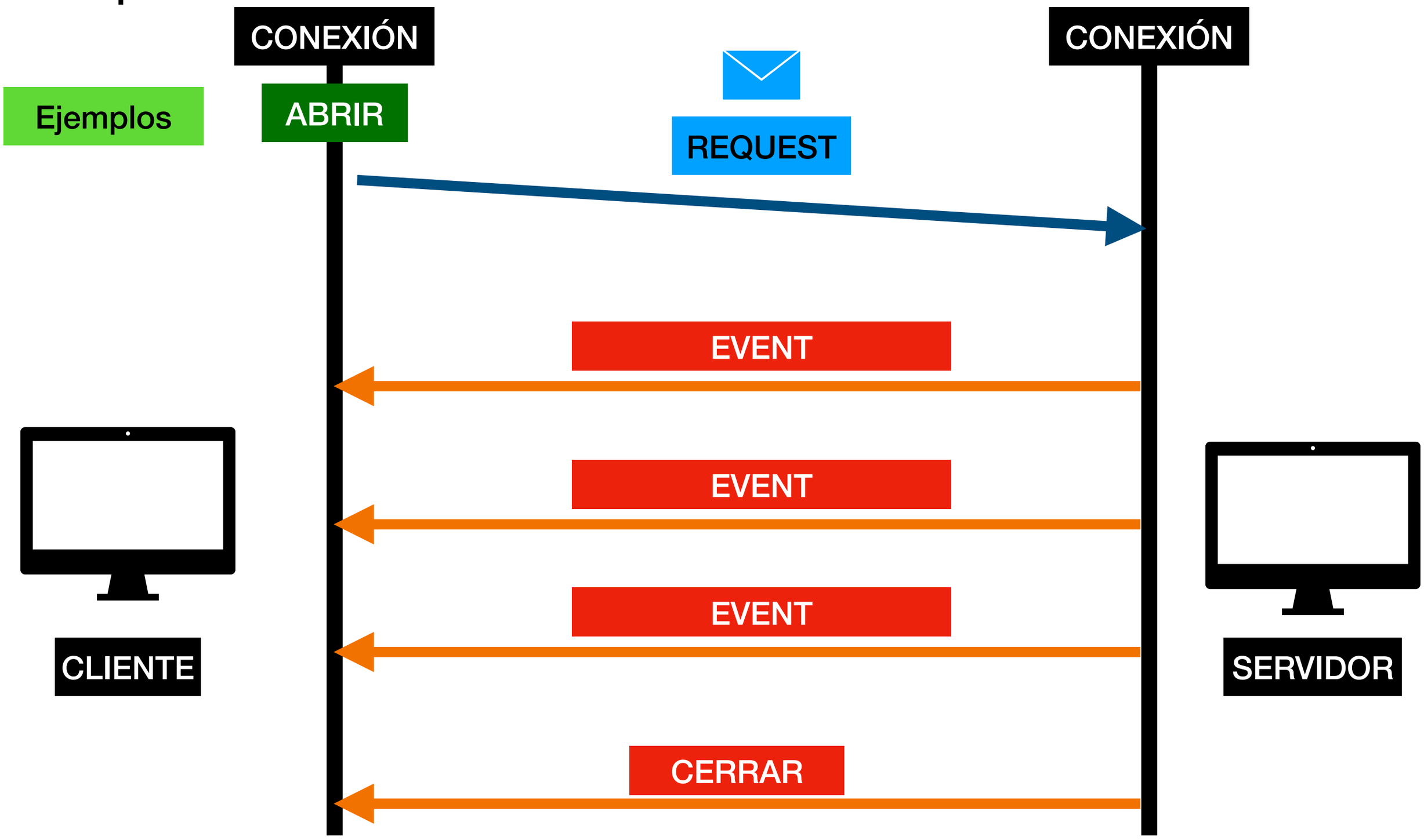
WebSockets

Mantengo una conexión abierta persistente, no hay sincronia entre Request y Response



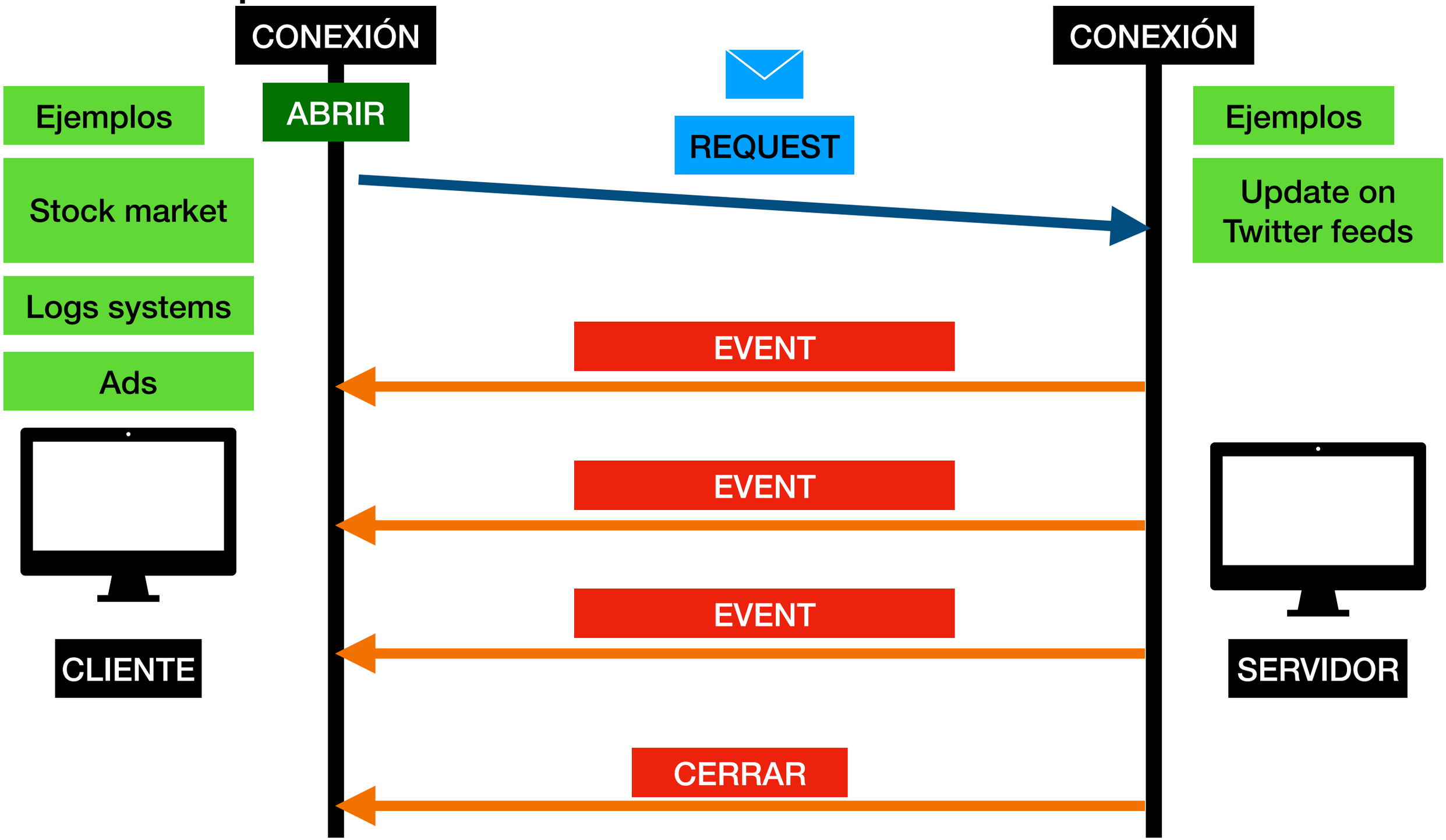
Server-Sent Event

Cliente establece conexión con servidor y este envia eventos cuando quiere. Cliente acepta eventos



Server-Sent Event

Cliente establece conexión con servidor y este PROACTIVAMENTE envia eventos.
Cliente acepta eventos



Prototipo

Server-Sent Event

https://github.com/UC-IIC-3143-Software-Development/prototypes/tree/main/04-streaming-logs

CI/CD Pipeline Dashboard

ID: ca206fb4-170f-4aad-8a8e-2fee1aceeeac | [2025-03-16 22:48:13] Project: frontend | Stage: test | Environment: development | Status: FAILURE | Duration: 192s

ID: 51d2a8b3-6a0b-481f-9edb-a5b4b9017f7f | [2025-03-16 22:48:10] Project: frontend | Stage: deploy | Environment: production | Status: FAILURE | Duration: 168s

ID: 98fac46a-5752-4481-98fc-a120919cc855 | [2025-03-16 22:48:07] Project: data-service | Stage: rollback | Environment: development | Status: SUCCESS | Duration: 106s

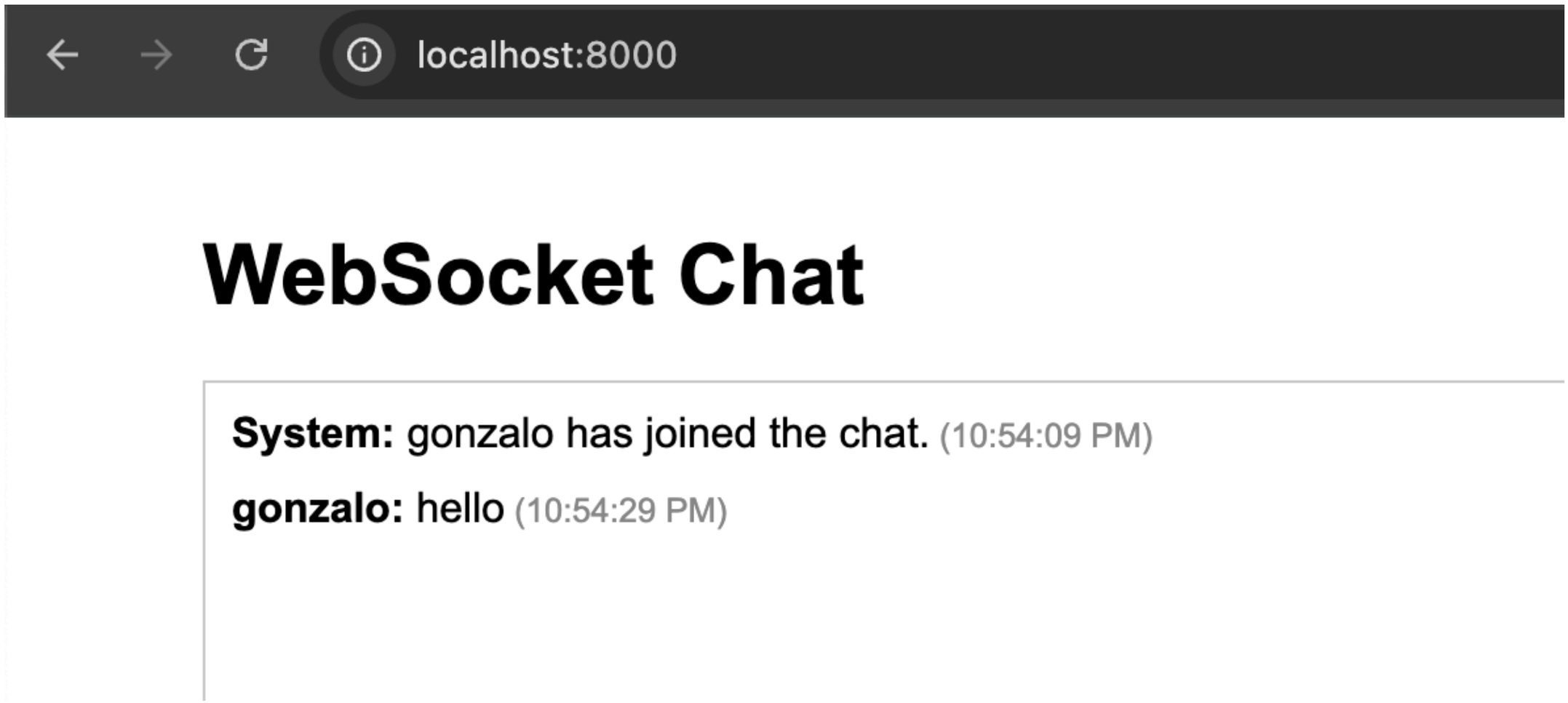
ID: 296d5ac9-51ed-496a-b654-9dda06550293 | [2025-03-16 22:48:04] Project: frontend | Stage: build | Environment: production | Status: SUCCESS | Duration: 265s

ID: a869e856-c511-42bd-bcc2-2f563c3ad850 | [2025-03-16 22:48:01] Project: auth-service | Stage: rollback | Environment: development | Status: FAILURE | Duration: 172s

ID: e731650a-42f8-4b86-ac0b-2a8e618f8a0b | [2025-03-16 22:47:58] Project: data-service | Stage: rollback | Environment: production | Status: SUCCESS | Duration: 48s

Pista: Prototipo Chat websocket

https://github.com/UC-IIC-3143-Software-Development/prototypes/blob/main/05-ws-chat-broadcasting/ws-chat-server.py



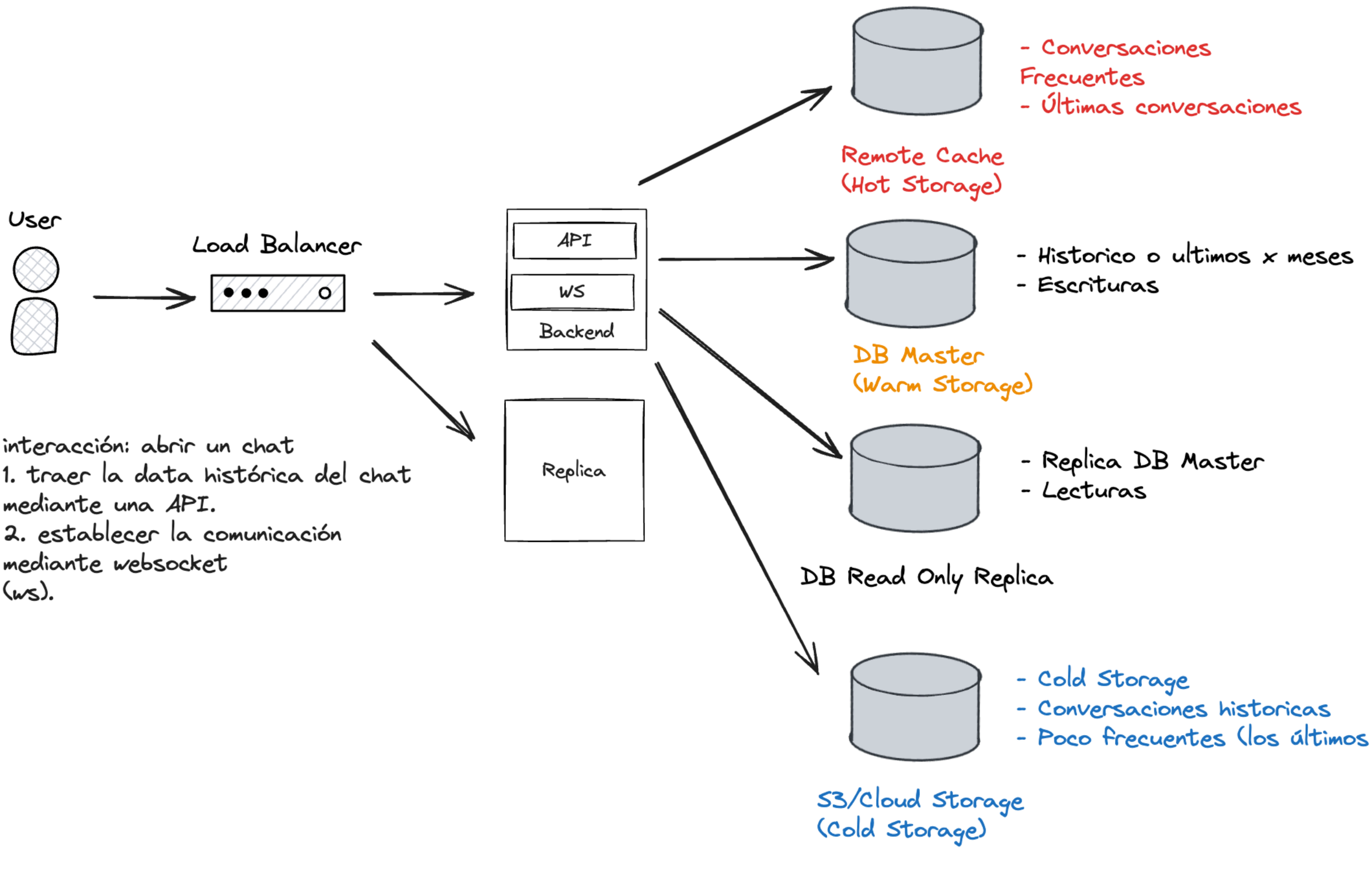
Ejercicio:

Diseñemos un sistema como ChatGPT

Solución en pizarra

Posterior a la clase actualizo la slide

Chat (Google Meet/Slack/Discord)



¿Qué aprendimos hoy?

- Toma de decisiones técnicas en base a costo
  - Hacer la pregunta: ¿Realmente lo necesito?
- Poner en perspectiva los costos por cada decisión de arquitectura (tradeoffs)

- Hoy nos centramos en:
  - Costo de comunicación
  - Cache (performance vs costo)

Siguiente clase profundizaremos en base de datos

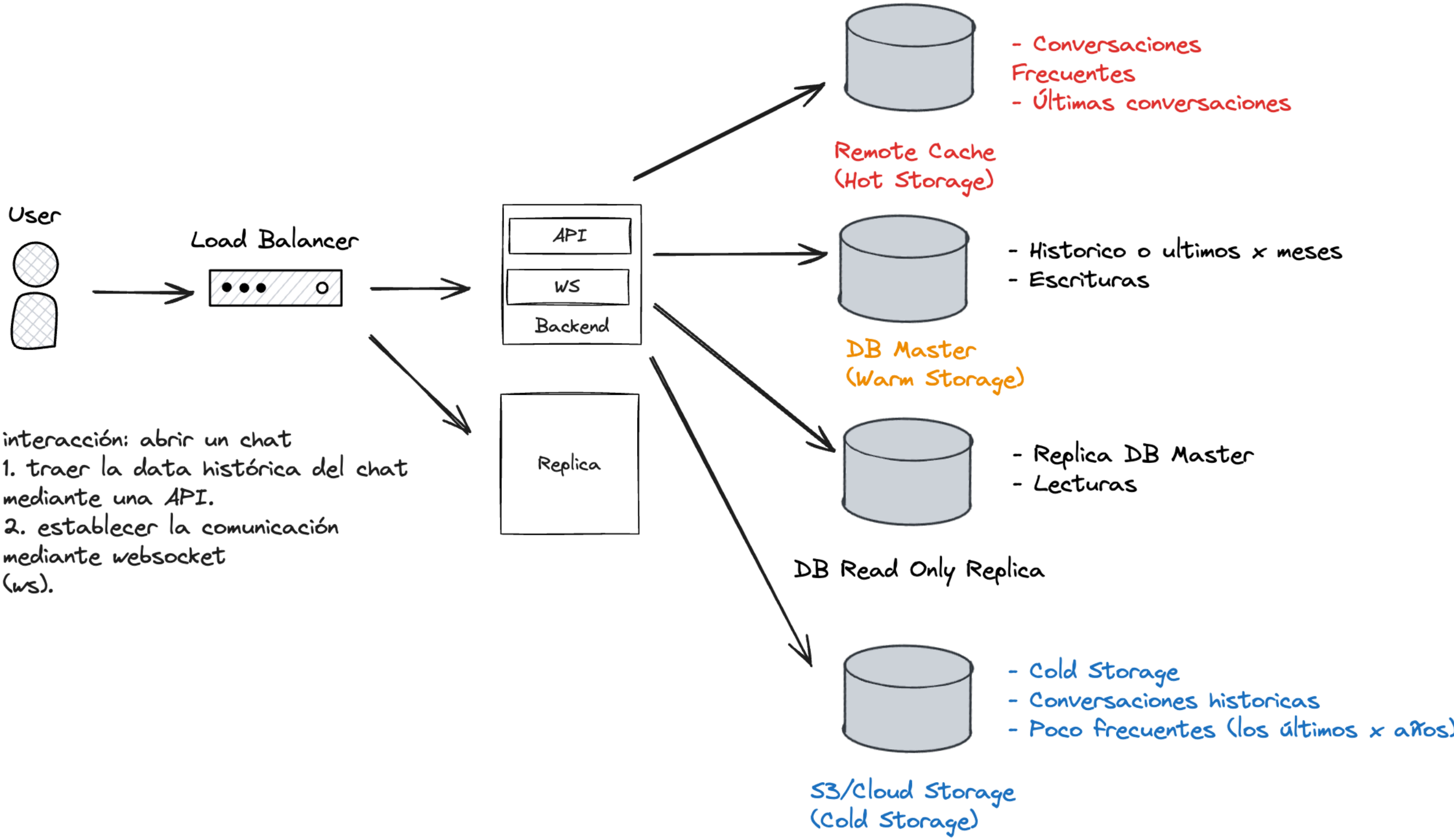
Introducción a System Design - Clase 4

Base de Datos Parte 1
Modelado de datos
8.20 - 9.30 am



Gonzalo Martinez, 2025

Diseño Última Clase
Chat (Google Meet/Slack/Discord)



Contenido hoy

- Diseño de modelo en base de datos
- WAL (Write Ahead Log)
- Connection Pool
- ¿Cómo decidir el tipo de base de datos?
  - Relacional vs No Relacional

Base de Datos - Schema

Primero necesitamos pensar en los valores a ocupar

Diseño de sistema: sistema de blog (ejemplo: medium.com)

Antes de pensar en la tecnología, pensamos en el caso de uso

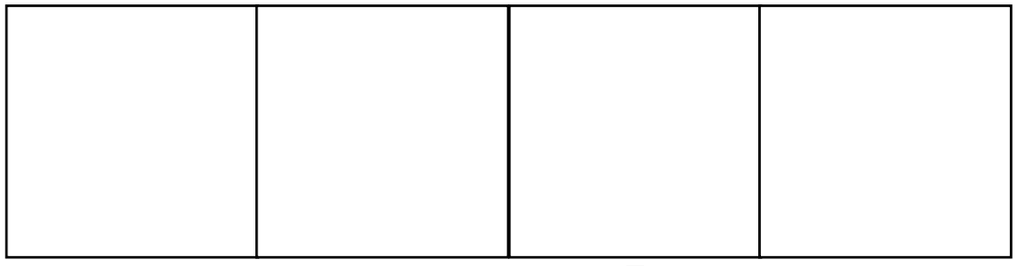
Users
id
name
bio

Blogs
id
author_id
title
is_deleted
published_at
body

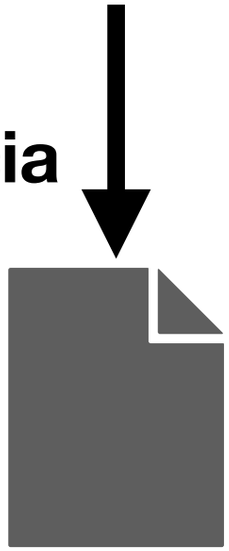
Base de Datos - Strings

Columna: bio vs body

Long Text



Almacenado como referencia



1 Forma: Text

Máximo 1 GB

Short Text



Se almacena junto con los demás atributos

2 Formas:
char(máximo)
varchar(máximo)

Máximo depende de BD

Char, Varchar y Text

Las bases de datos por lo general proveen 3 tipos de datos para almacenar texto

- **Char(n)** : almacena un string hasta n caracteres.
  - Si se intenta almacenar string > n caracteres
    - **Arroja error (a menos que los caracteres extra sean espacios)**

- Ejemplo:
  - name char(5)
    - Insertar “Gon” -> Se almacena como: “Gon_ _” (_ son espacios, imaginen 3 espacios).
    - Insertar “Gonzalo” -> Error
    - Insertar “Gon_ _ _” (3 espacios) -> Se almacena como: “Gon_ _”

Char, Varchar y Text

Las bases de datos por lo general proveen 3 tipos de datos para almacenar texto

- **Varchar(n)** : lo mismo que char(n) pero sin agregar espacios (es decir, sin padding)
    - Ejemplo:
        - name varchar(5)
            - Insertar “Gon” -> Se almacena como: “Gon”
            - Insertar “Gonzalo” -> Error
- Límite máximo de n: 10 MB (10x1024x1024) -depende de la BD-
- **No hay razones para usar char en vez de varchar (ni por computo o eficiencia de storage).**

Char, Varchar y Text

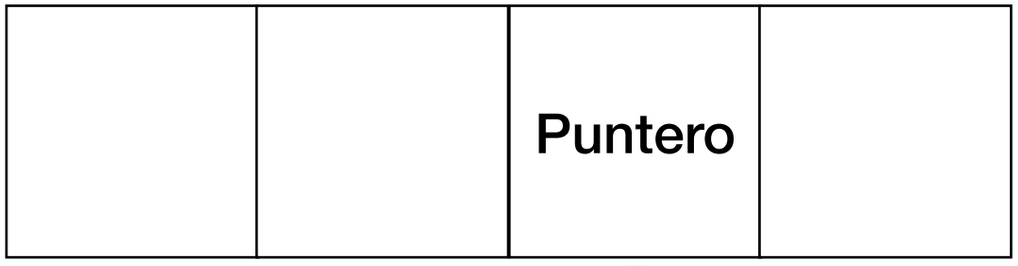
Las bases de datos por lo general proveen 3 tipos de datos para almacenar texto

- **Text** : permite almacenar un string de cualquier largo.
    - Cada BD provee una implementación propia de Text.
    - Límite máximo de Text: 1 GB (10x1024x1024x1024)
    - **Nota: no es recomendado crear Llave Primaria o Llave Foránea (FK) o index en columnas TEXT**

¿Cómo se almacena Text en el disco?

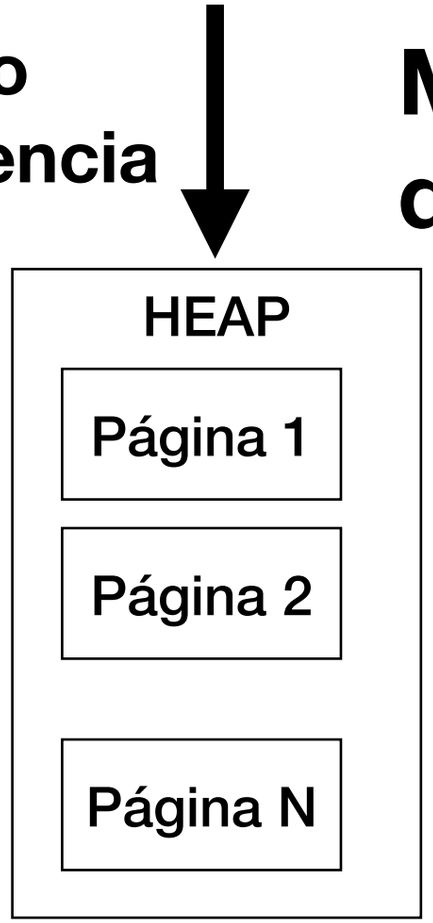
Depende del largo. Importante: Varchar funciona igual

Long Text



Almacenado como referencia





Más costoso de acceder*

Short Text



Se almacena junto con los demás atributos (inline)

* = No necesariamente

Hay 3 mitos referentes a Text:

1. VARCHAR siempre se almacena inline con el resto de atributos = FALSO
2. TEXT siempre se almacena cómo punteros (pointers) = FALSO
3. Varchar es menos costoso que Text = FALSO

Varchar vs Text

Al día de hoy, no hay diferencia de performance real entre ambos

¿Entonces cuál ocupar?

1. Para strings, usar Varchar.
    1. username, email, name, address, etc.
2. Para texto, usar Text.
    1. bio, body, description, etc.
3. Para validaciones de largo máximo, usar Varchar.
4. Para Llave foranea (FK) usar Varchar.

Base de Datos - Fechas

Datetime vs Timestamp

Date: almacena fecha pero no tiempo.

(1000-01-01 a 9999-12-31)

Datetime: almacena fecha y tiempo -Soporta microsegundos-

(1000-01-01 00:00:00 a 9999-12-31 23:59:59)

Timestamp: almacena fecha y tiempo (usando Unix Epoch como integer) -Soporta microsegundos-

(1970-01-01 00:00:00 UTC a 2038-01-19 03:14:07 UTC)

= 0 a 2^32 segundos

Base de Datos - Fechas

Costo en espacio

Tipo de dato	Tamaño (bytes)
Date	3
Datetime	5 + fractional seconds
Timestamp	4 + fractional seconds

Base de Datos - Fechas

¿Cuando usar Datetime?

Cuando queremos almacenar un caso de uso con un tiempo especifico.

ej: appointment, schedule_at

Cuando queremos realizar cálculos dentro de la base de datos (ej: MySQL).

ej: DATE_ADD(purchased_at, ‘INTERVAL 1 DAY’)

Cuando queremos un formato human-readable y fácil de procesar

Costo: 5 bytes (MySQL)

Desventaja: No fuerza formato UTC

Base de Datos - Fechas

¿Cuando usar Timestamp?

- **Usar timestamp cuando queremos crear un sistema de registros/ records basado en el tiempo. Timestamp se almacena como int (4 bytes) y representa el tiempo en UTC.**

  - **ej: created_at, transaction_time, etc**

- **La conversión de zonas horarias (Timezones) sucede cuando la consulta retorna un valor y cuando la consulta almacena el valor.**

- **El valor de retorno cambia dependiendo de la zona horaria de la conexión (cliente quién realiza la consulta).**

Costo: Timestamp 4 bytes (MySQL) vs Datetime (5 bytes)

Desventaja: Valor de retorno cambia dependiendo del cliente

Base de Datos - ENUM

Enumeraciones, Categorías - ENUM

- **El tipo de dato ENUM sirve para definir un tipo de columna que puede contener un valor de una lista predefinida de opciones. Es útil cuando se tiene un conjunto limitado de opciones, cómo por ejemplo categorías o una máquina de estados.**

```sql
CREATE TABLE pedidos (
    id INT PRIMARY KEY AUTO_INCREMENT,
    cliente VARCHAR(100),
    estado ENUM('pendiente', 'enviado', 'entregado', 'cancelado') DEFAULT
'pendiente'
);

INSERT INTO pedidos (cliente, estado) VALUES
('Juan Pérez', 'pendiente'),
('María García', 'enviado'),
('Carlos López', 'entregado');
```

Base de Datos - ENUM

Enumeraciones, Categorías - ENUM

```sql
CREATE TABLE pedidos (
    id INT PRIMARY KEY AUTO_INCREMENT,
    cliente VARCHAR(100),
    estado ENUM('pendiente', 'enviado', 'entregado', 'cancelado') DEFAULT
'pendiente'
);
```

```sql
INSERT INTO pedidos (cliente, estado) VALUES
('Juan Pérez', 'pendiente'),
('María García', 'enviado'),
('Carlos López', 'entregado');
```

Beneficio: BD almacena el valor como número (4 bytes) y no varchar

Beneficio: Sólo se puede almacenar valores predefinidos

Desventaja: En ocasiones el ORM no soporta la conversión a ENUM

Base de Datos

Schema - Primera aproximación

User
id: PK int
name: varchar(50)
bio: varchar(100)
is_deleted: bool
last_updated: datetime

Blog
id: PK int
author_id: FK int
title: varchar(30)
category: enum
is_deleted: bool
published_at: datetime
last_updated: datetime
body: varchar(>255) o quizás Text

Primary Key: int autoincremental

Field in database: int id AUTOINCREMENT

- **Problemas:**
  - **Github Outage 2020: Integer Overflow in Database (May 5)**

May 5 00:45 UTC (lasting for two hours and 24 minutes)

During the incident, a shared database table's auto-incrementing ID column exceeded the size that can be represented by the MySQL Integer type (Rails int(11)): 2147483647. When we attempted to insert larger integers into the column, the database rejected the value and Rails raised an ActiveModel::RangeError, which resulted in 500s from our API endpoint.

This impacted GitHub apps that rely on getting installation tokens. The top affected GitHub apps internally included Actions, Pages, and Dependabot.

GitHub's monitoring systems currently alert when tables hit 70% of the primary key size used. We are now extending our test frameworks to include a linter in place for int / bigint foreign key mismatches.



https://github.blog/2020-07-08-introducing-the-github-availability-report/

Github Outage 2020

Integer Overflow in Database

¿Cómo Github lo soluciono?
No se menciona.

Posible soluciones:

1. Cambiar a ID Unsigned / BigInt (es lo mismo)

Unsigned/Big Int es útil cuando la tabla es pequeña y no se usan ID's negativos

Range: SIGNED: -2^31 a 2^31 - 1
UNSIGNED: 0 a 2^32 - 1

2. Mover todo a una nueva tabla

Required Storage for Int

MySQL

https://dev.mysql.com/doc/refman/8.3/en/integer-types.html

Table 13.1 Required Storage and Range for Integer Types Supported by MySQL

Type	Storage (Bytes)	Minimum Value Signed	Minimum Value Unsigned	Maximum Value Signed	Maximum Value Unsigned
TINYINT	1	-128	0	127	255
SMALLINT	2	-32768	0	32767	65535
MEDIUMINT	3	-8388608	0	8388607	16777215
INT	4	-2147483648	0	2147483647	4294967295
BIGINT	8	-2^{63}	0	$2^{63}-1$	$2^{64}-1$

Hay algunas base de datos especializadas que proveen el tipo de dato: varint -> NO ES EL CASO DE MySQL, PostgreSQL, MongoDB

Github Outage 2015

Integer Overflow in Database

Un valor autoincremental se detiene cuando el valor máximo es alcanzado

En el caso de integer:
1, 2, 3, 4, 5, 6… 100, …. 2147483647, 2147483647, 2147483647, 2147483647, ….

MySQL: 2147483647 + 1 = 2147483647

Problema Github Outage 2015: cada nuevo registro se almacena con el mismo ID

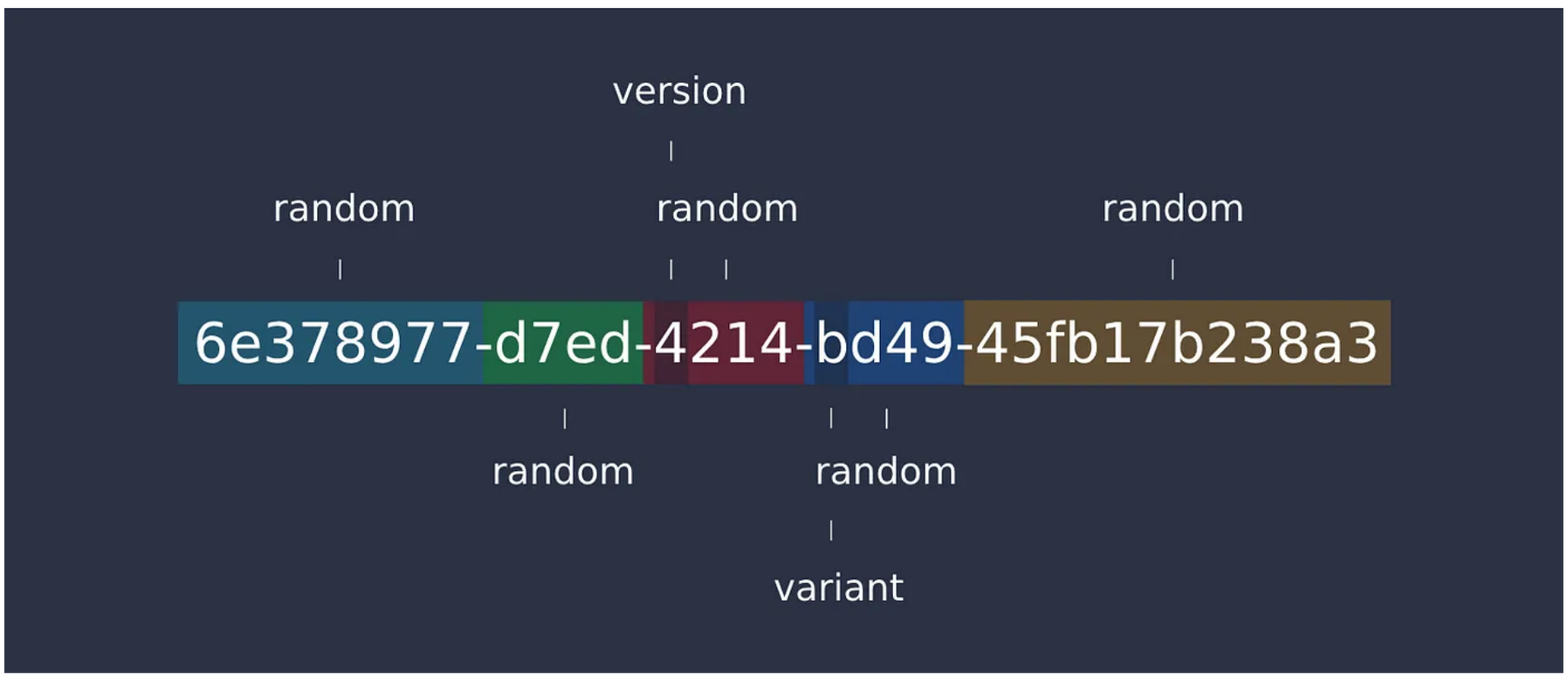
DUPLICATE KEY ERROR

Primary Key: UUID

UUID4 es la versión UUID más utilizada

- **Ventaja: Baja probabilidad (cercano a 0%) de colisión o repetir valores.**
  - **Ideal para sistemas distribuidos y microservicios**
- **Desventaja: Costoso (CPU + Memoria). Index bloat -> Más storage y costo I/O**

Próxima clase: ejemplo práctico análisis Primary Key



Database

No Relacional (NoSQL)

- La mayoría de las bases de datos no relacionales proveen ***sharding*** (particionamiento) -escalabilidad horizontal- out-the-box.

- **Existen 3 tipos de bases de datos no relacionales:**

  - **Documentales, Key-Value, Graph Databases**

  - Documentales: MongoDB, ElasticSearch

  - Key-Value: Redis, DynamoDB, Browser LocalStorage

  - Graph (costosas en términos de dinero $$$): Neo4J

Database

No Relacional (NoSQL) - Documental

- Documentales: MongoDB, ElasticSearch
    - Mayoritariamente JSONs
    - Soporta consultas complejas (al igual que bases de datos SQL): ejemplo aggregations
- Partial Updates donde es posible.
    - Total_posts += 1
- Casos de uso:
    - Notificaciones in-app, servicios de catalogos

```
{
   "user_id": 1,
   “total_posts": 10
}
```

Database

Mito: BD Relacionales no escalan

¿Por qué las BD no relaciones “escalan”?

- **No existen relaciones y constantes entre tablas**
- **Los datos se modelan para no requerir interdependencia**

- **Si relajamos el requerimiento anterior, las BD relacionales también escalan (igual de bien que BD no relacional)**
    - No usar llaves foráneas (FK) ni validaciones entre tablas.
    - Minimizo el uso de relaciones entre tablas.

Database

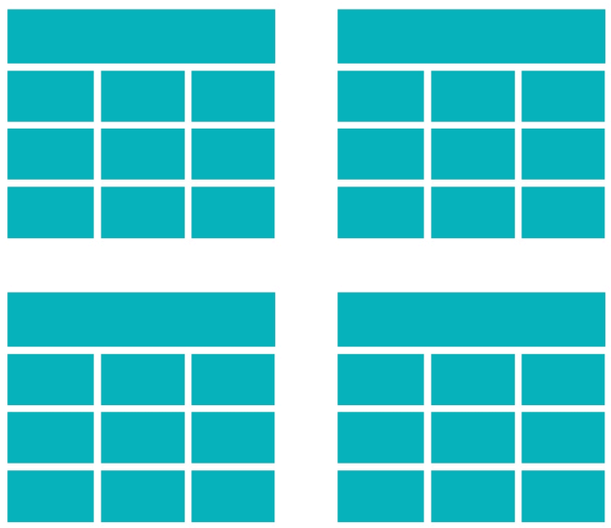
Key-Value (ejemplo: Redis, DynamoDB, LocalStorage)

- Extremadamente simples.
- Caso de uso: cache.
  - Hashmap: O(1)
  - Funcionalidades limitadas (GET, PUT, DEL) *
    - Redis soporta funcionalidades más complejas.
  - No soporta consultas complejas (agreggations)
- Casos de uso: Profile data (perfiles), Order data (ordenes), Auth data, mensajes (mayoritariamente se usa para mensajes).
  - Hot Storage.

Database

Relacional (SQL) vs No Relacional (NoSQL)

- No es una pelea, no necesitamos elegir un “bando”.
- **Cada base de datos esta diseñada para resolver realmente bien un problema en particular.**
- Actualmente la separación entre features es difusa (overlap)
- (ej: PosgreSQL soporta documentos en v9.3, MySQL v8)



Filas, Columnas (tablas)

Relacional



No Relacional

Documentos

Database

Connection Pool

- Existe un límite de conexiones simultaneas que puede tener una base de datos. Nada es ílimitado.

- MySQL: máximo de 150 conexiones simultaneas + reserva 1 conexión para un admin (Superusuario) = 151 conexiones

- PostgreSQL: máximo de 100 conexiones (configurable en archivo pgconf).

- MongoDB: máximo de 65.536 conexiones (configurable)

- **Estos límites son los teoricos, en la práctica el sistema operativo limita en base a los recursos de la máquina.**

- **Si las base de datos tienen limites de conexiones que sucede si se alcanza este límite?**

  - **Exception: Max connections reached**

  - **La base de datos rechaza conexiones entrantes**

Database

Connection Pool

- **Solución: Connection Pool**
- **Mantiene un conjunto de conexiones disponibles para que la BD lo reuse.**
  - **MySQL se puede realizar mediante configuración**
  - **PostgreSQL es necesario usar un componente distinto a la BD en la misma máquina, PGBouncer.**
- **IMPORTANTE: EN PRODUCCIÓN NECESITAMOS CONNECTION POOL**

Database

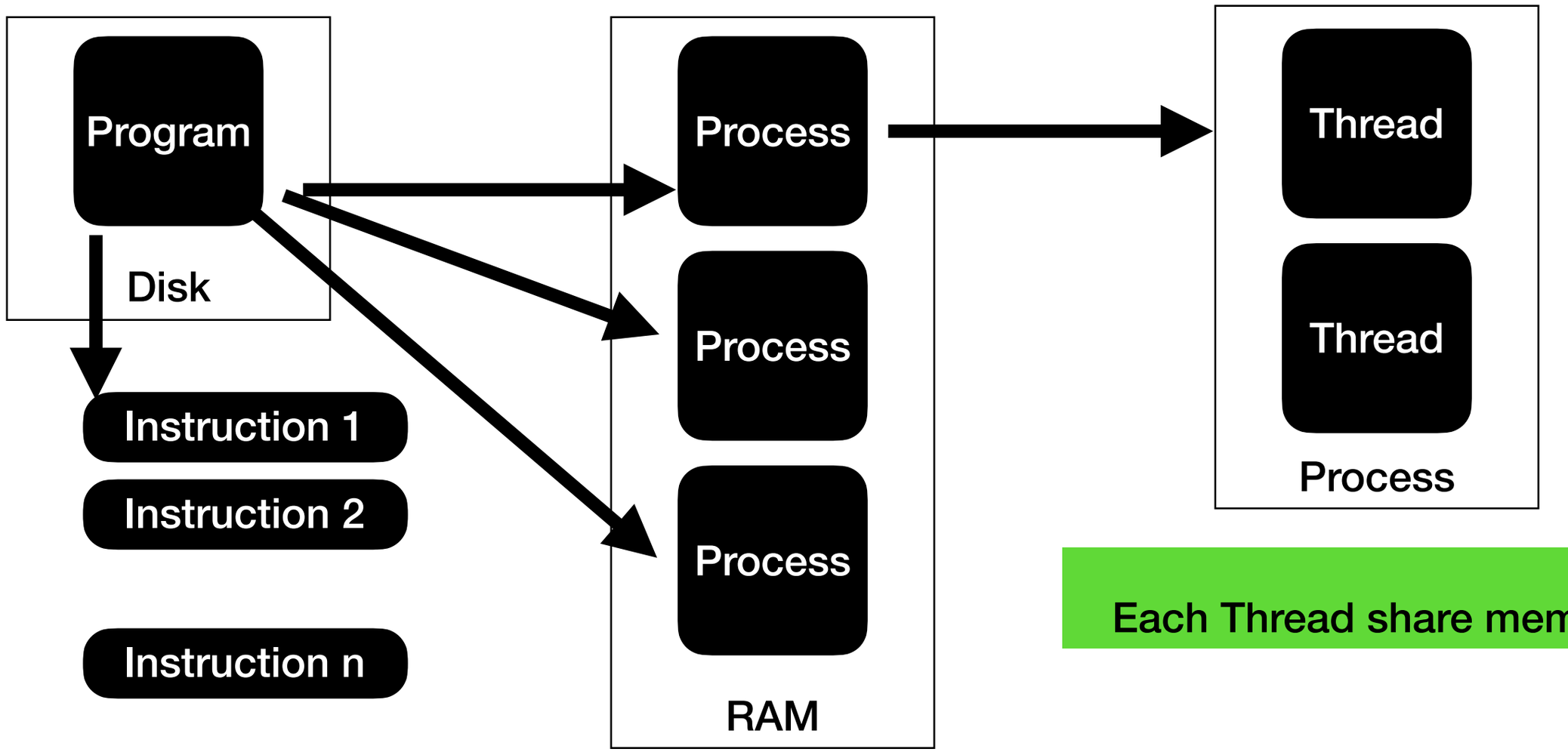
Connection Pool

- **PostgreSQL: Cada conexión es un PROCESO (PROCESS) dentro del sistema operativo.**
- **MySQL: Cada conexión es un THREAD.**
- **MongoDB: Cada conexión es un THREAD.**
- **Cada enfoque tiene tradeoffs (discusión en clases).**
- **Crear un Proceso es más costoso computacionalmente que crear un Thread a nivel del sistema operativo.**

```
# Ejemplo configuración con Python y MySQL Connector
import mysql.connector
from mysql.connector import pooling
connection_pool = pooling.MySQLConnectionPool(
    pool_name="counter_pool",
pool_size=32, **db_config
)
```

Sistemas Operativos

Thread vs Process



Each Thread share memory

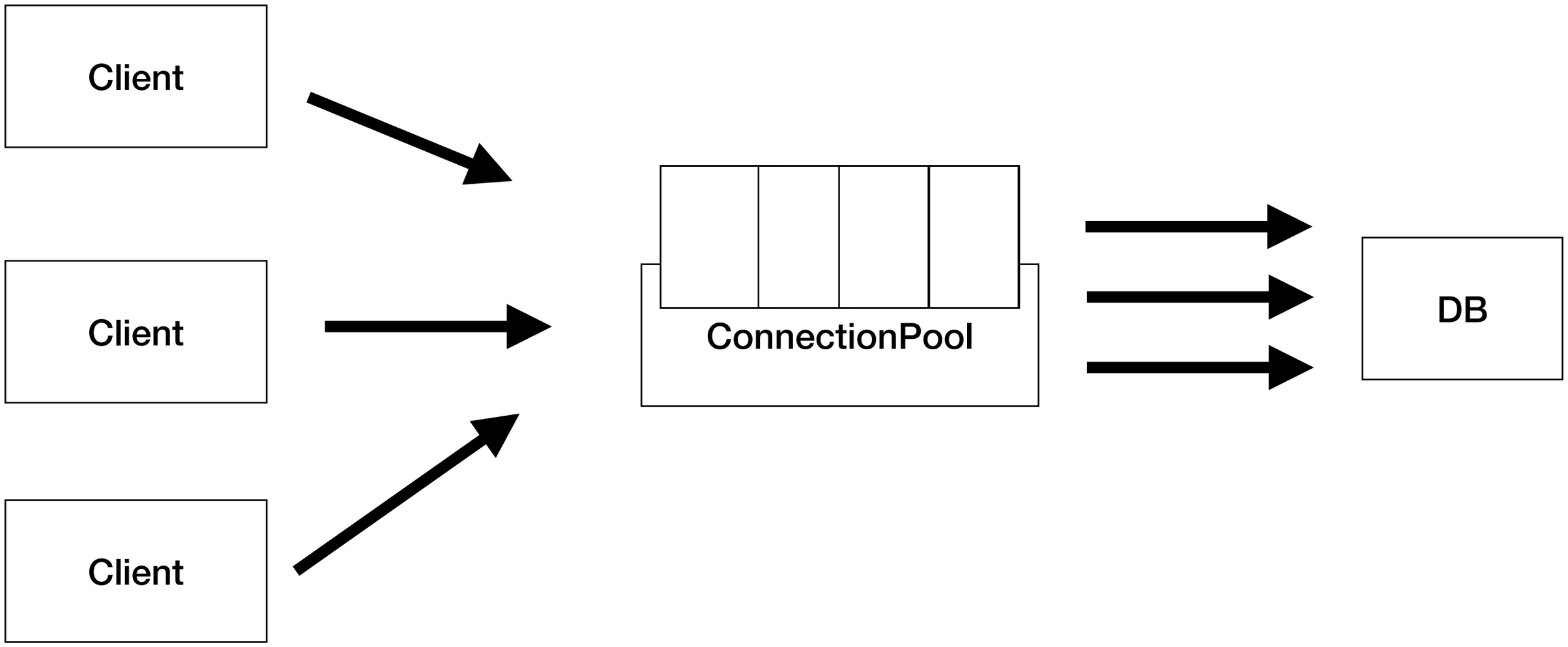
Each process is memory islated from another

Create a process is more CPU costly

Communication between IPC (InterProcess Communication)

Database

Connection Pool



Base de datos

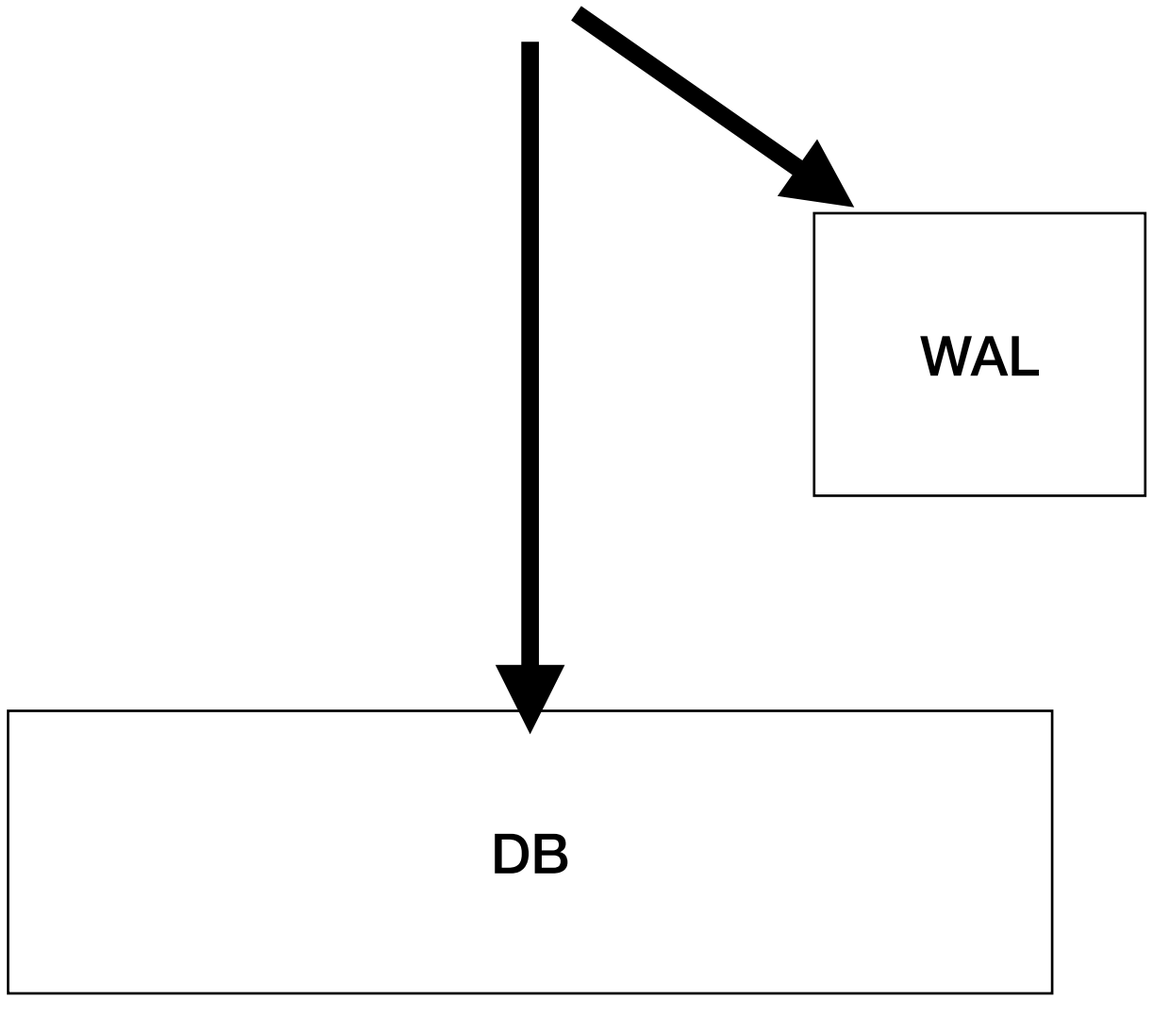
Transacciones - Write Ahead Log (WAL)

Cada operación en la base de datos se le llama Transacción (Tx).

Al momento de enviar una consulta (query) a la BD, este crea un Query Plan y cálcula si la consulta provoca deadlock, dependiendo de la BD y la configuración probablemente cancele la consulta.

Al momento que llega la query, la BD escribe la operación en un log (Write Ahead Log, WAL) escrito en disco.

MySQL lo llama: RedoLog y también usa BinLog
PostgreSQL lo llama: WAL



Cada vez que una BD se inicia, lo primero que realiza es verificar la integridad de lo escrito mediante la comprobación del archivo WAL

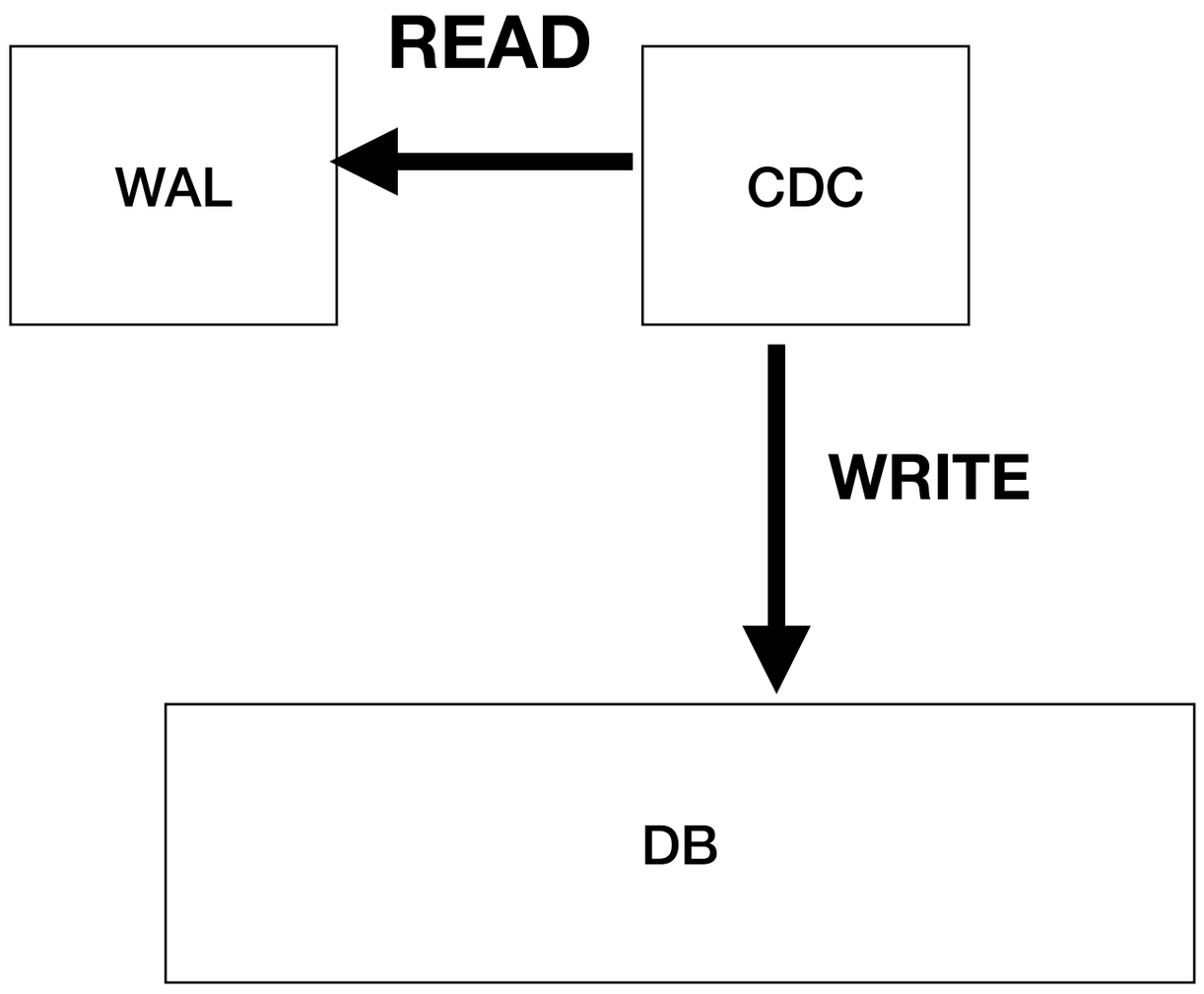
Base de datos

Write Ahead Log (WAL) - Migraciones de datos

Una ventaja de contar con WAL es que las migraciones de datos desde una BD a otra se utiliza un CDC.

Change Data Capture (CDC) es tipo de herramienta que lee el WAL y lo traspasa a otra base de datos.

Ej: Debezium, KafkaConnect, Airbyte



Cada vez que una BD se inicia, lo primero que realiza es verificar la integridad de lo escrito mediante la comprobación del archivo WAL

PostgreSQL

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
);
```

MongoDB

```
{
  "_id": "ObjectId(...)",
  "id": 1,
  "name": “Nombre Apellido",
  "email": "nombre.apellido@example.com",
}
```

Airbyte

Uso de WAL

```
connection_config = {

  "name": "PostgreSQL to MongoDB User Migration",

  "sourceId": postgresql_configuration,

  "destinationId": postgresql_configuration,

  "catalog": catalog,

  "schedule": {

      "units": 24,

      "timeUnit": "hours"

  },

  "syncCatalog": catalog,

  "operationIds": [],

  "status": "active"

}
```

Modelo de Base de Datos

Primero necesitamos pensar en los valores a ocupar

Antes de pensar en la tecnología o si es relacional, no relacional, debemos pensar en el caso de uso

Users
id
name
bio

Blogs
id
author_id
title
is_deleted
published_at
body

Importancia de is_deleted (soft delete)

Cuando un usuario quiere borrar un producto, en vez de BORRAR, ACTUALIZAMOS

Razón: Recoverability, Archival, Audit, Performance (B+ Tree)

Piensen en experiencia de uso (discusión en clase)

Modelo de Base de Datos

Delete - ¿Entonces nunca borramos?

NO, siempre debemos limpiar la base de datos. Debería siempre estar limpia y actualizada.

Con el valor de **is_deleted** podemos llevarlo a otra base de datos (S3) para realizar un archive (archivar).

También podemos borrar, pero esto no debe afectar el SLA con el cliente.

```sql
DELETE FROM product
WHERE is_delete = TRUE
LIMIT 1000;
```

Base de datos

¿Por qué es costoso eliminar un registro de la base de datos?

¿Es costoso insertar o actualizar registros?

Para responder esto primero debemos entender:
¿Cómo las bases de datos almacenan los datos?

¿Cómo elegir una base de datos?

No es un listado exhaustivo pero van a entender la idea

Caso 1: Toda la data cabe en un sólo nodo/máquina

- **¿Necesitas *strong consistency*? y/o *data correctness* es critico?**
  - Relational Database
- **Necesitas consultas complejas, agregaciones?**
  - Relational Database - Tradeoff: pierdes velocidad pero ganas consistencia
    - Pero antes deben definir: ¿qué es una consulta compleja? (discusión en clases)
- **Necesitas un acceso llave/valor pero lo necesitas muy rápido?**
  - Redis - Tradeoff: ganas velocidad pero pierdes consistencia
- **Necesitas un acceso muy rápido a valores recientes?**
  - Redis - Tradeoff: ganas velocidad pero pierdes consistencia

¿Cómo elegir una base de datos?

No es un listado exhaustivo pero van a entender la idea

Caso 1: Toda la data cabe en un sólo nodo/máquina

- **Necesitas estructuras de datos avanzadas y algoritmos complejos?**
  - Redis (georeferential, geo approximations)
  - PostgreSQL (PostGIS para aplicaciones geoespaciales)
  - PostgreSQL (PGVector para BD vectoriales, ej: LLM)

¿Cómo elegir una base de datos?

No es un listado exhaustivo pero van a entender la idea

Caso 2: Toda la data no cabe en un sólo nodo/máquina

- **Has trabajado sólo con SQL?**
  - Relational Databases
- **Necesitas un acceso llave/valor?**
  - MongoDB, DynamoDB (AWS Cloud), Redis como Cache (ElasticCache en AWS, RedisCloud en AWS, MemoryStore in GCP)
- **Necesitas algoritmos de grafos?**
  - Utiliza alguna BD de grafos, Neo4J
- **No sabes que necesitas pero quieres estar preparado para el futuro?**
  - MongoDB

¿Qué aprendimos hoy?

- Modelo de datos
  - Pensar en tipos de datos y su costo
- Connection Pool
- Decisión de tipo de base de datos basado en:
  - Modelado de Datos
  - Uso
  - Componentes Internos (WAL, Connection Pool, Thread, Process)

Siguiente clase profundizaremos en concurrencia en base de datos

Introducción a System Design - Clase 5 Final

Base de Datos Parte 2
Database Internals - Concurrencia
8.20 - 9.30 am



Gonzalo Martinez, 2025

Contenido hoy

- Funcionamiento interno de lecturas y escrituras en base de datos
- Concurrencia
- Pessimistic Locking
- Optimistic Locking

Tabla lógica

emp_id	emp_name	emp_dob_month	emp_salary
10	Gonzalo	november	100
20	Don Ramon	december	200
30	Alf	january	300
...	...	...	...
9999	Superman	may	700

Row_ID

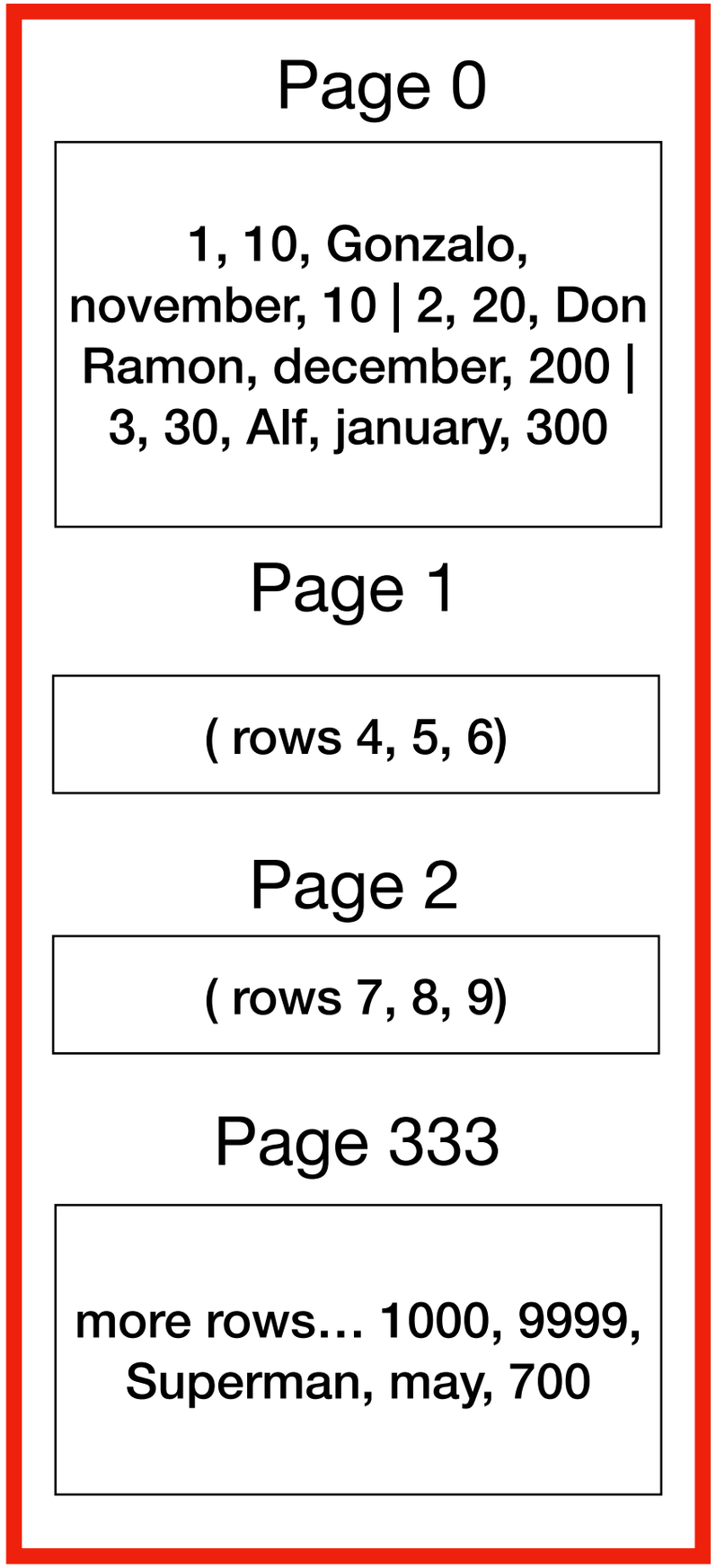
- Las bases de datos mantienen un sistema interno de rows.
- En algunas bases de datos (MySQL - InnoDB) esta fila (row) es la misma que la llave primaria (PK) pero en otros, como Postgres, tienen un campo tuple_id.

row_id	emp_id	emp_name	emp_dob_month	emp_salary
1	10	Gonzalo	november	100
2	20	Don Ramon	december	200
3	30	Alf	january	300
…	…	…	…	…
1000	9999	Superman	may	700

Page

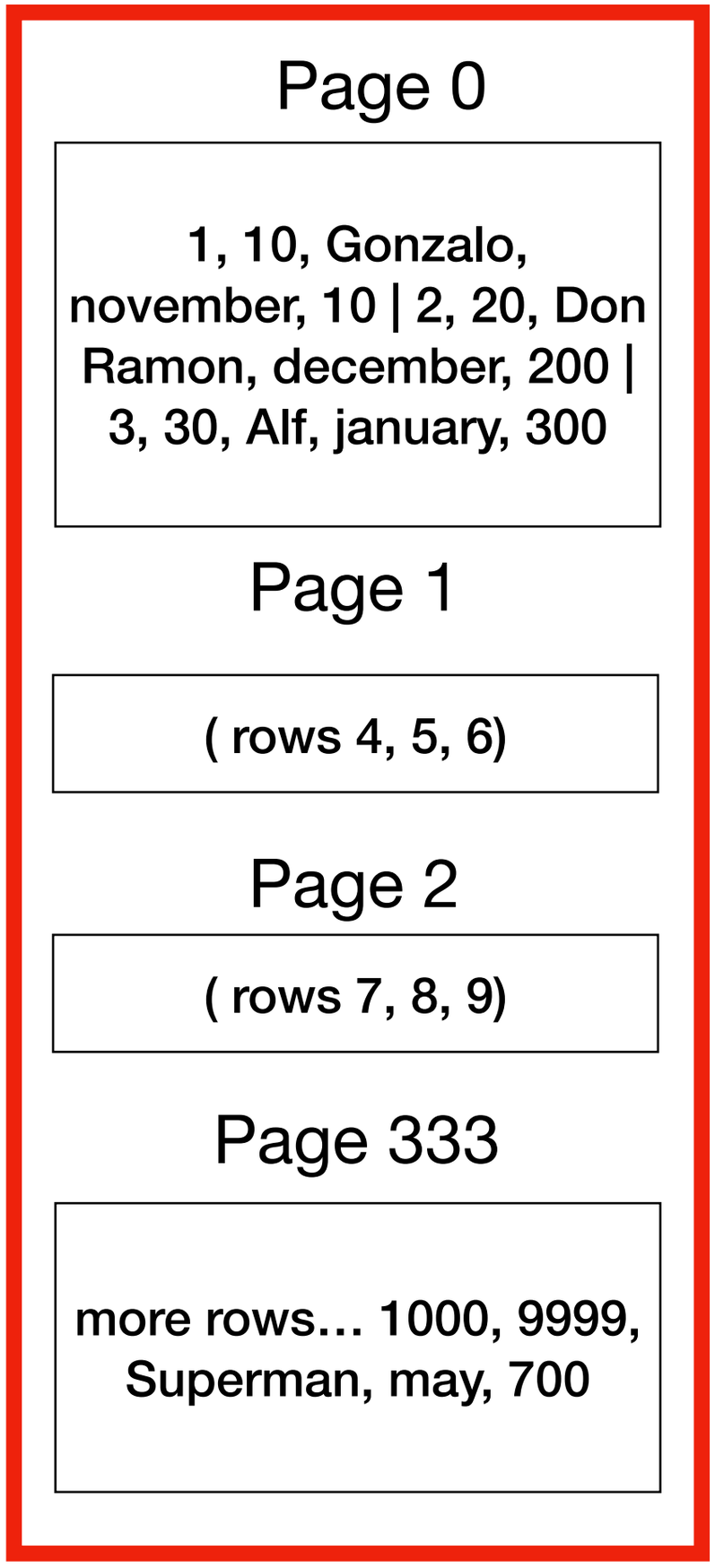
- Las filas son almacenadas en páginas (page).
- Una base de datos no lee una sola fila (row), lee un conjunto.
- Cada página tiene un tamaño (Postgres 8KB, MySQL 16 kb).

row_id	emp_id	emp_name	emp_dob_ month	emp_salary
1	10	Gonzalo	november	100
2	20	Don Ramon	december	200
3	30	Alf	january	300
…	…	…	…	…
1000	9999	Superman	may	700



Page

- Una operación IO (input/output) es una petición de lectura al disk (disk read).
- Una IO no puede sólo leer una fila (row), lee todo el contenido de la página.
- El objetivo es minimizar el IO porque es costoso.
- Algunas operaciones IO van hacia el disco o hacia el cache o hacia memoria ram (depende del cache del sistema operativo).

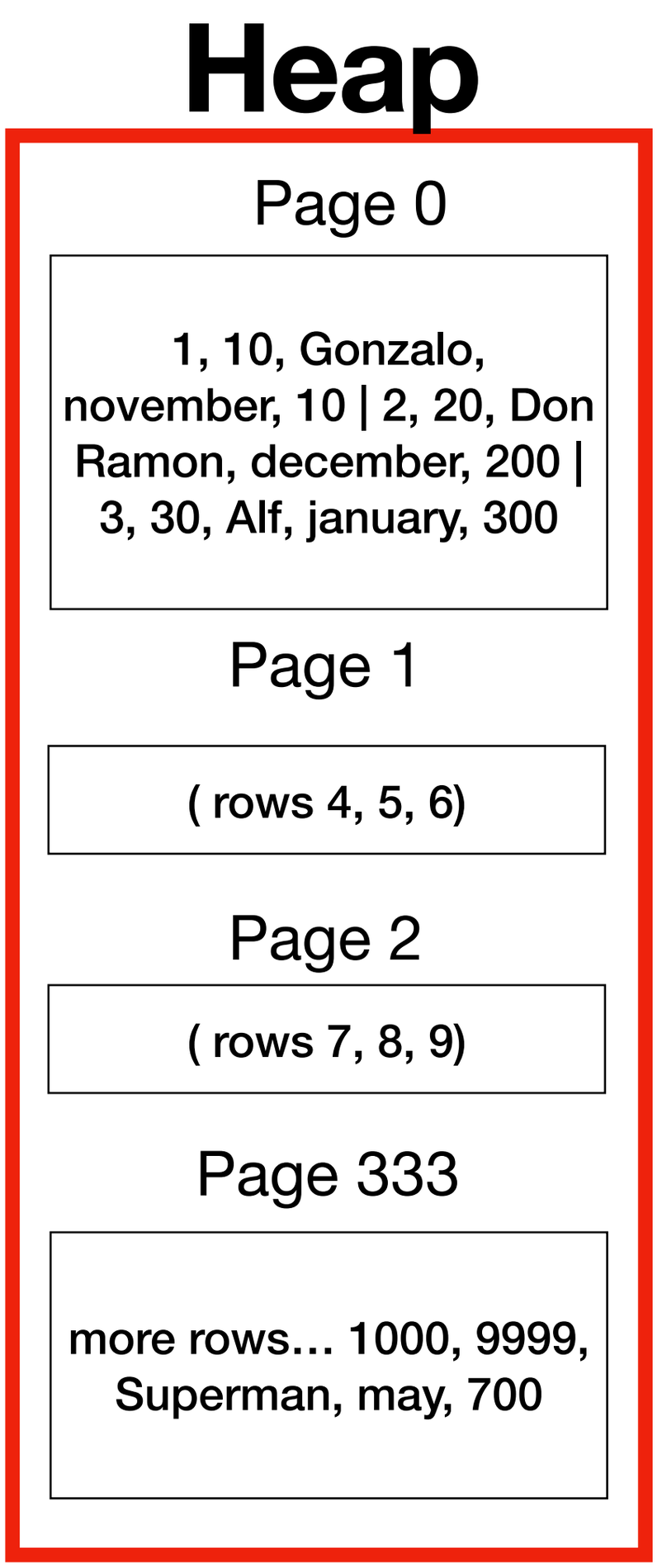


B Tree

Heap

- Heap es una estructura de datos que contiene todas las páginas (page) una tras otra.
- Acá es donde los datos se almacenan.
- Recorrer el Heap es costoso.
- Necesitamos un Indice (Index) que nos diga en que página (page) especifica se encuentra nuestra data.

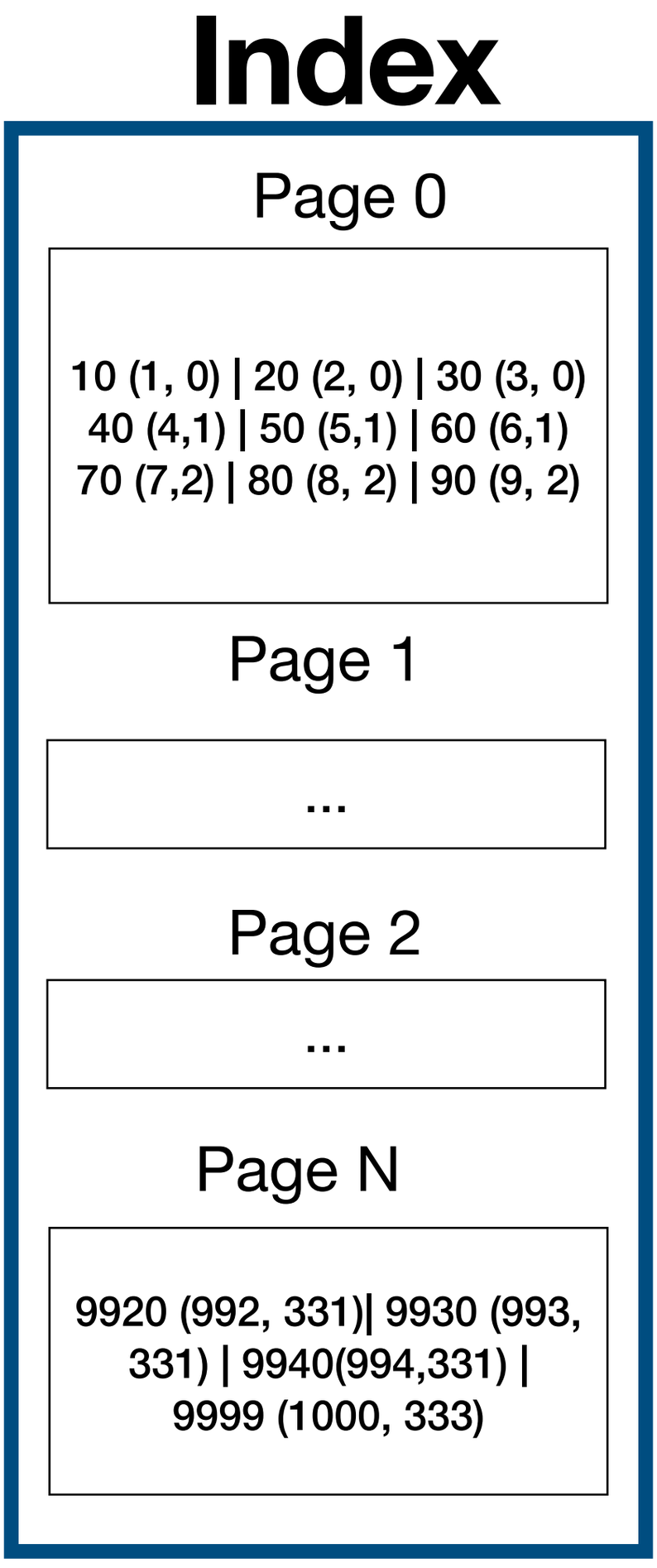
B Tree



Index

- Index es una estructura de datos separada del heap que tiene “punteros” hacia el heap.
- Una vez que se encuentra el valor en el indice (index) puede ir al heap a buscar datos.
- El índice también se almacena como pages.
- Popularmente el Index usa B-Trees.
- El Primery Index se origina mediante las Primary Keys (PK). Luego todos los otros indices que uno crea se llaman Secondary Index.

B Tree



Index on EMP_ID

IO1: se ejecuta en el indice para encontrar la page/ row

B Tree

Index

Page 0

10 (1, 0) | 20 (2, 0) | 30 (3, 0)
40 (4,1) | 50 (5,1) | 60 (6,1)
70 (7,2) | 80 (8, 2) | 90 (9, 2)

Page 1

...

Page 2

...

Page N

9920 (992, 331)| 9930 (993, 331) | 9940(994,331) | 9999 (1000, 333)

IO2: se ejecuta en el heap hacer pull (recuperar) la página encontrada en el indice

B Tree

Heap

Page 0

1, 10, Gonzalo, november, 10 | 2, 20, Don Ramon, december, 200 | 3, 30, Alf, january, 300

Page 1

(rows 4, 5, 6)

Page 2

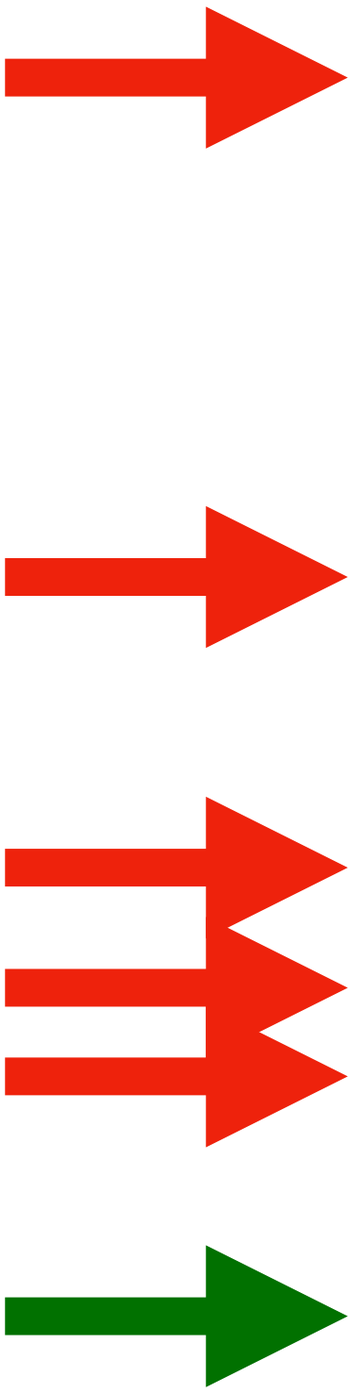
(rows 7, 8, 9)

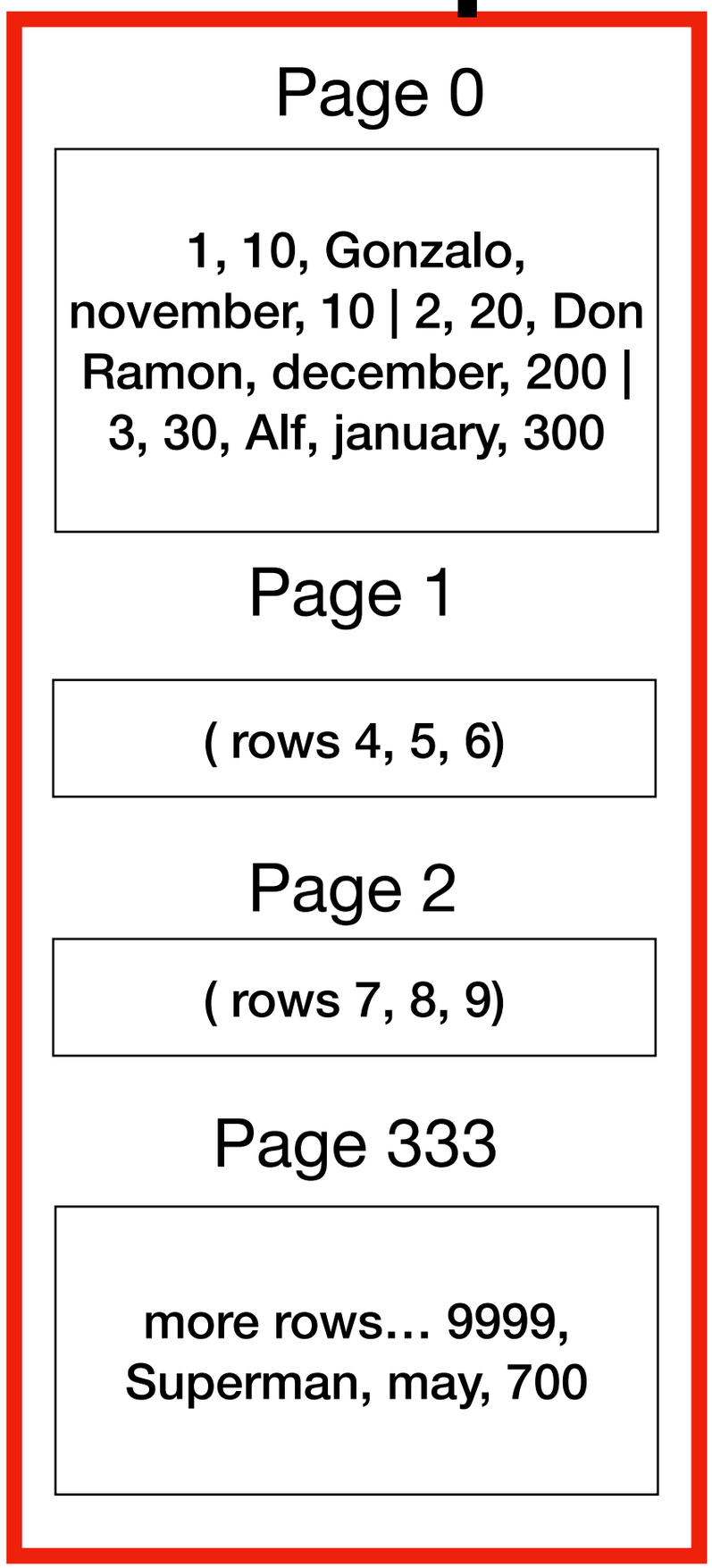
Page 333

more rows… 9999, Superman, may, 700

No Index -

SELECT * FROM EMP WHERE EMP_ID = 9999;





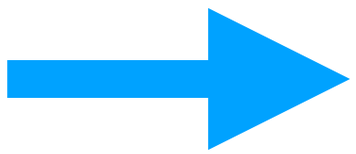
B Tree

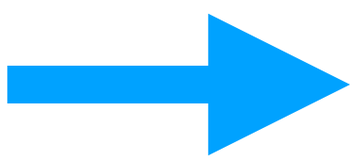
With Index -

SELECT * FROM EMP WHERE EMP_ID = 9999;

Index on EMP_ID







Index

Page 0

10 (1, 0) | 20 (2, 0) | 30 (3, 0)
40 (4,1) | 50 (5,1) | 60 (6,1)
70 (7,2) | 80 (8, 2) | 90 (9, 2)

Page 1

...

Page 2

...

Page N

9920 (992, 331)| 9930 (993, 331) | 9940(994,331) | 9999 (1000, 333)

O(log n)

IO1: se ejecuta en el indice para encontrar la page/ row

B Tree

9999 (9999, 333)

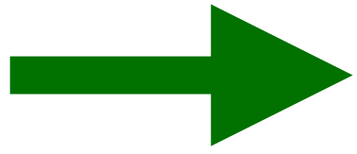
Fetch page 333, and pull row 9999

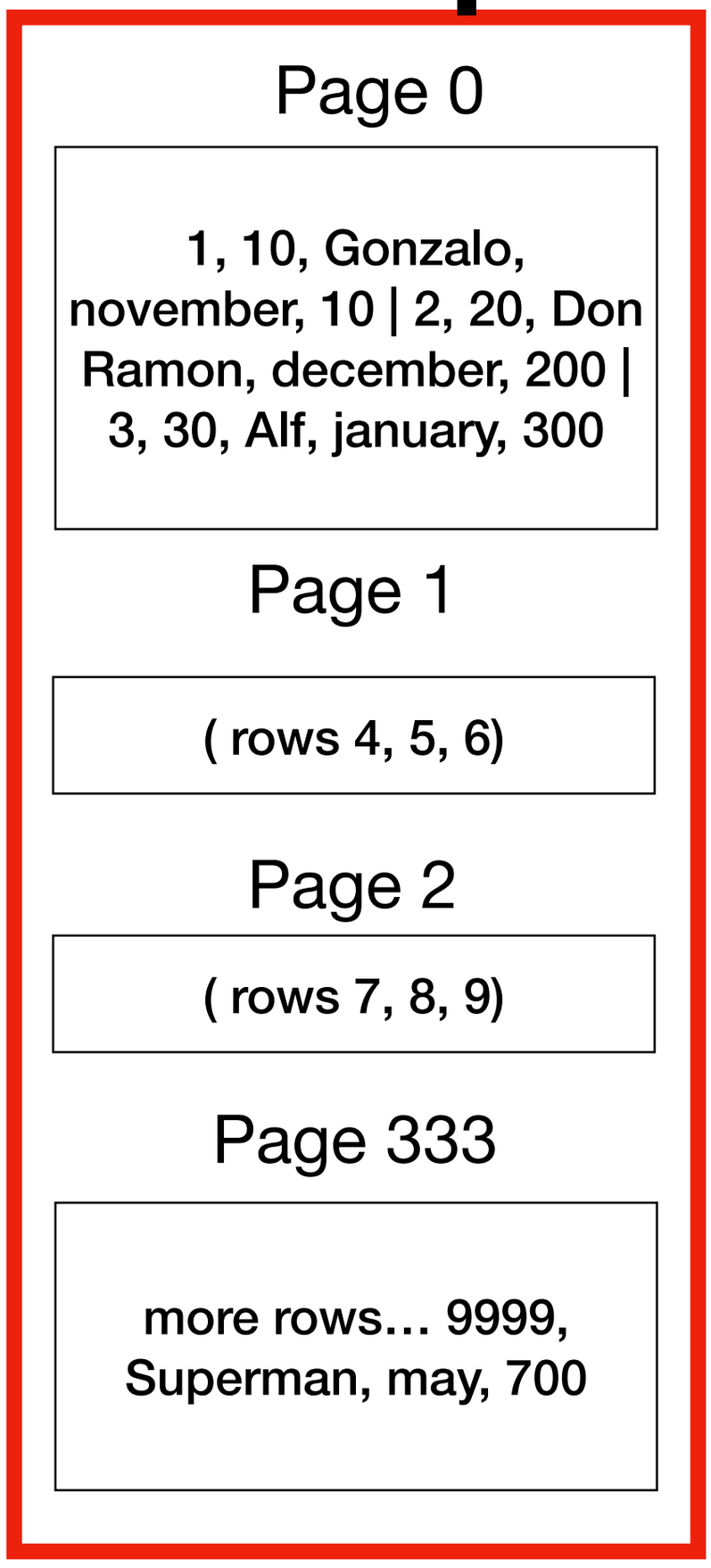
With Index -

SELECT * FROM EMP WHERE EMP_ID = 9999;

B Tree

IO2: se ejecuta en el heap hacer pull (recuperar) la página encontrada en el indice





También puedo crear índices usando atributos de la tabla.

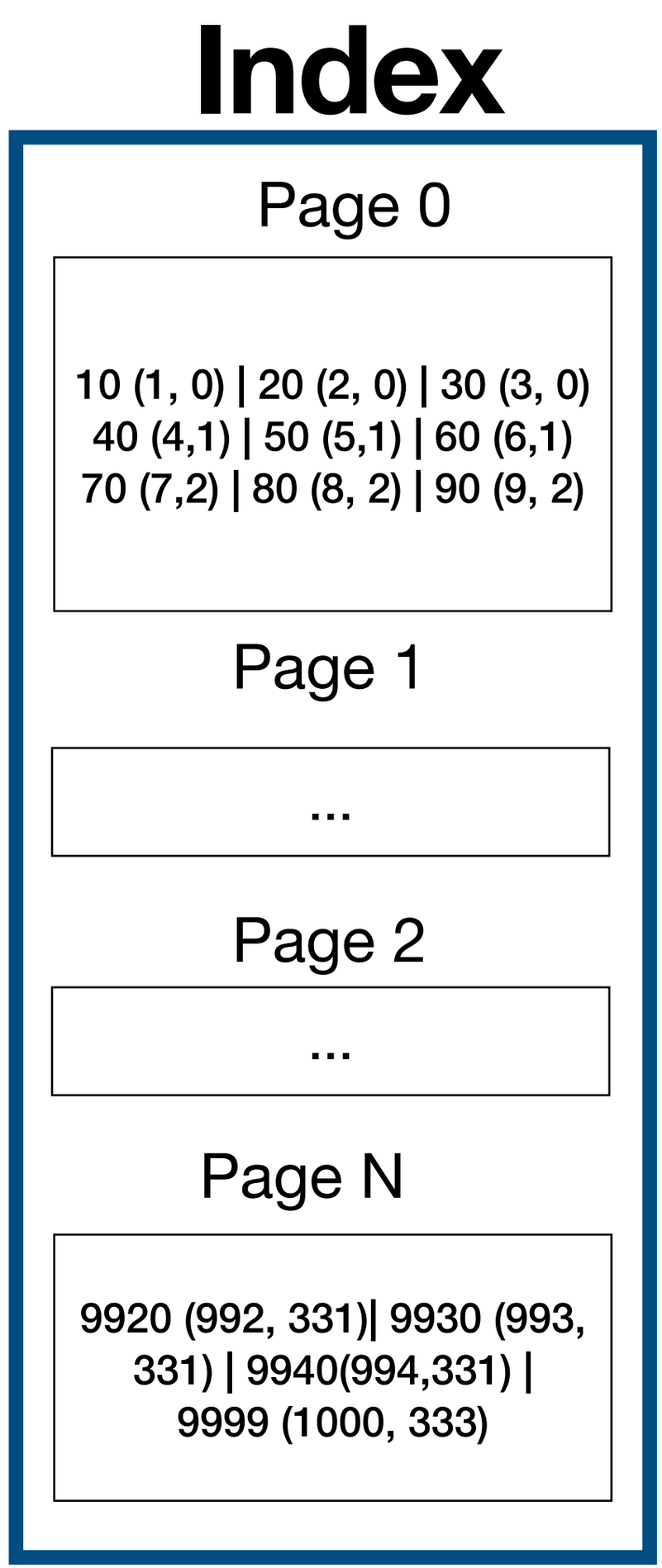
Para la creación de índices es necesario tener en cuenta el tamaño del índice y el espacio de RAM.

IO1: se ejecuta en el indice para encontrar la page/ row

B Tree

Index on EMP_ID

O(log n)



Ejemplo Práctico

Primary Key: Cómo seleccionar PK

RUT vs ID SERIAL vs UUID vs Otros métodos

- **Discusión y demostración en clases**
  - **Si es autoincremental, ¿qué sucede si la BD se cae? cómo considero las replicas y shards?**
    - **Si utilizan múltiples replicas de la BD o sharding, lo autoincremental ya no se puede tomar en cuenta, a menos que se haga una estrategia para esto.**
- **¿Qué sucede con el índice?**

Database High Concurreny

En escenarios de alta concurrencia debemos pensar en contención (contention) cuando tenemos escrituras

- Cuando ocurren casos de concurrencia (múltiples transacciones Tx intentan acceder al mismo tiempo a un mismo registro) ocurre Deadlock.

- Deadlock es una situación en donde dos o más procesos/ transacciones no pueden proceder debido a que cada una esta esperando a que la otra libere los recursos que se encuentra utilizando. Esto provoca una dependencia circular donde nadie puede continuar.

- Existen dos formas para lidiar con la concurrencia: Pessimistic Locking y Optimistic Locking.

- Por lo general, las BD relacionales (MySQL, PostgreSQL, Oracle) proveen mecanismos para Pessimistic Locking.

- BD documentales por lo general utilizan Optimistic Locking.

Pessimistic Locking: FOR UPDATE

MySQL >= 8.0
PostgreSQL

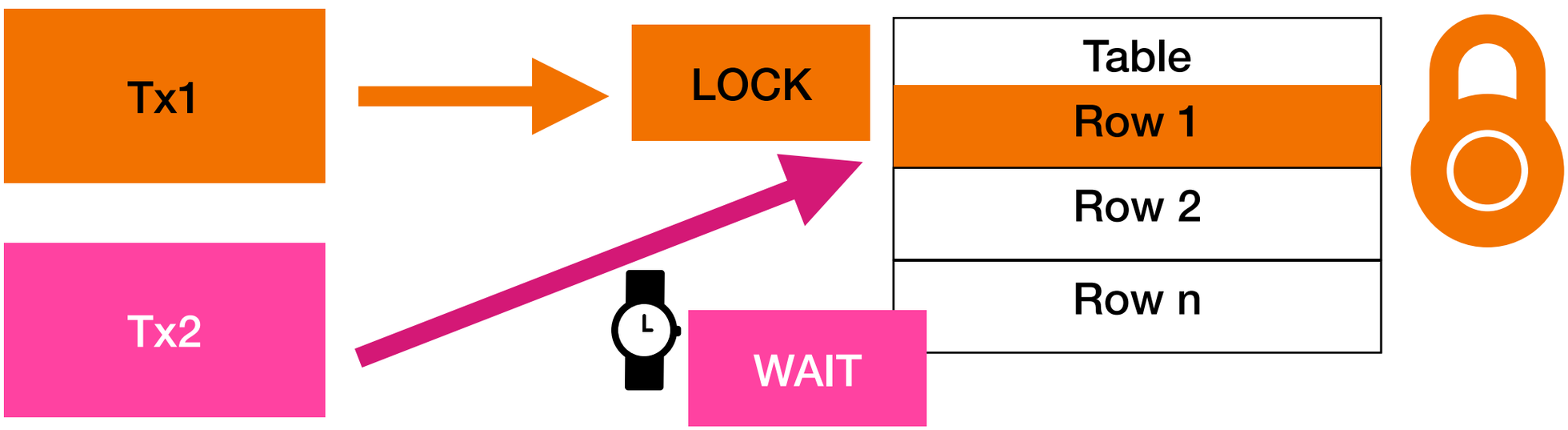
```
SELECT id, seat_number FROM seats

WHERE user_id IS NULL

ORDER BY id LIMIT 1
```

FOR UPDATE

- It **locks** the selected row(s) for the duration of the current transaction.
- **Prevents** other transactions from modifying these rows **until the current transaction is completed** (either committed or rolled back).



Tx1 tiene el lock en Row1 y nadie más puede modificarlo hasta que la transacción de Tx1 se complete (commit or rollback)

Tx2 espera hasta que LOCK se libere o ocurra TIMEOUT

Pessimistic Locking: FOR UPDATE SKIP LOCKED

MySQL >= 8.0
PostgreSQL

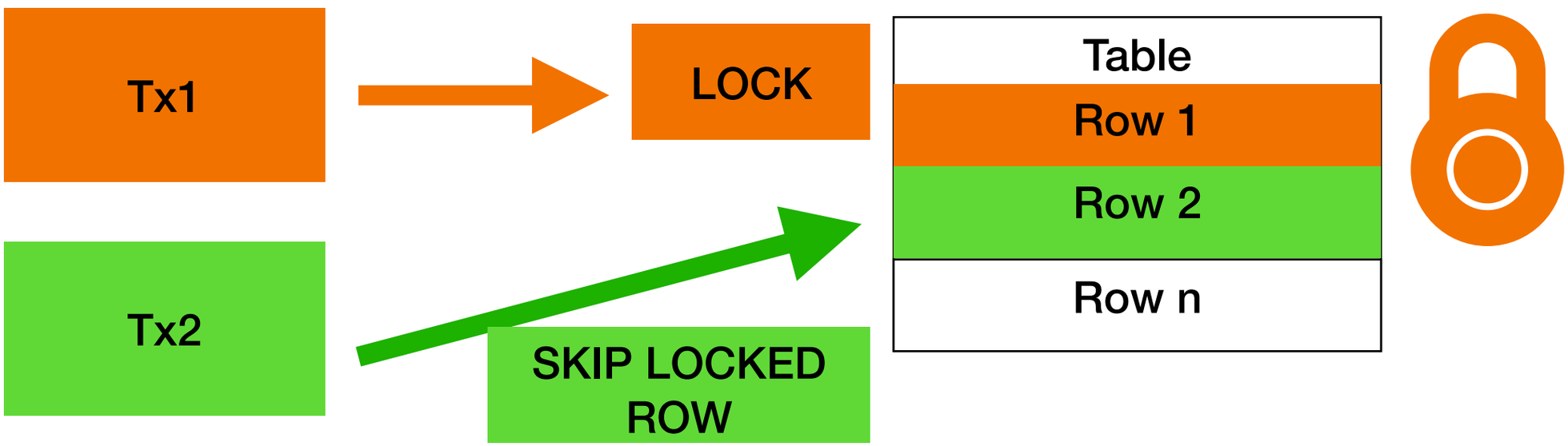
```
SELECT id, seat_number FROM seats

WHERE user_id IS NULL

ORDER BY id LIMIT 1
```

FOR UPDATE SKIP LOCKED

- It attempts to **lock** the first available row as before.
- If the row is already locked by another transaction, instead of waiting, it simply **skips that row and moves on to the next one**.
- **This process continues until it finds an unlocked row or exhausts all possible rows.**



Tx1 tiene el lock en Row1 y nadie más puede modificarlo hasta que la transacción de Tx1 se complete (commit or rollback)

Tx2 AVANZA HASTA LA SIGUIENTE ROW hasta encontrar un ROW disponible (no lock).

Pessimistic Locking: FOR UPDATE NO WAIT

MySQL >= 8.0
PostgreSQL

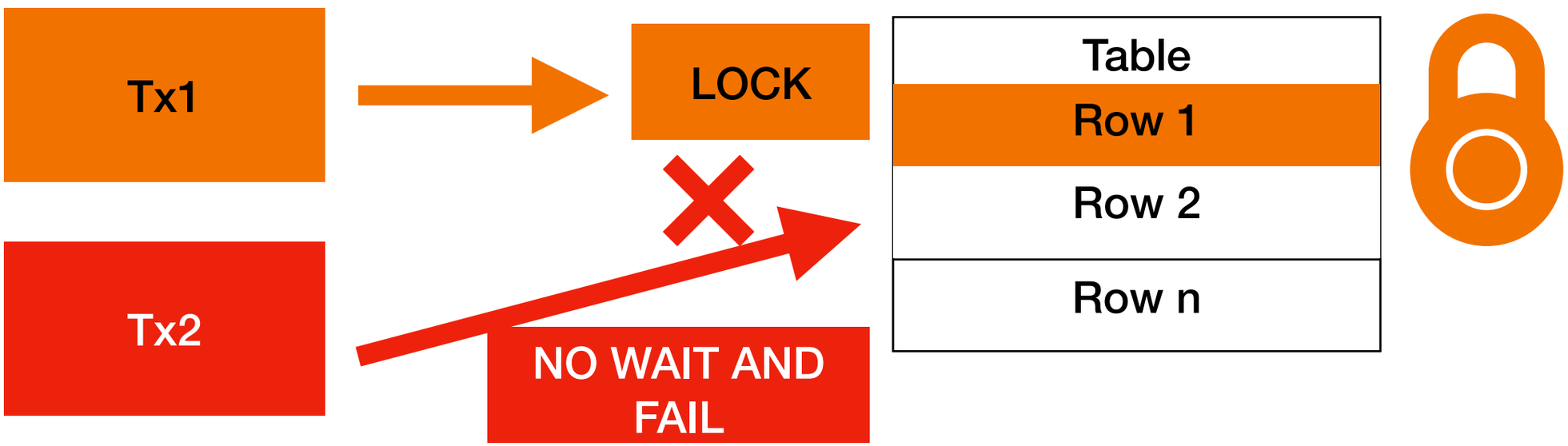
```
SELECT id, seat_number FROM seats

WHERE user_id IS NULL

ORDER BY id LIMIT 1
```

- It attempts to **lock** the first available row as before.
- **If the row is already locked by another transaction, instead of waiting, it immediately throws an error.**

FOR UPDATE NO WAIT



Tx1 tiene el lock en Row1 y nadie más puede modificarlo hasta que la transacción de Tx1 se complete (commit or rollback)

Tx2 NO ESPERA Y FALLA.

Finalmente

Cómo diseñar sistemas

- Antes de seleccionar cualquier tecnología (base de datos, entre otros) o diseño o patrón de arquitectura:
    - Entiendan que realizará el usuario y cómo garantizo la calidad del producto al usuario (downtime minimo, entre otros).
    - Entender las acciones del usuario.
        - ¿**Qué datos** voy a almacenar?
        - ¿**Cuánta data** voy a almacenar?
        - **¿Cómo voy a acceder a esos datos?**
        - ¿Qué **tipo de consultas** realizaré?
        - ¿Cualquier **feature extra** (*time to live, expiration*) ?

Sean pragmáticos:
- Hecho es mejor que perfecto
- Piensen en el usuario, ante cualquier decisión de diseño debemos priorizar la calidad de servicio